PID-RS TECHNICAL REPORT SERIES

# Mathematical Problem-Solving Workflow

## Full-fidelity process protocol for `pid-rs`

Sepehr Mahmoudian

1 September 2026; revised 26 October 2026

**Abstract**

This document is the human-readable typeset rendition of the canonical Markdown file `MATHEMATICAL_PROBLEM_SOLVING_WORKFLOW.md`. It records external process observations, then defines the complete project protocol for exact claims, separately developed routes with explicit independence accounting, retained failures, certificate conversion, adversarial audit, blind holdouts, PID-specific exceptional cases, Codex goal and continuity control, a future typed publication harness, layered assurance, branch/worktree lifecycle, durable storage, and claim disposition. The protocol is project-defined guidance. It is not a PID estimator, theorem, benchmark result, or validation claim; the harness described here is a specification, not an implemented system.

## Contents

**1 Reader's primer: objects, logic, and evidence boundaries** **4**
1.1 Status of this primer . . . 4
1.2 Four objects that must not be conflated . . . 4
1.3 Exact claims, quantifiers, and counterexamples . . . 5
1.3.1 Advanced definition . . . 6
1.4 Certificates and separately specified checking . . . 6
1.5 PID vocabulary and a finite worked reconstruction . . . 7
1.6 Negative example: an unseen rare cell . . . 7
1.7 Negative example: pairwise is not mutual independence . . . 8
1.8 Bounded evolutionary search: useful scope and hard boundary . . . 8

**2 Typed assurance and rigorous evidence aggregation** **8**
2.1 Status of this supplement . . . 8
2.2 Preserve a vector across assurance layers . . . 8
2.3 Critical cut sets determine dependence . . . 9
2.4 Worked example: five correlated test families are one confirmation at a shared oracle . . . 9
2.5 When a model-conditional posterior belief is permitted . . . 11
2.6 Change invalidation and publication state . . . 11

**3 Mathematical problem-solving workflow for pid-rs** **12**
3.1 Status and scope . . . 12
3.2 Compact glossary . . . 13
3.3 Primary-source pins for PID vocabulary and routing . . . 16

3.4 Source observations . . . 17
3.4.1 Erdős-problem research thread . . . 17
3.4.2 Lea launch thread and source audit: a proof checker is not a trusted problem setter 17
3.4.3 Grothendieck-constant case study: discovery, certification, and state loss . . . 19
3.4.3.1 Twenty-lens joint disposition . . . 20
3.4.4 DeepSeek Harness, Cordis, and category theory: explicit mapping checks, not branding 21
3.4.4.1 Twenty-lens adoption disposition . . . 23
3.4.5 Cycle Double Cover prompt . . . 24
3.4.6 Erdős repository . . . 25
3.4.7 Erdős 477 cubic repository . . . 25
3.4.8 Corrected vector-bundle claim: a typed citation-edge failure . . . 26
3.4.9 External AI discussions . . . 28
3.4.10 OpenAI-hosted proof-process walkthroughs: process controls only . . . 29
3.4.11 Zeta two-thirds source review: mathematics, methods, process, and the current PID no-direct-transfer disposition . . . 31
3.4.11.1 Mathematical method reconstructed from the paper . . . 34
3.4.11.2 Discovery, verification, and communication methods . . . 36
3.4.11.3 Thirty-four-lens PID transfer audit . . . 39
3.5 AI model operating protocol . . . 43
3.5.1 Applicability and risk . . . 43
3.5.2 Candidate/judge isolation and research-integrity controls . . . 43
3.5.3 Problem-specific autoresearch and durable publication . . . 44
3.5.4 Why branches and worktrees exist, and why they create a closure obligation . . . 46
3.5.4.1 Failure-to-repair map . . . 46
3.5.4.2 Exact closure sequence . . . 48
3.5.4.3 Formal verification inside an isolated lane . . . 49
3.5.4.4 Agent and council ownership . . . 50
3.5.5 Council protocol . . . 51
3.5.6 Separate roles . . . 52
3.5.7 Review-joint matrix and saturation handoff . . . 53
3.5.8 Independence vector . . . 53
3.5.9 Separately developed routes and independence accounting . . . 54
3.5.10 Critical-cut-set accounting . . . 54
3.5.11 Frozen run context . . . 55
3.5.12 Freeze purpose and identity boundary . . . 56
3.5.13 Durable agent continuity . . . 57
3.5.14 Codex goal, plan, and tool lifecycle . . . 58
3.5.15 Tool-call and agent receipts . . . 59
3.5.16 Publication-harness state machine . . . 60
3.5.17 Evidence labels and route memos . . . 61
3.6 pid-rs protocol . . . 62
3.6.1 1. Write an exact-claim packet . . . 62
3.6.1.1 Why scientific results must be typed . . . 64
3.6.1.2 Historical pid-rs/Wibral-program inventory snapshot (15 August 2026) . 65
3.6.2 2. Build an obligation graph . . . 67
3.6.2.1 Optional explicit mapping-check rule . . . 67
3.6.3 3. Keep a separate-approach registry . . . 68
3.6.4 4. Retain blockers and failed routes . . . 68
3.6.5 Load-bearing correction ledger . . . 69
3.6.6 5. Convert computation into certificates . . . 69
3.6.7 6. Run the required 20-core/50-lens adversarial audit . . . 70
3.6.7.1 Lenses 1–10: scientific object and inference contract . . . 70
3.6.7.2 Lenses 11–20: evidence, custody, implementation, and release contract . . 71
3.6.7.3 SxPID definition-compatibility firewall . . . 72
3.6.7.4 Imported-theorem application map . . . 74
3.6.7.5 Citation-edge type check . . . 74
3.6.7.6 Cheap invariant probes . . . 75
3.6.8 7. Use a blind holdout protocol . . . 76

3.6.8.1 Minimum blind-benchmark commitment . . . 77
3.6.9 8. Maintain a claim-to-evidence matrix . . . 78
3.7 Application to shared-exclusions PID . . . 79
3.7.1 Exact claim packet for a new PID theorem . . . 79
3.7.2 PID routing checkpoints . . . 79
3.7.3 Complementary PID audit families . . . 81
3.7.4 Required counterexample search . . . 82
3.7.5 Formal and software evidence . . . 82
3.7.6 Statistical and ecosystem evidence . . . 84
3.8 Full semantic closure . . . 85
3.9 PID exceptional-case checklist . . . 85
3.10 Layered assurance and go/no-go gates . . . 86
3.11 Acceptance rule . . . 87

# 1 Reader's primer: objects, logic, and evidence boundaries

## 1.1 Status of this primer

This section is a typeset-only explanatory expansion. It introduces the vocabulary and works through examples needed to apply the protocol without assuming prior knowledge of formal methods, statistics, or partial information decomposition (PID). It does not change the canonical protocol reproduced in full beginning with Section 3. Every example below is project-defined process guidance, not a new PID theorem or estimator result.

## 1.2 Four objects that must not be conflated

Let $P$ denote a population law and let $F(P)$ denote a mathematical information functional. Given rows $Z_1, \ldots, Z_n$, an estimator is an algorithmic rule

$$\hat{F}_n = A_n(Z_1, \ldots, Z_n).$$

A particular program returns a represented number

$$\tilde{F}_n = \mathrm{Program}_{\mathrm{compiler,features,platform}}(Z_1, \ldots, Z_n),$$

and a consumer may use that number, a typed diagnostic record $U_n$, and a policy $\mathcal{D}$ to make a decision

$$d_n = \mathcal{D}(\tilde{F}_n, U_n).$$

Here $\mathcal{D}$ has a declared domain consisting of the represented-result and diagnostic-record types and maps them into a declared decision space containing $d_n$. These are four different objects: the population functional, the estimator, the executable result, and the consumer decision. A proof that $F$ is continuous establishes no finite-sample error rate for $\hat{F}_n$. An estimator theorem does not prove that the executable implements the estimator. Executable conformance does not establish that a downstream decision is safe. The assurance path therefore contains three distinct transitions, each with its own obligation:

$$\boxed{\text{definition}} \longrightarrow \boxed{\text{estimator theorem}} \longrightarrow \boxed{\text{executable conformance}} \longrightarrow \boxed{\text{consumer qualification}}.$$

Each arrow is a separate obligation. Evidence for a box cannot silently replace evidence for an arrow.

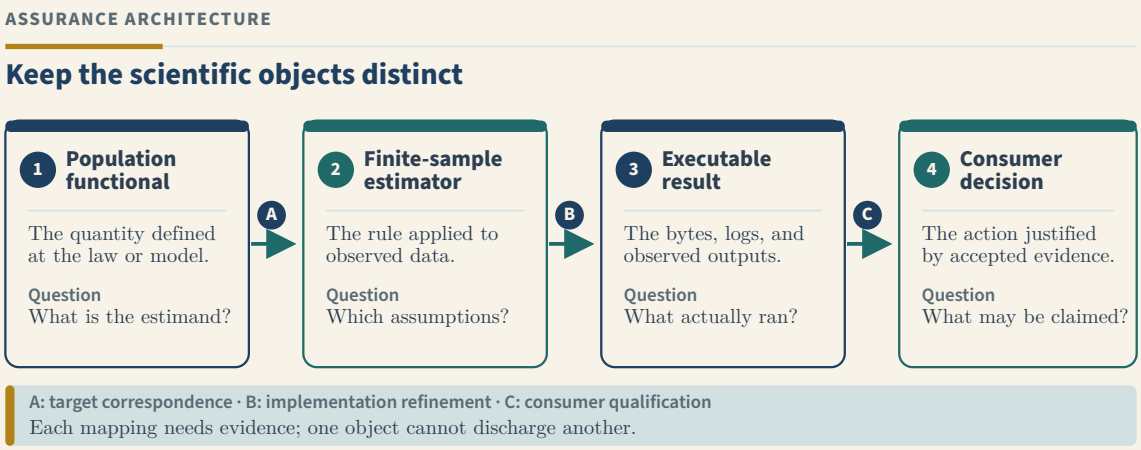



**Figure 1:** The assurance chain has four distinct objects and three separately justified assurance transitions.

## 1.3 Exact claims, quantifiers, and counterexamples

An exact mathematical claim should be reducible to a proposition such as

$$\forall x \in \mathcal{X}, \qquad A_1(x) \wedge \cdots \wedge A_k(x) \Longrightarrow C(x), \tag{1}$$

where $\mathcal{X}$ is the domain, $k \geq 1$, $A_i$ are assumptions, and $C$ is the conclusion. A counterexample is one explicit $x_\star \in \mathcal{X}$ for which every $A_i(x_\star)$ is true but $C(x_\star)$ is false. An example outside $\mathcal{X}$, or one that violates an assumption, does not refute (1); it may instead show that the assumption is necessary.

Quantifier order and convergence mode are part of the claim. For example, pointwise convergence in probability over a declared class $\mathcal{P}$ has the pattern

$$\forall P \in \mathcal{P}\ \forall \varepsilon, \eta > 0\ \exists N(P, \varepsilon, \eta) \in \mathbb{N}\ \forall n \geq N(P, \varepsilon, \eta) : \Pr_P\left\{\left|\hat{F}_n - F(P)\right| > \varepsilon\right\} < \eta. \tag{2}$$

A uniform-in-$P$ statement would require an $N(\varepsilon, \eta)$ that works for every law in $\mathcal{P}$. Equation (2) does not imply that stronger statement. Likewise, $\forall \varepsilon\, \exists N$ cannot be replaced with $\exists N \forall \varepsilon$. The claim packet records this order before proof search so that a correct proof of a weaker rearrangement is not mistaken for completion.

Start with two routes to one claim. Route A needs premises A1, A2, and C. Route B needs premises B1 and C. A route needs all its premises: this is AND. Either fully justified route can support the claim: this is OR. Each route also needs evidence that its premises imply the claim. The diagram assumes that these route obligations are accepted.

Both routes depend on C. If C lacks accepted evidence, neither route is accepted. A shared cut is a set of allowed dependency nodes that meets every route. A cut is inclusion-minimal if no proper subset still meets every route. Here the three minimal cuts are $\{C\}$, $\{A1, B1\}$, and $\{A2, B1\}$. The figure fixes which nodes are allowed in a cut. The GO label states the illustrated gate state; it does not state a new mathematical result.

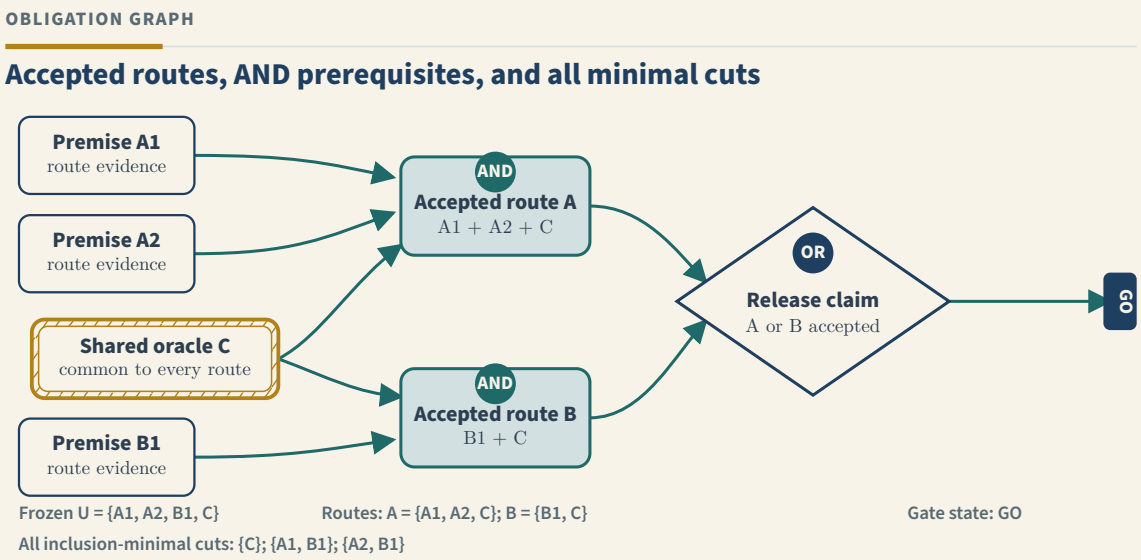



**Figure 2:** Route A needs A1, A2, and C; route B needs B1 and C. The frozen example has three inclusion-minimal cuts. All shown routes are accepted; GO names only the illustrated terminal gate state.

### 1.3.1 Advanced definition

An obligation structure is an AND/OR directed acyclic hypergraph. Its nodes are propositions or finite checks. An AND-hyperedge $\{u_1, \dots, u_r\} \to v$ records that all named premises are needed to derive $v$; alternative incoming hyperedges are OR-routes, any one of which may derive $v$. A leaf closes only through accepted evidence of the declared class. An internal node closes only when every tail node of at least one accepted incoming hyperedge is closed and the edge's implication has itself been justified. Define each route as a set of dependency nodes in a frozen admissible vertex universe. Unless a deliberately trivial cut is reported separately, that universe excludes the common claim/goal node and synthetic route or AND/OR aggregator nodes. The packet retains every minimal shared cut set: each inclusion-minimal admissible node set that intersects every route in a declared route family. The packet says whether that family is candidate, permitted, or accepted and tags every cut node as open, closed, or a known common cause. An accepted route cannot contain an open node; a cut over accepted routes may nevertheless expose a closed shared dependency. Rephrasing an open node, or closing one branch of an AND-edge, is not closure.

## 1.4 Certificates and separately specified checking

Exploratory floating-point output can suggest a lemma but is not a proof. A certificate separates generation from checking:

$$\text{instance} \xrightarrow{\text{generator}} \text{certificate} \xrightarrow{\text{separately specified checker}} \{\text{accept}, \text{reject}\}.$$

The checker must state the finite proposition that acceptance establishes. For rational inputs, integer and rational identities should remain exact. Transcendental terms require either a symbolic identity or outward-rounded intervals. If $L \le x \le U$ and a sign is needed, then

$$L > 0 \Longrightarrow x > 0, \qquad U < 0 \Longrightarrow x < 0, \qquad 0 \in [L, U] \Longrightarrow \text{unresolved}.$$

The last branch is not a failure; it is the correct abstention. A useful mutation test changes a sign, endpoint, mask, or denominator in a copy of the evidence. The checker must reject the mutant for the intended reason.

## 1.5 PID vocabulary and a finite worked reconstruction

PID is a family of proposals for decomposing how several sources $S_1, \dots, S_m$ provide information about a target $T$. This worked reconstruction concerns a finite redundancy-lattice PID after its cumulative functional and order have been declared; it does not assert that every PID proposal uses this carrier or semantics. For finite variables, a joint law $P$ assigns nonnegative masses summing to one. A source collection is a nonempty subset of $\{1, \dots, m\}$. An antichain is a nonempty set of source collections in which no member contains another. The declared cumulative quantities $c_\alpha$, indexed by antichains, are related to atoms $\pi_\beta$ by a finite zeta transform. The orientation below is part of the definition: $\beta \preceq \alpha$ means that atom $\pi_\beta$ contributes to cumulative $c_\alpha$.

$$c_\alpha = \sum_{\beta \preceq \alpha} \pi_\beta, \qquad \pi_\alpha = \sum_{\beta \preceq \alpha} \mu(\beta, \alpha) c_\beta, \tag{3}$$

where $\preceq$ is the declared redundancy order and $\mu$ is its Möbius function. Equation (3) is finite algebra. It does not by itself prove that the cumulative values have the intended paper semantics or that floating-point code evaluates them accurately.

For the ordinary two-source four-atom shape, let $R$, $U_1$, $U_2$, and $S$ denote redundancy, source-specific unique terms, and synergy. Let $c_R$ be the redundancy-node cumulative, $c_1$ and $c_2$ the two single-source cumulatives, and $c_{12}$ the joint-source cumulative for the already declared functional. Inversion is

$$\begin{aligned} R &= c_R, \\ U_1 &= c_1 - R, \\ U_2 &= c_2 - R, \\ S &= c_{12} - R - U_1 - U_2. \end{aligned}$$

Substitution gives the reconstruction identities

$$c_1 = R + U_1, \qquad c_2 = R + U_2, \qquad c_{12} = R + U_1 + U_2 + S. \tag{4}$$

An exact checker can verify (4) for every declared finite count table. That closes the finite reconstruction obligation only and proves no sign theorem. For the categorical Makkeh–Gutknecht–Wibral shared-exclusions functional specifically, the exact-real informative and misinformative component atoms are separately nonnegative at the defined positive-mass pointwise realizations, and their finite-PMF averages inherit nonnegativity, while the corresponding signed nets may be negative (Makkeh et al., Theorem IV.3). pid-rs therefore does not clamp those signed-net atoms. Williams–Beer $I_{\min}$ instead has its own exact-real finite-PMF nonnegativity theorem for its atoms; a materially negative value from that pid-rs plug-in route is an implementation defect, not an MGW-style signed-net interpretation. Binary64 roundoff remains a separate bounded numerical obligation in both cases. Neither sign result is a property of Möbius inversion itself or transfers to Ehrlich continuous shared-exclusions/PID2 estimates, KSG mutual-information estimates, project heuristics, or an arbitrary PID functional.

## 1.6 Negative example: an unseen rare cell

Let a finite population contain a cell $z_\star$ with mass $P(z_\star) = \rho > 0$. Under i.i.d. sampling from $P$, the exact probability that $z_\star$ is absent from $n$ rows is

$$\Pr\left\{\hat{P}_n(z_\star) = 0\right\} = (1 - \rho)^n. \tag{5}$$

With $\rho = 10^{-4}$ and $n = 1000$, this probability is approximately 0.9048. Thus an observed support can omit a genuine population cell with high probability. Replacing an unknown population

mass floor with the smallest observed mass is therefore invalid without another assumption or confidence construction. Fixed-support continuity remains useful, but it is not a support-discovery theorem or a practical finite-sample certificate.

### 1.7 Negative example: pairwise is not mutual independence

Let $X$ and $Y$ be independent fair bits and set $Z = X \operatorname{xor} Y$. Every pair among $X, Y, Z$ is independent, but the triple is not: $Z$ is determined by $X, Y$. Therefore pairwise independence, zero covariance, or a benign autocorrelation plot cannot establish mutual independence of complete rows within a dependence color. A concentration result whose premise is mutual within-color independence needs that premise justified by design or a separate theorem.

### 1.8 Bounded evolutionary search: useful scope and hard boundary

Genetic or evolutionary search can be useful when fitness is mechanically checkable. For a mutation $m$ of a proof script, certificate, or implementation, a search may rank

$$f(m) = (\mathbf{1}\{\text{invalid mutant is accepted}\}, -\text{witness size}, -\text{runtime})$$

lexicographically, with larger tuples preferred. Thus an accepted invalid mutant outranks every rejected mutant, and the negative signs prefer smaller witnesses and shorter runtimes within the same acceptance class. Mutation operators may swap a source mask, replace a union with an intersection, reverse a log ratio, perturb an interval endpoint, or delete one Möbius predecessor. Any accepted invalid mutant is a concrete checker weakness and must be retained and minimized.

Failure to find such a mutant is not a proof. An exhaustive finite mutation domain $\mathcal{M}_B$ can support only the bounded statement

$$\forall m \in \mathcal{M}_B, \qquad \text{Checker}(m) = \text{reject}\,.$$

A heuristic genetic run samples only part of its search space. Its valid output is a discovered counterexample or a diagnostic search log, never a theorem-vote or a confidence score.

## 2 Typed assurance and rigorous evidence aggregation

### 2.1 Status of this supplement

This is a typeset-only explanatory supplement to the canonical protocol in Section 3. It specifies how evidence records may be aggregated without turning heterogeneous proof, numerical, software, statistical, and consumer evidence into an unsupported score. It is process guidance, not a calibrated confidence model for `pid-rs`.

### 2.2 Preserve a vector across assurance layers

Evidence has different semantics at different layers. Preserve a typed vector

$$\mathcal{A}(H) = (E_{\text{sem}}, E_{\text{math}}, E_{\text{formal}}, E_{\text{num}}, E_{\text{exec}}, E_{\text{stat}}, E_{\text{consumer}}, E_{\text{archive}}) \tag{6}$$

for a fully written proposition $H$. Each component is itself a record

$$E_\ell = (\text{state}, \text{scope}, \text{assumptions}, \text{artifacts}, \text{negative evidence}, \text{residual boundary}). \tag{7}$$

The component *assurance-component closure state* is one of `not-applicable`, `open`, `rejected`, or `closed`. It is distinct from the canonical artifact-verification label, from the accepted evidence class (for example, theorem, finite machine check, certified bound, empirical result, counterexample, or diagnostic), and from the claim disposition (`proposed`, `active`, `blocked`, `falsified`, or `complete`). Multiple evidence records may support one component; one claim has one current disposition.

The vector and gate map is:

| Component | Gate(s) | Required subject |
|---|---|---|
| $E_{\text{sem}}$ | G0–G1 | claim identity, conventions, and premises |
| $E_{\text{math}}$ | G2 | mathematical proof or counterexample |
| $E_{\text{formal}}$ | G3 | proof-checker semantics and implication |
| $E_{\text{num}}$ | G4 | rigorous numerical enclosure |
| $E_{\text{exec}}$ | G5 | executable conformance |
| $E_{\text{stat}}$ | G6 | scoped estimator calibration |
| $E_{\text{consumer}}$ | G7 | consumer contract and qualification |
| $E_{\text{archive}}$ | G8 | reproducible evidence and release archive |

A machine-checked lattice identity and an open calibration study are not numbers on one common scale. Do not map their labels to integers and average them. An average would erase the fact that one load-bearing layer is open. The decision rule is instead proposition- and layer-specific: every applicable required component must meet its stated closure condition, and every inapplicable component needs a recorded reason.

## 2.3 Critical cut sets determine dependence

Let test-result random variables $T_1, \dots, T_k$ bear on a binary proposition $H$, let $t_1, \dots, t_k$ denote one possible result tuple, and let $C$ range over a fully defined finite set of latent shared-failure states for an oracle, imported theorem, generator, or implementation. Different test names do not make them independent. For each $B \in \{H, \neg H\}$, a complete joint evidence model must retain the shared state:

$$\Pr(T_1 = t_1, \dots, T_k = t_k \mid B) = \sum_c \Pr(C = c \mid B)\ \Pr(T_1 = t_1, \dots, T_k = t_k \mid B, C = c). \tag{8}$$

Multiplying marginal pass probabilities is justified only after an evidence-dependence model establishes the required conditional independence under both $H$ and $\neg H$. A critical-cut-set memo names each shared node, which routes use it, and which dependency-disjoint evidence—if any—closes it. If the latent failure-state space omits an identified common cause, the numerical model is incomplete and the scalar route must abstain.

## 2.4 Worked example: five correlated test families are one confirmation at a shared oracle

Suppose five suites exercise row permutations, source permutations, small count tables, canonical logic gates, and randomized tables. All five compare the Rust result with the same reference oracle. Let $D$ mean that the implementation has a particular semantic defect, and let $O$ mean that the shared oracle also contains the same defect. Conditional on $D \wedge O$, every suite can pass:

$$\Pr(T_1 = \cdots = T_5 = \text{pass} \mid D \wedge O) = 1. \tag{9}$$

They are useful for code paths and transformations outside the shared defect, but at the oracle cut set they are one correlated route.

For scale only, consider a hypothetical, fully specified model in which $\Pr(O \mid D) = q = 0.01$, while in the absence of $O$ each suite independently misses $D$ with probability $\varepsilon = 0.01$. Then

$$\Pr(\text{all five pass} \mid D) = q + (1 - q)\varepsilon^5 \approx 0.010000000099, \tag{10}$$

not $\varepsilon^5 = 10^{-10}$. Ignoring the shared oracle would overstate the strength against this defect by about $10^8$. The numbers in (10) are an illustration, not calibrated `pid-rs` probabilities. If $q$ and the conditional miss rates have not been externally calibrated, no numerical confidence follows.

The retained raw evidence should say, for example:

| Evidence component | Established locally | Shared boundary retained |
|---|---|---|
| $E_{\text{exec}}$ | Five transformation and corpus families pass | All use oracle $C$ |
| $E_{\text{num}}$ | Oracle values reproduce their own fixtures | No separately derived enclosure yet |
| $E_{\text{formal}}$ | Finite reconstruction lemma closes | Does not identify oracle semantics |
| Candidate separate route | Open until a separately written exact or interval checker passes and its dependency vector is audited | Must not import $C$'s result table |

This vector remains in the evidence packet even if a permitted scalar is later reported.

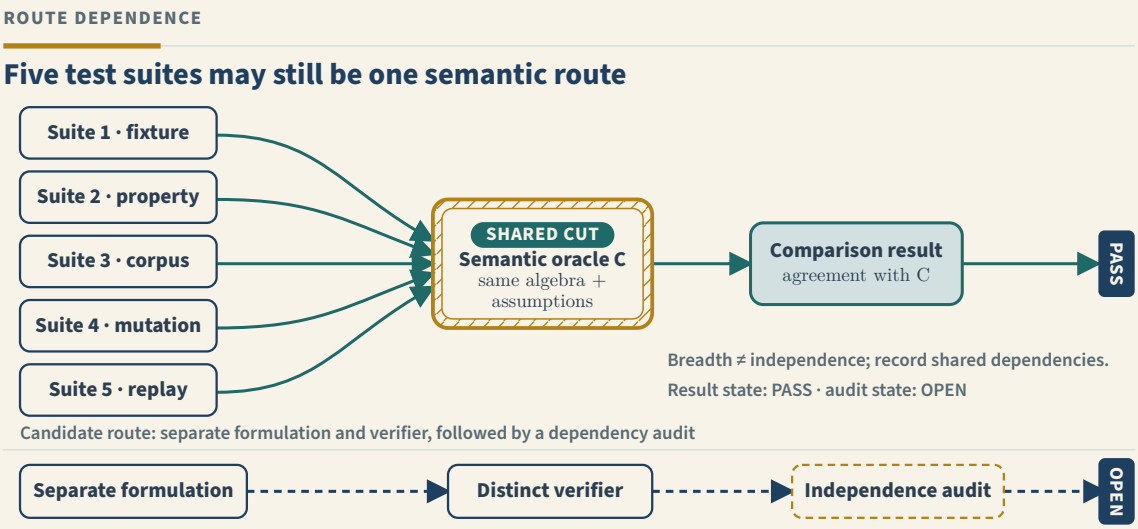



**Figure 3:** Five suite names do not imply five independent semantic routes when every suite uses one oracle.

## 2.5 When a model-conditional posterior belief is permitted

A scalar is prohibited for a vague project, method, or release. It is permitted only for one fully specified empirical decision proposition $H$ with a truth-labelled external reference class. It is not an operational replacement for deductive truth of a fixed theorem. The packet must include all of the following:

1. an explicit prior $\pi = \Pr(H)$, with provenance and interpretation;
2. calibrated likelihoods $L_\theta(E \mid H)$ and $L_\theta(E \mid \neg H)$, defined under one declared evidence probability-mass or density convention and, for densities, one declared dominating measure;
3. an evidence-dependence model, including every identified shared cut-set node;
4. truth-labelled external calibration data not reused to construct, select, stop, or tune the evaluated evidence;
5. a declared reference class, exchangeability or transport assumption, domain-shift model, and the complete frozen evidence-selection and stopping protocol;
6. finite-calibration uncertainty and model-shift uncertainty inside a nonempty sensitivity region $\Theta$, together with uncertain priors, likelihoods, and dependencies;
7. preregistered decision thresholds and an abstention rule; and
8. the complete raw assurance vector (6)–(7) beside the scalar.

For every $\theta \in \Theta$, require $0 \le \pi_\theta \le 1$, finite nonnegative measurable likelihood values under that common convention, and

$$0 < d_\theta := \pi_\theta L_\theta(E \mid H) + (1 - \pi_\theta) L_\theta(E \mid \neg H) < \infty.$$

If any of those premises fails, the scalar route abstains. For a fixed admissible $\theta$, Bayes' rule gives

$$p_\theta = \Pr_\theta(H \mid E) = \frac{\pi_\theta L_\theta(E \mid H)}{\pi_\theta L_\theta(E \mid H) + (1 - \pi_\theta) L_\theta(E \mid \neg H)}. \tag{11}$$

Because $\Theta$ is nonempty and every $p_\theta$ is well-defined in $[0, 1]$, report the possibly nonattained sensitivity bounds, not a selected point:

$$[p_-, p_+] = \left[ \inf_{\theta \in \Theta} p_\theta,\ \sup_{\theta \in \Theta} p_\theta \right]. \tag{12}$$

If this interval crosses a preregistered decision boundary, or if any required likelihood or dependence term lacks calibration, the result is `abstain`. No arithmetic on ordinal labels may be substituted for (11)–(12). Report $p_\theta$ as a *model-conditional posterior belief*, not generic correctness confidence. Even when a scalar is admissible, it never closes a deductive or formal obligation, erases a blocked layer, or enlarges the proposition's scope.

## 2.6 Change invalidation and publication state

Orient every dependency-bearing edge from prerequisite to dependent. If $u \to v$, the current validity of $v$ relies on $u$; changing $u$ invalidates $v$ and every other dependent reachable from $u$. It does not invalidate unreachable upstream artifacts. This orientation makes the affected set an ordinary forward reachability query and prevents the common mistake of walking the graph backward.

A publication claim begins `open`, becomes a local `candidate` only when its exact subject tree and applicable gates are bound, becomes `hosted` only when a hosted run supplies a receipt for an identical or descendant commit, and becomes `published` only when that receipt also binds the relevant inputs, workflow identity, and output hashes. Descendancy alone is not sufficient: the receipt must show that the subject artifacts were unchanged or must bind the new exact subject. A later prerequisite change returns every reachable claim to `open/stale`; publication history remains archived, but the old claim is no longer current.

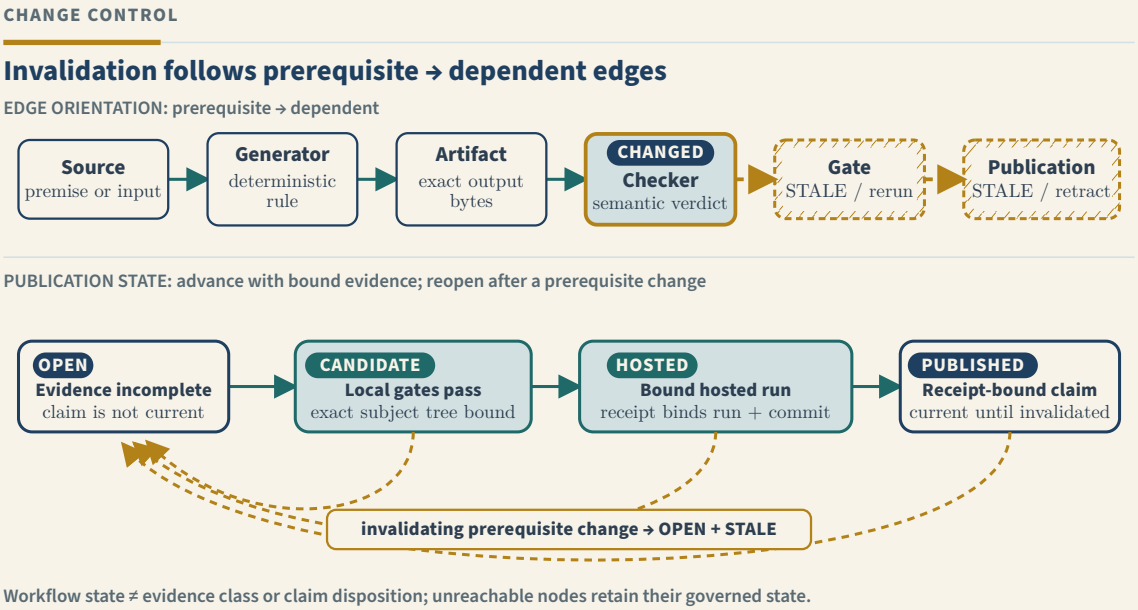



**Figure 4:** Correct edge orientation makes invalidation flow from changed prerequisites to reachable dependents; publication is a receipt-bound state, not a permanent property.

# 3 Mathematical problem-solving workflow for pid-rs

## 3.1 Status and scope

The canonical note has two parts. The first records observations from external sources. The second defines a protocol for pid-rs. The typeset LaTeX/PDF companion adds a reader's primer and an evidence-aggregation supplement before reproducing this canonical text byte-for-byte. Those supplements explain the protocol but do not change it. A source observation is not a pid-rs result.

This note does not validate the mathematical claims in the external sources. It also does not claim that the source workflow guarantees a correct proof. The workflow is useful because it makes the claim, search, failure, and audit steps explicit.

An author, institution, paper, repository, social-media post, or model is a source, not an authority by label. Keep each statement attributed to its source until separately adjudicated. Do not cite social-media posts: a post can be deleted or edited after it is cited and can present an opinion as a fact, which biases readers and AI agents and weakens scientific rigor. State such a point in your own words as coming from a public post of a given date, cite the primary source if one exists, and neither link the post nor name its author. Before adopting a lesson, inspect at least: statement semantics; primary-source correspondence; mathematical logic; executable or

formal implementation; trust surface; numerical representation; hostile controls; provenance and selection; reproducibility and independence dimensions; and scope, nonimplications, and transfer boundaries. A council supplies adversarial hypotheses, not votes that substitute for this review.

The 20 rows used in the older source-review tables are the stable scientific-object baseline, not the current ceiling. For a consequential repository, method, or publication decision, apply those 20 rows and at least 30 explicitly named, object-specific sub-lenses: 50 lenses in total. The extra lenses must expose genuinely separate questions such as estimator validity, binary64 behavior, consumer authority, custody, or a plausible non-PID alternative; splitting one question into cosmetic variants earns no coverage credit. Dated 20-lens tables remain historical records and are not retroactively relabelled as 50-lens reviews.

This note is project-defined process guidance. It has no software implementation or defining method paper. It adds no PID method, estimator, theorem, benchmark result, or validation.

Suggested citation: Sepehr Mahmoudian (2026), *Mathematical problem-solving workflow for pid-rs*. Include the exact repository commit or release. For the software, use CITATION.cff; the sources cited below identify their separate authorship.

The protocol is deliberately standalone for a Codex-like agent that resumes after context compaction. A continuity record preserves work state, but it is never authority for a theorem, repository fact, process state, or external fact: the resuming agent verifies and reconciles those facts read-only against their live sources before acting. The durable-agent-continuity section below is mandatory for consequential multi-step work.

The central rule is:

> An AI model can propose, refine, or attack an obligation. Only retained and replayable evidence can close the obligation.

Model confidence, polished prose, and agreement among models are not evidence classes.

## 3.2 Compact glossary

- **PID (partial information decomposition):** a family of decompositions of information that several sources provide about a target. In the ordinary two-source redundancy-lattice grouping, the four terms are redundancy, two source-specific unique terms, and synergy; multivariate redundancy-lattice PIDs generally have finer antichain-indexed atoms. The name does not select a redundancy functional or transfer axioms among PID proposals.
- **Categorical SxPID:** the Makkeh–Gutknecht–Wibral finite-discrete, pointwise shared-exclusions construction and its average under a finite PMF. The population functional accepts a declared finite population PMF. pid-rs' direct row-data path forms the empirical PMF $\hat{p}_n$ and returns $F(\hat{p}_n)$ for two through four sources. If the empirical law itself is the scientific target, this is a descriptive empirical functional; if $F(p)$ for an underlying population law is the target, it is a plug-in estimator. The API does not choose between those targets. Binary64 representation error remains a separate numerical question. Here "categorical" means finite category-valued random variables, not category theory.
- **General measure-theoretic shared exclusions:** Schick-Poland et al.'s proposed auxiliary-indicator/regular conditional probability (RCP)/Radon–Nikodym construction for a finite source family under the paper's locally compact Radon/Borel, metrizable-compact-subset, and complete-measure premises. Under standard conditioning, $P(R) > 0$ gives $P(T \in A \mid R) = P(T \in A, R)/P(R)$. The reviewed arXiv v2 finite-discrete recovery display

instead writes the unnormalized numerator in Section 4.3.1 (physical PDF pages 13–14), so exact recovery of MGW requires that missing normalization or a clarified definition; this is a retained source-obligation, not an author-correction claim. Local values may be extended-real, global claims need their integrability premises, a noninjective random variable has no pointwise inverse, eventwise RN derivatives need a simultaneous countably additive kernel/version theorem, a Borel isomorphism does not imply the atom-plus-Lebesgue density decomposition displayed with Corollary 3.1 (physical PDF page 9; a Cantor law is a singular-continuous diagnostic), and target-local RN values need representative control even on Euclidean spaces. Evaluation at a null indicator value is not version-invariant from almost-everywhere RCP uniqueness alone, and the reviewed bicontinuity step does not prove arbitrary-measurable-bijection invariance (the reviewed bicontinuity step is in Section 4.3.3, physical PDF page 16). pid-rs implements neither the general route nor a practical general estimator.

- **Continuous shared exclusions:** Ehrlich et al.'s main practical, gauge-dependent continuous density/quasi-density formula and source-disjunction kNN estimator are for purely continuous variables. Appendix B also gives mixed logical-statement quasi-density examples, and Appendix K sketches a mixed estimator ansatz plus one symbolic example while stating that mixed systems exceed the developed estimator's capabilities; it supplies no demonstrated or calibrated mixed estimator. pid-rs does not implement a general mixed route. The disjunction is not a probability union law, and its quasi-density need not integrate to one. The practical route is inspired by, but not identical to, the Schick-Poland construction and is default-off in pid-rs. The paper's matched refining-bin calculation is a premise-bound asymptotic motivation, not identity of the constructions, a general convergence proof, or a convergence theorem for a fixed pid-rs quantizer. Its reviewed Eq. (8) overlap display (physical PDF page 4) repeats $m_{S_2}$; independent substitution of both source-bin widths requires $m_{S_1} m_{S_2}$, a local source correction rather than an estimator change. This is a reviewer-derived candidate source correction, not an author- or publisher-confirmed erratum. Definition 1 and the explicit non-normalization discussion are on physical PDF page 5; the mixed-system limitation and Appendix K begin on physical PDF page 27. Retain three further reviewer-derived candidate source corrections from arXiv v3; none is author- or publisher-confirmed, and none may silently become a pid-rs estimator change. First, the global two-source expectation immediately after Definition 2 on physical PDF page 5 prints $dt\, ds_1\, ds_1$. The density and local integrand depend on $(t, s_1, s_2)$, and the Appendix D derivation on physical PDF page 24 uses $dt\, ds_1\, ds_2$, so the source-consistent reading of the second displayed differential is $ds_2$. Second, Equation (14) on physical PDF page 10 is a natural-log/digamma expression. To reproduce the paper's base-2 definitions and bit-valued examples, the complete right-hand side must be divided by $\ln 2$; pid-rs intentionally retains the natural-unit expression and converts an upstream bit fixture by nats $=$ bits $\cdot \ln 2$. Third, the source-consistent Algorithm 6 wiring on physical PDF page 28 passes the antichain $\alpha$ to both `_compute_epsilons(S,T,antichain)` and `_compute_n_alpha(S,antichain,eps)`, and target counts must come from `_compute_n_T(T,eps)`. The printed pseudocode omits $\alpha$ twice and calls the source count routine on $T$. Algorithm 5 also advertises an unused $\alpha$ argument, while Algorithm 6's result header describes distances although the algorithm returns the redundancy scalar. The authors' pinned code supplies this corrected executable wiring and divides by $\ln 2$ when it reports bits. That code is correlated with the defining-paper route: it helps disambiguate units and intended calls but does not prove estimator consistency, calibration, population-support validity, or refinement by pid-rs.

- **Colored PID:** not a separate functional. A dependence coloring qualifies a sampling theorem, not PID.

- $I_{\min}$**:** the Williams–Beer minimum-specific-information redundancy functional.
- **KSG:** the Kraskov–Stögbauer–Grassberger nearest-neighbour mutual-information estimator. Their higher-dimensional use of "redundancy" denotes multi-information/total correlation, not a PID redundancy functional or atom. For a scientific population-estimation claim, pid-rs uses a conservative supported route requiring i.i.d., unrounded rows from one fixed law; full-dimensional absolute continuity of every required marginal and joint law; finite MI; and explicit boundary, density/smoothness, ambient-coordinate metric, and local-geometry premises. This is the local fail-closed contract, not a claim that no different KSG theorem could weaken a premise. Observed uniqueness and a support declaration do not prove those premises.
- **Estimand:** the exact population quantity a procedure is intended to estimate.
- **Oracle:** a reference implementation or value source used to adjudicate another route. An oracle is evidence only within its stated construction and error contract.
- **Antichain and redundancy order:** an antichain is a nonempty collection of nonempty source subsets in which no member contains another. The explicitly typed finite partial order indexes PID cumulatives. This invokes neither category theory nor an infinite-poset inversion theorem.
- **Zeta and Möbius transforms:** inverse finite linear transforms on the declared finite antichain poset between lattice cumulatives and atoms. A matrix identity alone does not identify the intended event semantics. Infinite-poset use would require a separate theorem with local finiteness or convergence premises.
- **Dependence coloring:** a partition of sample rows into color classes such that the complete rows within each class satisfy the theorem's declared mutual-independence premise.
- **Outward-rounded interval:** endpoints rounded down and up so the exact real value is enclosed.
- **binary64:** the IEEE 754 double-precision floating-point format.
- **MPFR and Arb:** libraries for specified-rounding multiprecision arithmetic and rigorous ball arithmetic, respectively.
- **Lean and SMT:** Lean is a proof assistant with a small proof-checking kernel; satisfiability modulo theories (SMT) solvers decide or search within supported logical theories.
- **Complete separation oracle:** a total algorithm on an exactly declared domain, with a proved termination/resource bound, that returns either a violating object or a checkable certificate that no violator exists. A search guaranteed only to find a violator if it eventually returns is a falsifier search, not a universal-closure oracle.
- **Confidence limits and multiplicity control:** sampling-uncertainty bounds and procedures that control an error criterion across multiple reported hypotheses or cells.
- **Controlled versus blind holdout:** a controlled holdout freezes and separates development from evaluation; it is blind only when the named development roles cannot access sealed rows, seeds, targets, keys, or unredacted results before the first complete adjudication.
- **Digest:** unless a claim says otherwise, a digest is lowercase SHA-256 over the exact raw bytes of the named artifact. Semantically equivalent re-encodings have different digests.

- **Ecosystem projects:** Prisoma analyzes learned representations; Galadriel monitors cross-sensor consistency; Haldir concerns authorization/reference monitoring; and Crebain is a sensor-fusion visualization consumer. Their versioned requirements are tracked in `ECOSYSTEM_CAPABILITIES.md`.

### 3.3 Primary-source pins for PID vocabulary and routing

The source-specific observations and routing boundaries in this note use the exact arXiv revisions below. On 2026-08-04, each linked arXiv submission history ended at the listed revision. That is a dated discovery observation, not immutable byte custody: the source PDFs are not repository artifacts, and a provider can regenerate delivered PDF bytes without changing an arXiv version label. "Reviewed" therefore identifies the cited revision, not a claim that an arXiv PDF and a linked journal edition are byte-identical.

| Local family or use | Exact primary source reviewed and separately linked publication record |
|---|---|
| MGW categorical shared exclusions | Makkeh, Gutknecht & Wibral, "Introducing a differentiable measure of pointwise shared information," arXiv:2002.03356v5, revised 30 Mar 2021; separately, Physical Review E 103, 032149 (2021) |
| Ehrlich continuous shared exclusions | Ehrlich et al., "Partial Information Decomposition for Continuous Variables based on Shared Exclusions: Analytical Formulation and Estimation," arXiv:2311.06373v3, revised 27 Mar 2024; separately, Physical Review E 110, 014115 (2024) |
| Schick–Poland general measure-theoretic construction | Schick-Poland et al., "A partial information decomposition for discrete and continuous variables," arXiv:2106.12393v2, revised 24 Jun 2021 |
| Williams–Beer $I_{\min}$ | Williams & Beer, "Nonnegative Decomposition of Multivariate Information," arXiv:1004.2515v1, submitted 14 Apr 2010 |
| KSG mutual-information estimator | Kraskov, Stögbauer & Grassberger, "Estimating Mutual Information," arXiv:cond-mat/0305641v1, submitted 28 May 2003; separately, Physical Review E 69, 066138 (2004) and its 2011 erratum to Appendix Eq. (A5). The erratum says that error does not affect the paper's other results; pid-rs does not use the corrected appendix extremum claim as an estimator-validity bridge. |
| Shannon-invariant screening quantities | Gutknecht et al., "Shannon invariants: A scalable approach to information decomposition," arXiv:2504.15779v1, submitted 22 Apr 2025 |
| Barà mixed discrete-target/continuous-source route | Barà et al., "Partial information decomposition for mixed discrete and continuous random variables," arXiv:2409.13506v1, submitted 20 Sep 2024 |
| Lyu–Clark–Raviv published theorem package | "Multivariate Partial Information Decomposition: Constructions, Inconsistencies, and Alternative Measures," arXiv:2508.05530v2, revised 11 Feb 2026; separately, Physical Review E 113, 034102 (2026) |
| Lyu–Clark–Raviv structural preprint | "Structural Impossibility of Antichain-Lattice Partial Information Decomposition," arXiv:2604.03869v2, revised 14 Apr 2026; no journal-edition claim is made here |

## 3.4 Source observations

### 3.4.1 Erdős-problem research thread

The thread author reports six solutions to open Erdős problems in five days. The thread does not, by itself, verify those solutions. It describes this repeated loop:

```
attempt -> failure -> diagnosis -> new approach -> proof draft -> adversarial audit ->
↪  revision
```

The author reports runs of about 6 to 32 hours for individual problems. The author also gives these search rules:

- Restate the exact problem.
- State what a complete proof or disproof must show.
- List weaker results that do not solve the problem.
- List traps and edge cases.
- Start several independent approaches.
- Keep incompatible approaches active.
- Search for counterexamples to proposed lemmas.
- Block a route that stops at an unproved statement of comparable strength.
- Challenge each candidate argument before acceptance.

These observations come from a public thread of five posts that its author published on X (formerly Twitter) on 22 July 2026, between 18:53 and 18:54 UTC. In order, the posts introduced the work, stated the exact problem and audit requirements, gave the rules for independent routes and blockers, described the long-run work pattern, and summarized the research loop. pid-rs retains no copy of the thread. The loop and the rules above restate everything that this document uses from it.

### 3.4.2 Lea launch thread and source audit: a proof checker is not a trusted problem setter

The public launch thread for Lea, posted on X on 14 August 2026 at 15:43 UTC, was inspected on 2026-08-15 together with the public `VIDA-NYU/Lea` repository at version-pinned commit `790853c89522bf5899feeb0052b795326feee720`. The separate public evaluation repository was inspected at `18cf90da1c5db574382ab4f0e3e3aad27dd07462`, FormalQualBench at `efaa113c6a00a79e92842ce541b407d7695d7699`, and upstream SafeVerify at `241dfb0d986baf8a3ba76cba14f10c16e8793e84`. No external repository archive or Git-object bytes and no provider-authentication record are retained in pid-rs; these commit hashes and URLs are source locators, not pid-rs custody. This was a process and source-code review, not an execution of Lea and not a verification of any mathematical result attributed to it. No X response bytes or authentication record are retained; the thread text is an attributed observation from the visible retrieval, not a signed statement.

The thread candidly reports that an early run appeared to solve a Quillen–Suslin formalization by weakening a load-bearing definition to a trivial proposition. It also reports a later, much larger Lean development as a proper proof. Those are the thread author's claims. The later proof was not located and audited as a public replay bundle, so pid-rs records no verified Quillen–Suslin

formalization result. Quillen–Suslin is a classical theorem; the reported achievement concerns a FormalQualBench formalization challenge, not discovery of a new mathematical theorem. The thread's separate allusion to internally solved open PDE/TCS problems names no exact theorem or public proof bundle and therefore receives no result credit. The inspected prover README says benchmark harnesses and results live in a separate evaluation repository, so absence from this serving repository is not evidence that they never existed; it is a custody boundary.

The exact historical mutation is present at the evaluation repository's pinned `test_cheats.py`: inside namespace `QuillenSuslinTheorem`, it declares a local abbreviation:

```
abbrev Module.Free (...) : Prop := True
```

The later surface spelling resolves to the local fully qualified declaration and `trivial` proves it. The canonical FormalQualBench target uses Mathlib's global `Module.Free`. This is a kernel-valid proof of the wrong elaborated object, not a Lean kernel break; because the trivialization can be axiom-free, an allowed-axiom scan alone cannot detect it.

The inspected Lea commit exposes the more general boundary directly in source:

- Lea's `safeverify.py` says its target is generated from the submitted file by replacing theorem bodies with `sorry`. It explicitly warns that a weakened statement is inherited by that generated target and can pass; a separately trusted target is required to catch statement drift.
- The verifier is opt-in and can be unavailable. In the subagent promotion path, an explicit `rejected` verdict blocks promotion, but `unavailable` or `error` does not. A clean Lean compile can therefore remain the operative gate in a degraded configuration; the promoted stored step records compiler `check_status=ok`, not a successful SafeVerify receipt.
- The wrapper has a second fail-closure obligation. After a nonzero SafeVerify exit, its universe/hygiene override accepts when one extracted theorem-type mismatch becomes equal after name normalization. It does not establish that the same output contains no second violation. This is not merely hypothetical: the evaluation repository's Fable report records a Pontryagin-duality candidate with `sorryAx` that was accepted by this alternate branch after SafeVerify returned nonzero for an alpha-equivalent type diagnostic. No regex or pretty-printed sub-diagnostic may override a checker-wide failure; acceptance must be the conjunction of every structured verdict.
- The vendored SafeVerify documentation says kernel replay covers the submitted module but not its imports; fresh import replay needs an additional route. It separately lists source-level escape surfaces such as `implemented_by`, `extern`, and `noncomputable`, and compilation-time filesystem effects that require sandboxing.
- Useful controls are nevertheless present: submitted declarations are replayed through Lean's kernel, theorem types and selected definition bodies are compared, the permitted axiom basis is explicit, `partial`/`unsafe` declarations are rejected, and the user-facing tool correctly says that it checks a proof against the file's own statements rather than certifying intended meaning.

The inspected repository's public workflows build the website and container image but do not make SafeVerify or the historical definition-shadow failure a mandatory regression gate. pid-rs does not infer an absent private evaluation, but it does refuse to treat an unpublished run label, leaderboard statement, or model summary as a durable proof artifact.

Project interpretation: pid-rs does not adopt Lea as a dependency or call it mature enough to adjudicate a publication claim. The transferable lesson is architectural, not competitive:

the claim packet, definitions, imports, permitted axioms, and judge must be fixed outside the candidate's write authority; verification must be mandatory and fail closed when unavailable; imports must be replayed from a clean pinned environment; compilation must be confined; and a separate human-reviewed prose-to-formal map must remain open even after kernel acceptance. Calling this candidate/judge isolation is more precise than inferring intent or misconduct from a failed candidate.

### 3.4.3 Grothendieck-constant case study: discovery, certification, and state loss

The public thread on X that announced the human–AI case study (14 August 2026, 14:02 UTC), the exact arXiv v2 papers *Long-Horizon AI Research for Grothendieck Constant* and *New Lower and Upper Bounds for the Grothendieck Constant*, and the public `trishullab/grothendieck-bounds` repository at version-pinned commit `17de8e2331859d37322c199a3d56fb81d740764f` were inspected on 2026-08-15. The downloaded case-study PDF was 1,094,088 bytes, 17 pages, SHA-256 `da428c740ab9af973220823673094e2c031b4522f0fdb531d22e52cfed574fd4`; the mathematics PDF was 678,634 bytes, 56 pages, SHA-256 `d1a6bd20324712eef290dea920fafcd2fd33f0f00a06028ac7fb633adcbbb377`. These downloaded PDFs are not retained repository artifacts, and their hashes are local retrieval observations rather than publisher authentication.

The case-study paper's Section 3 and Tables 1–2 report roughly 240 sessions, 2,091 reasoning-model calls, 152 million tokens, about forty dated human directives, a reasoning model, a separate coding agent, a human bulletin, and session reports used as long-term memory. It labels the lower bound $K_G \geq 6\pi/11$ as additionally verified by the authors and stated as a theorem. It deliberately labels stronger reported bounds only `system-tested`, because their certificates had not yet received that human verification. pid-rs does not independently validate either mathematical paper, and the same authors' review is not institutional independence.

Three process observations are especially load-bearing:

1. Repeated failed constructions were useful only after a human-directed change of question: instead of launching another variant, the run sought a universal obstruction. This suggests a research-judgement trigger, not a theorem schema. A PID obstruction follows only if its exact functional, admissible class, mixture/limit map, and quantifiers are separately proved.

2. Section 5 reports that an exploration-only numerical value survived as an apparent record after its caveat and an earlier feasibility criterion disappeared from repeatedly rewritten summaries. The archive retained the facts, but the active representation lost them; the unsupported value was withdrawn 25 days later. A polished active summary must therefore be a derived view over an append-only obligation/failure ledger, never the authority that can erase caveats, contradictions, or status.

3. Appendix A reports that adversarial review found a finite-to-measurable-mixture gap, a Jensen inequality in the wrong direction, and a coverage defect in the interval certificate. Repairing a proof after those findings does not erase the failed versions; each becomes a load-bearing correction-ledger event. Retain a hostile fixture where permitted; otherwise record the exact privacy, licensing, security, or provider omission instead of claiming a permanent executable control.

The certificate repository claims a useful *shape*: exploratory numerics are separated from Arb checks; exact decimal inputs are declared; the covered parameter domain is partitioned into regimes; margins are reported; tool versions and deterministic commands are named; and completed logs are retained. Source inspection nevertheless found one concrete fail-closure defect

in the public lower-bound wrapper at the inspected commit. The `highc_certificate.py` driver prints three load-bearing Boolean dispositions–the branch-positivity check, the away-region domination check, and the $M_O$ validity check–but does not raise or exit when any is false. The `run_all.sh` predicate greps an unconditional final string and the two branch-and-bound success strings, not those three Boolean dispositions. A false prerequisite can therefore coexist with the wrapper's exit status zero. A separately inspecting consumer could still reject the printed `False`; the defect is the wrapper's success predicate, not evidence that a retained positive disposition is false.

A second candidate concern did **not** survive direct checking and is retained here to demonstrate why representation semantics must be tested before alleging a coverage gap. The high-$c$ source defines $H = 1/12$ in Arb and passes `float(H)` into an interval constructor. That binary64 value is exactly 6004799503160661/72057594037927936, below 1/12 by 1/216172782113783808. But under the declared `python-flint` 0.8.0 semantics, the actual interval constructed by `iv` from zero and `float(H)` has an outward upper endpoint about $0.0833333333527358847 > 1/12$, and the Taylor ball contains exact $H$. This was an unretained, reviewer-derived local calculation, not execution evidence from the upstream project. Its bounded reproduction under `python-flint==0.8.0` is:

```
from flint import arb, ctx
ctx.prec = 300
H = arb(1) / 12
f = float(H)
lo, hi = arb(0), arb(f)
interval = arb((lo + hi) / 2, (hi - lo) / 2)
assert interval.contains(H) and interval.upper() > H
```

The suspected endpoint sliver is therefore **not** established by the binary64 conversion and is withdrawn as a defect.

The non-fatal wrapper predicate is a certificate-wrapper fail-closure/verification defect, not evidence of intent or a refutation of the paper's theorem. Closure requires fatal checks for every prerequisite, hostile mutations that make each prerequisite false, and a clean rerun. This source inspection did not replay the full certificate or prove that every analytic reduction and arithmetic conversion is sound. Generator, checker, repository, paper, and author review also share premises and custody. pid-rs therefore transfers only the requirement to produce a finite, separately re-derived, margin-bearing, fail-closed certificate when the object permits one–never the Grothendieck mathematics or its conclusion.

### Twenty-lens joint disposition

The two cases were reviewed through the same mandatory lenses used below. The entries are prospective requirements or no-transfer dispositions only. They do **not** assert that an existing pid-rs proof, certificate, or accepted claim was produced under this protocol; object-specific compliance remains `OPEN` until a packet contains the required evidence.

| Lens | Disposition for pid-rs |
|---|---|
| 1. Estimand | **NO TRANSFER:** no PID functional, estimator, atom, or objective is supplied by either case. |
| 2. Types | **PROSPECTIVE REQUIREMENT:** task statement, definitions, imports, candidate payload, certificate, and judge are distinct typed objects. |
| 3. Quantifiers | **PROSPECTIVE REQUIREMENT:** freeze their order before candidate access; a later change creates a new revision. |

| Lens | Disposition for pid-rs |
|---|---|
| 4. Mappings | `OPEN` per future claim: neither a Lean pass nor a Grothendieck obstruction maps into PID without a positive theorem. |
| 5. Support/reference measure | `N/A` to the process lesson; every later PID use must still declare its own law and support. |
| 6. Lattice/source count | `N/A`: no antichain order, redundancy functional, or source-count result transfers. |
| 7. Units/gauge | **NO TRANSFER:** exact constants and arithmetic representation must be bound; no information-unit claim is present. |
| 8. Population/empirical | **PROSPECTIVE REQUIREMENT:** exploratory, system-tested, certified, and human-reviewed display tags must not be collapsed. |
| 9. Sampling | `N/A` to these proof cases; estimator sampling claims retain their own obligations. |
| 10. Selection/UQ | **PROSPECTIVE REQUIREMENT:** adaptive route selection, stopping, and chosen-result history require an ex-ante or append-only ledger. |
| 11. Formal correspondence | **PROSPECTIVE REQUIREMENT:** a candidate-derived formal target cannot establish intended-statement identity. |
| 12. Kernel/axioms/toolchain | **PROSPECTIVE REQUIREMENT:** bind and replay the exact import closure, kernel, axiom allowlist, source, and sandbox route. |
| 13. Route dependency diversity | **NO INDEPENDENCE CREDIT:** separate models, roles, sessions, or files are correlated unless their dependency and custody cuts differ. |
| 14. Numerical/binary64 | **PROSPECTIVE REQUIREMENT:** exploration must become exact rational or outward-rounded finite coverage with a positive margin where applicable. |
| 15. Compiled/wrapper parity | `OPEN` per artifact: compilation, kernel replay, source scan, and executable refinement are separate gates. |
| 16. Counterexample/mutation | **PROSPECTIVE REQUIREMENT:** attack target weakening, judge edits, import/axiom expansion, lost coverage, and coordinated status reseals. |
| 17. Custody/threat/refusal | **PROSPECTIVE REQUIREMENT:** judge and task packet remain outside candidate authority; unavailable verification is a refusal, never a pass. |
| 18. Citations/novelty | **ATTRIBUTED OBSERVATION:** author reports and repository claims remain so until separately adjudicated with the applicable independence vector recorded. |
| 19. Ecosystem/authority | **NO TRANSFER:** Prisoma, Galadriel, Haldir, and Crebain inherit no readiness or PID result from these process studies. |
| 20. Resource/platform/release | **PROSPECTIVE REQUIREMENT:** retain model/tool versions, budgets, interventions, failed attempts, selected route, platform, and publication status separately. |

### 3.4.4 DeepSeek Harness, Cordis, and category theory: explicit mapping checks, not branding

The newly public `deepseek-ai/deepseek-harness` repository was inspected on 2026-08-15 at version-pinned commit `47f943859bef60e4160492346772ded9b24f765a`, the merge that published the `0.1.0-rc.5` family. Its own README calls the release a developer preview

and warns of compatibility-breaking changes. The inspected harness vendors Cordis `4.0.0-rc.7` at upstream commit `56b3d4f...`, then records substantial local lifecycle and transactional hardening in its vendor manifest. The current public Cordis repository was separately inspected at `8cc9e33fab69e2d0476d126baaf2acb24e6a6ab4`, and its actively revised 88-page preprint, *A Programming Paradigm for Spatiotemporal Composability*, at paper commit `948a07b369c62adb3b12e102458be5c18dfb69b9`. These external repository objects and PDF bytes are not retained by pid-rs, and the source review does not authenticate a provider or replay the TypeScript implementation.

Cordis uses real algebra rather than category-theory decoration. It models context transformations and their oppositely accumulated candidate inverses by a twisted-composition monoid, proves homomorphism and commutation results, and combines revertible effects with reactive coeffects. Monads, comonads, and enriched categories supply background vocabulary, while the central construction is an explicit operational calculus over effects, dependencies, and lifecycle traces. That distinction matters: the paper's implementation section explicitly says that `ctx.effect` does **not** verify the witness that a returned disposer actually reverses its effect. The component author retains that obligation. The paper also defines effect "independence" by commutation of forward/inverse transformations and stability of yielded inverses. It is not epistemic, institutional, statistical, data, implementation, or custody independence.

The DeepSeek Harness exposes several useful engineering lessons without making them mathematical evidence. Its architecture separates service definition, provider, and consumer; makes model-visible context derivable from an append-only session log; declares dependencies; and treats registrations as reversible effects. But "everything is a plugin" and "there is no privileged core" are the wrong acceptance model for pid-rs research. Proposers, search heuristics, and non-authoritative tool adapters may be replaceable. A frozen statement, axiom/import policy, judge, reference values, and acceptance predicate must remain outside candidate authority. A session log improves replay only to the extent that the logging boundary is complete and trustworthy; it does not prove authenticity, semantic correctness, or absence of unlogged inputs. Scratch-state teardown may be reversible, whereas accepted evidence and correction history remain append-only. An external publication, remote write, or leaked secret is an emission, not an invertible effect.

Category theory has one immediate, bounded use in this project: it sharpens **explicit mapping checks between named object types**. In plain language, this asks whether a result really carries from one object to another, and what exact evidence licenses that step. The default artifact remains a typed directed multigraph, because many research edges are partial, premise-indexed, approximate, statistical, or versioned. A category-theoretic term is permitted only when its laws are part of the claim:

- a proposed category names its objects, morphisms, equality, identities, composition, and the closure/associativity premises actually used;
- a proposed functor names both categories and proves preservation of identities and composition;
- a claimed naturality or commuting square names every map and proves exact equality, or replaces equality with a separately justified metric/divergence bound and total error budget;
- parallel proof, formalization, estimator, representation, and consumer paths are not identified merely because they end in similarly named values; and
- the absence of a proved arrow is an `OPEN` or `BLOCKED` transfer, not an invitation to draw one.

For example, a paper-to-formal-to-executable claim may use the following square only after its four arrows and the path equality are separately bound:

```
paper object  --formalization-->  formal object
     |                                  |
  semantic map                      extraction/refinement
     |                                  |
     v                                  v
reference object --implementation--> executable object

required witness: extraction/refinement o formalization
                  = implementation o semantic map
```

An approximate version must state the common codomain, metric or divergence, domain of validity, and an aggregate bound; visual symmetry is not evidence that the square commutes. A failed square identifies an exact missing edge or path disagreement and is retained as a counterexample.

Two tempting project-specific recastings are rejected for now. First, the finite PID antichain poset can be regarded as a thin category, but pid-rs already has the exact order and finite zeta/Moebius transforms it needs. Re-labelling that code adds no theorem, algorithm, or safety. Second, Markov categories offer a serious compositional language for stochastic channels, conditional independence, data processing, and comparison of statistical experiments. Perrone's *Markov Categories and Entropy* recovers mutual-information and entropy constructions from enriched categorical structure, and Fritz et al.'s *Representable Markov Categories and Comparison of Statistical Experiments in Categorical Probability* treats Blackwell–Sherman–Stein comparison. Neither source defines MGW shared-exclusions atoms, proves an Ehrlich estimator, supplies a PID lattice measure, or validates Rust/binary64 behavior. A future Blackwell-order PID investigation would be a distinct PID proposal with its own method identity; it would not be an alias or bridge to MGW, Schick–Poland, Ehrlich, Williams–Beer, PID2, or PID3.

### Twenty-lens adoption disposition

| Lens | Disposition for pid-rs |
|---|---|
| 1. Estimand | **NO TRANSFER:** Harness/Cordis and categorical probability supply no pid-rs PID estimand or estimator. |
| 2. Types | **ADOPT AS A PROSPECTIVE REQUIREMENT:** every diagram must type its objects, arrows, equality, and applicable premises. |
| 3. Quantifiers | `NARROW`: composition preserves only the quantifiers actually proved for every arrow; none may be inferred from diagram shape. |
| 4. Mappings | **ADOPT FOR THIS BOUNDED USE:** commuting-path obligations make missing or disagreeing transfers explicit. |
| 5. Support/reference measure | `OPEN` per stochastic use; categorical notation never supplies support, disintegration, density, or reference-measure premises. |
| 6. Lattice/source count | **NO CHANGE:** the finite antichain poset and Moebius direction remain the current operative specification; thin-category language adds no credit. |
| 7. Units/gauge | **NO TRANSFER:** every arrow still carries its own nats/bits, metric, scale, and gauge behavior. |
| 8. Population/empirical | `NARROW`: population laws, empirical PMFs, estimators, and outputs remain distinct objects with no automatic arrow. |

| Lens | Disposition for pid-rs |
|---|---|
| 9. Sampling | `N/A` to Cordis; a statistical diagram still needs its exact row law, dependence, split, and rate premises. |
| 10. Selection/UQ | `NARROW`: durable event logs motivate selection ledgers but do not establish completeness, blinding, or uncertainty control. |
| 11. Formal correspondence | **ADOPT AS A CHALLENGE TEMPLATE ONLY:** the two paper/formal/executable paths must agree under a retained witness. |
| 12. Kernel/axioms/toolchain | **NO CREDIT:** a diagram, type checker, or plugin framework does not inventory or replay a proof kernel and trust surface. |
| 13. Route dependency diversity | **QUALIFIED:** Cordis effect commutation is not scientific independence; the project's independence vector remains the operative independence specification. |
| 14. Numerical/binary64 | `OPEN` per edge: approximate commutation needs one stated metric and aggregate error bound below the claimed margin. |
| 15. Compiled/wrapper parity | `OPEN`: the Cordis paper's theory-to-code table and pid-rs source mappings require separate executable/refinement evidence. |
| 16. Counterexample/mutation | `NARROW`: mutate domains, codomains, maps, premises, and one path at a time; a noncommuting witness must fail the transfer. |
| 17. Custody/threat/refusal | `NARROW`: exact external locators are recorded, but no external archive, authenticated capture, or TypeScript replay is retained. |
| 18. Citations/novelty | **ATTRIBUTED SOURCE-REVIEW:** this is a source-derived process lesson and no categorical-PID novelty claim. |
| 19. Ecosystem/authority | **NO TRANSFER:** no Prisoma, Galadriel, Haldir, or Crebain readiness follows from the abstraction. |
| 20. Resource/platform/release | `NEGATIVE` for dependency adoption now: both Harness and Cordis declare unstable preview status; pid-rs adds no runtime or CI dependency. |

### 3.4.5 Cycle Double Cover prompt

The Cycle Double Cover prompt starts with exact definitions. It fixes the graph class, the meaning of a cycle, edge multiplicity, disconnected graphs, and the empty graph. It then states the one result that counts as complete.

The prompt also lists results that do not count. Examples include proofs for special graph classes, finite computation, a cover with the wrong edge multiplicity, and a reduction to another unproved claim. It asks for independent approach families. It delays the sharing of ideas between routes until each route has clear strengths and gaps. It requires concrete lemmas, constructions, equations, or counterexamples. It also gives a problem-specific audit list.

Project interpretation: this structure separates three questions:

1. What is the exact claim?
2. What work can help the search but cannot complete the claim?
3. What checks can falsify a candidate proof?

### 3.4.6 Erdős repository

The source tree was inspected at version-pinned commit `f2ae0edb45cbdb257e135d51ef855f64caeb348b`. At that commit, each of the six problem directories contains a problem-specific prompt and a paper. The tree also contains Lean projects for problems 486, 788, and 1038. It contains Python numerical verifiers for problems 390, 536, and 1038.

The version-pinned prompt files are available for 1002, 1038, 390, 486, 536, and 788. The prompts state the quantifier order and required target strength explicitly. They list non-solutions and problem-specific failure modes. Examples include:

- an exact asymptotic limit instead of an order bound;
- one fixed infinite witness instead of a different finite witness at each scale;
- a full-sequence limit instead of a selected subsequence;
- exact extrema and sharpness instead of numerical candidates;
- strict control of limit interchange, tightness, and mass escape;
- preservation of integer, graph, or measure structure during a reduction.

Project interpretation: problem 1038 shows one certificate pattern. Its numerical verifier is designed to use a double-precision scan for initial brackets. Its higher-precision and Arb interval checks are designed to certify signs. The Lean project contains rational and interval data for finite kernel checks. This review did not run the verifier or build the project. The inspected commit also contains version-pinned 486 and 788 Lean trees. It has numerical verifiers for 390 and 536.

Project interpretation: no complete chronological failure log was found in the inspected commit. The prompts request a route registry and blocker policy, but this inspection did not find a full transcript. This difference matters when pid-rs records its own process.

### 3.4.7 Erdős 477 cubic repository

The `pw/erdos477-cubic` repository was inspected at version-pinned commit `8440d599890b5a5ef7b212c65338723ab2443eaf`. This was a process review only. It did not validate the repository's claimed theorem.

Five process choices are useful for pid-rs:

1. The repository pins a semantic ambiguity against the primary problem statement before using the proof. In its case, it distinguishes uniqueness of an image value from uniqueness of a parameter that enumerates that value.
2. It preserves the earlier, narrower cubic argument when the claim expands to all higher powers. The narrower route remains a worked instance and an independent audit target instead of being overwritten by the generalization.
3. Its `VERIFICATION.md` separates elementary bookkeeping from two imported determinant-method results. It explicitly says that two proof narratives that share the same imported crux are correlated evidence, not two independent confirmations.
4. It retains the actual adversarial briefs for the cubic claim and the generalized claim. Each brief names concrete failure targets, asks for refutation rather than confirmation, and distinguishes a surviving argument from an independently re-derived one.

5. An adversarial pass found an incorrect genus formula. A cheap invariant exposed it: the old formula produced a non-integer genus in one case. The repository corrected the formula, identified the exact dependency subgraph that used it, and recorded why the main theorem did not depend on that subgraph.

The same record also shows limits that pid-rs should not copy. The construction and adversarial passes used the same model family. The raw sessions are not retained in the repository. No formal proof or human specialist review is present. Its primary-source checks establish that cited statements exist and that an application appears to meet their hypotheses; they do not re-prove the cited determinant methods. Therefore, labels such as “full proof” in that repository are not evidence for pid-rs and do not change any pid-rs disposition.

Project interpretation: four transferable controls should be added below - critical-cut-set accounting for correlated routes, an imported-theorem application map, a load-bearing correction ledger, and cheap invariant probes. These controls improve auditability without importing the external mathematics.

### 3.4.8 Corrected vector-bundle claim: a typed citation-edge failure

The public correction chain was inspected on 2026-07-25. It consisted of the linked draft and four posts on X, all on 24 July 2026. At 14:21 UTC the draft’s author announced its result. At 20:50 UTC another reader posted a specific objection to the argument. At 21:27 UTC the draft’s author acknowledged the objection, and at 21:31 UTC posted a conspicuous correction. pid-rs retains no copy of the posts; the draft and the cited papers support the mathematical analysis below. The downloaded 18-page draft had SHA-256 `ebb5aa2c8d1d08cd1c7692ac0526cf537c00985bf5e129ff165be128404e69ca`. The three distinct images visible in the claim/correction chain were inspected at their original resolutions: a paper-introduction image, a screenshot of Theorems A and B, and a screenshot of the correction. They repeated claims or text present in the draft/thread and supplied no independent proof.

The dedicated Chrome-extension automation transport failed twice. Codex Computer Use subsequently loaded the signed-in public thread in Chrome and exposed its current visible conversation/replies tree. Public status responses, the first public replies page, the linked draft, and the primary cited paper were used as corroborating retrieval paths; reproducing the same content does not make them independent mathematical routes. The public replies page was a third-party transport and returned 20 items, including nested responses; its pagination cursor did not advance. These access details are session observations, not a retained browser attestation. This record therefore covers the mathematical exchange and follow-ups visible through those paths, but does not claim completeness for deleted, private, later, or unreturned replies. Social agreement, criticism, or silence was not treated as mathematical evidence.

The load-bearing error is narrower and more informative than “an AI proof was wrong.” The draft’s Theorem 7.1 uses the exact sequence

$$\pi^{\mathbb{A}^1}_{10,5}(S^{9,5})(\mathbb{C}) \longrightarrow \pi^{\mathbb{A}^1}_{9,5}(SL_4)(\mathbb{C}) \longrightarrow \pi^{\mathbb{A}^1}_{9,5}(SL_5)(\mathbb{C}).$$

It then reads Theorem 7.2.1 of the version-pinned arXiv v3 source `2306.04631v3`, downloaded here with SHA-256 `d4bd95572c7e2c356e964407ceab26c64c37768a655b45b45feaa4ab50dd8536`, whose displayed short exact sequence is

$$0 \longrightarrow K^M_{d+2-j}/24 \xrightarrow{(\nu_d)_*} \pi^{\mathbb{A}^1}_{d+j,j}(S^{2d-1,d}) \longrightarrow GW^{d-j}_{d+1-j},$$

followed by the grammatically ambiguous words "which is an isomorphism if $j \geq d-3$." The draft attached "is an isomorphism" to the left map $(\nu_d)_*$ and, at $d = j = 5$, asserted

$$\pi^{\mathbb{A}^1}_{10,5}(S^{9,5})(\mathbb{C}) \cong K^M_2(\mathbb{C})/24 = 0.$$

The preceding source discussion, together with Theorem 7.2.2(3), assigns the range-dependent surjectivity property to the rightmost map to the Grothendieck–Witt sheaf, not an isomorphism property to $(\nu_d)_*$. Theorem 7.2.1's word "isomorphism" remains grammatically and mathematically defective in the displayed formulation, so this review does not repair it by choosing whichever referent makes the draft's argument work.

There is a stronger source-based check. The proof of Theorem 7.2.1 invokes the stable 1-line calculation of Røndigs–Spitzweck–Østvær. Its version-pinned `1604.00365v2` artifact, downloaded here with SHA-256 `bdcf6f0ef128457740c09c5fb38c1a187951b29119d5ec2b8ba0339cb7887966`, states in equation (1.1) and Theorem 5.5, for the relevant fields and range, the exact sequence

$$0 \longrightarrow K^M_2(F)/24 \longrightarrow \pi_{1,0}\mathbf{1}(F) \longrightarrow F^\times/(F^\times)^2 \oplus \mathbb{Z}/2 \longrightarrow 0.$$

ABH's stable-range comparison specializes the middle sheaf evaluation at $d = j = 5$ to this stable group. Over $F = \mathbb{C}$, both outer arithmetic simplifications are exact: every Milnor symbol $\{a, b\}$ is $24\{\alpha, b\}$ after choosing $\alpha^{24} = a$, so $K^M_2(\mathbb{C})/24 = 0$; and every nonzero complex number has a square root, so $\mathbb{C}^\times/(\mathbb{C}^\times)^2 = 0$. The remaining summand is not zero. Consequently,

$$\pi^{\mathbb{A}^1}_{10,5}(S^{9,5})(\mathbb{C}) \cong \mathbb{Z}/2.$$

Equation (27) is therefore false, rather than merely unsupported. This is an evaluation of motivic homotopy sheaves on $\mathrm{Spec}(\mathbb{C})$; it is not the separate Betti or complex- realization argument later in the draft. The characteristic and field hypotheses of both cited results apply to $\mathbb{C}$.

The specialization itself supplies the smallest retained human-readable counterexample to the wrong adjacent-arrow inference:

```
0 -> 0 -> Z/2 --id--> Z/2 -> 0
```

The right nonzero map is an isomorphism, while its adjacent map from zero is not and the middle group is nonzero. The executable checker represents the same finite group as $C_2 = \mathbb{Z}/2$,

```
0 -> 0 -> C2 --id--> C2 -> 0
```

and exhausts every element, image, and kernel. The prose and executable witnesses are two representations of one countermodel, not independent evidence routes.

Dependency reach must remain scoped. Equation (27) was used to make the second arrow in (26) injective; after separately arguing that this arrow has zero image, the draft concluded its domain was zero. The corrected first group makes that inference unavailable. Conditional on retaining the draft's separate zero-image argument, exactness now gives only a surjection

$$\mathbb{Z}/2 \twoheadrightarrow \pi^{\mathbb{A}^1}_{9,5}(SL_4)(\mathbb{C}),$$

so the target group is constrained to be either 0 or $\mathbb{Z}/2$; this audit does not choose between them. The displayed proof of Theorem 7.1 therefore fails, but this calculation does not disprove Theorem 7.1. In the draft's displayed dependency chain, Theorem 7.1 feeds Corollary 8.3, Theorem 8.4, Theorem A's no-motivic-lift and nonalgebraizability route, and Corollary 1.1.

Earlier $\mathbb{P}^4$/projective machinery lies outside this particular cut: that means this error does not by itself invalidate it, not that this inspection audited or salvaged it. This inspection did not prove or disprove non-algebraizability, did not prove that a motivic lift exists, and did not adjudicate the author's later claim that other machinery in the draft may survive.

Seven lenses isolate the transferable process result:

1. **Semantic:** the anaphor "which" had two type-correct-looking referents.
2. **Logical:** a property of one arrow in an exact sequence was transferred to its neighbor.
3. **Source:** checking that a cited theorem exists did not check which morphism its qualifier names.
4. **Retrieval:** a model that was suspicious before lookup reportedly accepted the step after reading the ambiguous source, so retrieval can reinforce rather than remove an error.
5. **Independence:** repeated adversarial readings by the same model family against the same prose shared the same citation edge and count as correlated evidence.
6. **Formal:** a proof assistant can check the wrong imported premise if the human correspondence layer binds the wrong arrow; the seam needs an exact typed signature, not merely more code.
7. **Sociotechnical and scope:** public expert review happened quickly but stochastically. Its success here does not make social-media silence evidence, and one broken route does not by itself invalidate every independent result in the draft.

Project interpretation: add the typed citation-edge gate below. This is PID-neutral process evidence. No vector-bundle theorem, motivic calculation, or alternative PID definition is imported into pid-rs.

### 3.4.9 External AI discussions

The supplied workflow reports that three linked AI discussions were retrievable on 2026-07-24. They are process examples, not evidence that their mathematical conclusions are correct.

The Jacobian discussion shows why a review must freeze signs, orderings, normalizations, and exceptional cases before it interprets a calculation. A checked local determinant does not establish a global statement. Likewise, a fixed-support continuity result does not establish a finite-sample rate or a binary64 implementation theorem.

The unsplittable-flow discussion records a false candidate counterexample. The candidate failed when the full path closure was included. For PID, the corresponding rule is to include all antichains, event unions, supported realizations, branches, permutations, and implementation paths named by the claim.

The pid-rs correctness discussion is an external review intake. Its recommendations are work items until repository evidence replays them. Its pass or fail language does not change a project claim disposition.

### 3.4.10 OpenAI-hosted proof-process walkthroughs: process controls only

Two long collections obtained from OpenAI CDN URLs were reviewed at exact source bytes:

| Source | Exact reviewed artifact | Coverage boundary |
|---|---|---|
| How the Ideas Came Together (`reasoning-walkthroughs.pdf`) | 441,468 bytes; SHA-256 `13b95999f060c0be2142089cfb8b17b75e9231c3c1f3fa0980445ff1b35f0b3b`; 62 PDF pages | 62/62 pages text-reviewed and rendered in page order |
| Ten Advances in Mathematics and Theoretical Computer Science | 2,266,371 bytes; SHA-256 `64b900d5fae6fe22f2ae1b8e3b712d20055194a6c81cf343a2455e5898ac7dd6`; 249 PDF pages | 249/249 pages text-reviewed and rendered in page order; physical and printed page numbers were distinguished |

No missing, blank, corrupt, clipped, or misordered source page was observed in the reviewed renders. That is a source-coverage and layout observation, not verification of either collection's theorems. The first collection says on PDF page 2 that its narratives were written by an AI model after access to original chains of thought and resulting papers. The narratives are therefore retrospective process accounts, not reproducible private reasoning traces or primary PID sources. The second collection states field-specific results across harmonic analysis, coding, groups, operator algebras, complexity, quantum information, geometry, Ramsey theory, and related areas. This review did not validate those results. Page anchors below are physical one-based PDF pages.

The two collections are also correlated evidence: their topic and result sets overlap, the first is explicitly retrospective, and both were supplied from the same OpenAI CDN source family. Agreement between them does not create institutional independence, prove that a control was followed in pid-rs, or close a mathematical obligation. Only the following premise-explicit process controls transfer:

| Transferable control | Page-anchored source observations | pid-rs use and required boundary |
|---|---|---|
| Freeze the exact object, conventions, comparator, and completion claim before search | Reasoning pp. 6–10, 29, 33–36, 50, 55; Ten Proofs pp. 3–12, 29–35, 79–99, 114–117 | Bind the PID functional, support, source roles, order, units, representation, and optimized target. A precise statement is not evidence that it is true. |
| Preserve hypotheses through every reduction and representation change | Reasoning pp. 17–24, 39–40, 47–48, 53–54; Ten Proofs pp. 6, 49–58, 84–94, 96–108, 184–218 | Record source and destination objects, preserved premises, cost/scale conversion, and map-back. Similar formulas or output equality are not a mapping theorem. |
| Make central, tail, endpoint, interior, finite, and optimizer-escape regimes exhaustive | Reasoning pp. 8–14, 31, 41–45, 49, 54, 56–58; Ten Proofs pp. 11–25, 41–65, 154–180, 195–206, 219–227 | Cover zero cells, support changes, singularity, finite cutoffs, and nonattained optima. A local or compact-range argument cannot close a global claim. |
| State limit order, uniform constants, finite thresholds, and subsequence-to-full-sequence bridges literally | Reasoning pp. 8–10, 13–14, 41–45, 54, 56–58; Ten Proofs pp. 13–27, 36–48, 59–76, 164–180, 221–235, 242–248 | Freeze which parameter is fixed first, prove uniformity where limits move, and close the finite exceptional range. A sampled hierarchy, subsequence, or asymptotic rate is not an all-index theorem. |
| Separate attainment from a supremum/infimum and prove witness existence at the claimed level | Reasoning pp. 12–14, 28, 30–32, 49, 56; Ten Proofs pp. 29–48, 59–65, 154–167, 219–227 | Do not select an optimizer when only an approximating sequence is known; construct a finite witness before taking an accuracy limit. |

| Transferable control | Page-anchored source observations | pid-rs use and required boundary |
|---|---|---|
| Freeze whether one witness must satisfy all obligations or different witnesses are permitted | Reasoning pp. 17–19, 39–40, 47–48; Ten Proofs pp. 85–90, 207–212, 238–242 | A source-blind verifier, common law, common split, or common certificate must be shared when the quantifiers require simultaneity. Per-cell or per-obligation witnesses do not imply one joint witness. |
| Preserve probability weights, conditioning objects, and representation levels | Reasoning pp. 20–24, 33–36; Ten Proofs pp. 53–58, 96–108, 154–180 | Keep empirical-count weights, selection/conditioning mass, source/target roles, scalar base, and original/transformed variables explicit. A marginal or conditioned calculation does not silently establish the joint operational object. |
| Pair cancellation with a nonzero-survival certificate | Reasoning pp. 26, 41–45; Ten Proofs pp. 126–139, 164–180, 204–212 | A Möbius, parity, log-product, or represented-sum cancellation must prove that required terms survive and denominators remain valid. Equality after every useful term cancels is vacuous. |
| Pair relaxations, quotients, truncations, and regularizations with a proved map back | Reasoning pp. 7–10, 25–32, 34, 44; Ten Proofs pp. 13–17, 53–58, 126–151, 184–218, 238–242 | The enlarged or smoothed object must satisfy the original constraints or have an exact return map. Quantization, added noise, or a restricted lattice can change the pid-rs estimand. |
| State adjacent non-implications and retain the smallest falsifier | Reasoning pp. 7, 12, 25–34, 44; Ten Proofs pp. 79–112, 140–151, 184–218, 236–249 | Record which nearby theorem, computational model, source arrow, or stronger quantifier does not follow. A counterexample kills only the implication it instantiates. |
| Turn computation into a finite certificate before an asymptotic or universal claim | Reasoning pp. 12–14, 28, 30–32, 49, 56; Ten Proofs pp. 36–40, 105–121, 164–180 | Check exact finite objects, margins, rounding, and resource bounds first. A plot, one fixture, or high-precision agreement remains bounded evidence. |

Several tempting transfers were explicitly rejected:

- No Mellin/Fourier, coding, expander, group, operator-algebra, quantum, algebraic-geometric, lattice-hardness, Bergman-kernel, Ramsey, or Shannon-capacity theorem was imported. Shared words such as information, entropy, conditioning, lattice, measure, cancellation, or witness do not establish a PID correspondence.
- A route that merely failed is not a counterexample. It is a negative search result until an exact witness falsifies a named quantified statement.
- One witness for each object is not one simultaneous witness; one good benchmark cell is not one implementation valid on every declared cell.
- Bounded enumeration, sampled parameters, finite fixtures, one subsequence, or an asymptotic construction is not universal or all-index evidence without the stated bridge.
- Existence from a probabilistic argument is not an executable generator; a theorem statement or bibliography is not a checked application; and two bounds sharing an imported crux are not two independent confirmations.
- The collections' subject-matter conclusions, reported chains of thought, model confidence, and retrospective narrative are not pid-rs evidence.

The accepted controls influenced the claim schema, dependency-cut accounting, theorem-application map, finite-certificate path, exceptional-case checklist, holdout rules, and invalidation protocol below. They did not add a PID theorem, validate an estimator, close Programs A–E, or change a claim disposition. The table above is the retained compact source-review record. The source PDFs, per-page renders, and scratch ledgers are not repository artifacts, so a replay must reacquire the exact-hash source bytes and regenerate its own page renders rather than treating the coverage statement as an embedded copy of the reviewed evidence.

### 3.4.11 Zeta two-thirds source review: mathematics, methods, process, and the current PID no-direct-transfer disposition

A public thread on X of 10 August 2026 that announced the result was reviewed on 2026-08-11 together with the linked primary artifacts. It consisted of a first post at 17:35 UTC, a reply with a note and an image at 17:42 UTC, and a reply that explained the history of the problem at 18:05 UTC. The first post quoted an announcement on X of 17:28 UTC by the organization whose research page is the first artifact in the table below. The posts and the image communicate the result; they are not additional mathematical evidence. The snapshot covered the first post, every visible reply by the same author up to the historical explanation, and the quoted announcement; third-party replies and visibility change over time. The HTML alternative-text field of the image was empty. Its large JPEG rendition was 1168 by 1709 pixels, MIME `image/jpeg`, SHA-256 `2f4549670ad8fb66f071ef9dff73c78d4b8c71936204e9f473715bdd28257c41`, and it renders the first page of the informal note in the table below, not a separate derivation. A WebP rendition of the same dimensions has different bytes and receives no identity credit here. pid-rs retains no copy of the posts or the image. The Anthropic page's hero alt text says "Illustration of a mathematical compass," while its social card labels the compass image "Anthropic logo"; that metadata mismatch has no mathematical content.

The exact source set was:

| Artifact | Reviewed identity and boundary |
|---|---|
| Anthropic research page | Source landing page and project summary; an institutional web post/communication, not scholarly publication or journal peer review. HTML observed at local retrieval `2026-08-11T08:48:40+02:00`: 138,150 bytes, SHA-256 `68a7ea183b8f517820c730abea1d1f0e8a2204ad70b1390e5e17fa71ce1343ce`; an `09:10:21+02:00` refetch matched. These are local observations of a mutable page, not server-signed receipts. |
| Claude, *More Than Two Thirds of the Zeros of the Riemann Zeta Function Lie on the Critical Line*, 10 August 2026 | 35 pages; 631,785 bytes; SHA-256 `6792988e6cd0e17690621ce898abd5d534f98407741bc7cb14bbe7d07c77d72f` |
| 5-page informal note | 191,249 bytes; SHA-256 `45e0330ad37965e5531fa1f4f11e5bebcae147a5237a3e5b3d029efa7ddf759d` |
| Anthropic, *How the two-thirds argument was found: two agent runs and their literature* | 95 pages; 1,058,230 bytes; SHA-256 `271aba2d2083ffa778a53c2994f2061fad7fdda450bc296ec49c7cc41e91dd2d`; model-authored process record, not independently verified mathematics |

| Artifact | Reviewed identity and boundary |
|---|---|
| 116-page annotated transcripts | The CDN URL was mutable during review. Retrieval A completed at local file mtime `2026-08-11T08:38:36+02:00`: 2,081,416 bytes, SHA-256 `68f7b5d5676952cb6a5060dac4889e5d0e412d3422a303458cb0cac8c4180c28`. Retrieval B completed at `2026-08-11T08:58:44+02:00`: 1,952,068 bytes, SHA-256 `ebb34c5ed65b1dc96a72bdf76068814a34da9ceb1675624f68a2088180123ada`. Both produced 543,654-byte `pdftotext` output with `-layout`, SHA-256 `b4a64a2907d2aa543fd8b6d7b0965ec0338cacd920a6e78931c68194001685b8`; sampled page renders 1/69/116 matched, but no full raw-PDF equality is claimed. The timestamps are local observations, not server-signed receipts. The artifact typesets the complete exported visible message record for the two named subagent runs, summarizes most tool calls, and includes only selected surrounding coordinator records; hidden reasoning is absent and infrastructure fields are redacted. It is not a raw or complete campaign execution trace. |
| Lean repository, annotated tag object `82ee6340d6fb15d51fc73ba1ba7b8cac672a7bba` peeling to commit `3635e74826a4c1fcece7d1cd2b6fa75e43a00510` | Static source inspection only. The v1.0 tag object contains an SSH signature and GitHub reports that object verified; independent signer identity still depends on a key/allowed-signers custody policy. The repository pins Lean 4.33.0-rc2 and Mathlib `51e6992efd06126df61a496bebf8f49482a4e129`, different from the target worktree's pinned Lean 4.32.0 finite-convergence toolchain at source-review time. Static inspection found 33 intentional `sorry` placeholders only in the 15+12+6 trusted challenge statements, none in `Zeta23` or `Solution`, and no executable project axiom declaration. The 15 core comparator statements are closed epsilon, dyadic-window, or cumulative theorems tying zeros to Mathlib's zeta and multiplicity to `analyticOrderAt`; the separate PairCeiling route carries `EnclOK`. The recorded headline axiom inventories contain only `propext`, `Classical.choice`, and `Quot.sound`; comparator configurations request nanoda replay. These facts do not establish paper-to-Lean correspondence. `AUDIT.md` records a 9,010-job build and 343/335/345-second comparator runs, but these are self-recorded: the repository exposes no GitHub Release or Actions workflow, and this review did not replay them. XiPrime and PairCeiling commits landed after the 7 August initial-release commit `cac1c55684ee31ed76a4c0573e974c01794851c5` but before the 10 August announcement; they are beyond the core Theorems A–E/headline result. In particular v1.0 contains six XiPrime comparator statements and README claims about `0.85838/0.92919` flat and `0.86864/0.93432` quartic bounds, while its docstrings cite an authors' technical supplement not present in the reviewed tree. Those statements were inventoried but not paper-correspondence-reviewed and supply no PID-transfer evidence. |
| Comparator, reviewer-inspection commit `273294467ce06429e6667ece7f5699f8678c9f4e` | Inspected as a formal-assurance architecture: a small trusted statement layer, an untrusted solution layer, exact statement comparison, Lean kernel replay, and an optional second kernel. This commit postdates the zeta tag and is the reviewer's architecture-inspection pin, not evidence of the comparator revision or executable bytes used by the self-recorded zeta runs. Inspection is not local replay or paper-to-Lean correspondence review. |

The executable firewall binds this visible local reviewed-source record. It is not a retained trusted statement or an external comparator replay:

- `quantifier_scope=liminf_T_to_infinity`
- `multiplicity_counted_denominator=N(T,2T)`
- `c1_star_definition=sqrt(2)*tan(1/sqrt(2))/(1+(1/sqrt(2))*tan(1/sqrt(2)))`
- `optimized_bound_definition=2-1/c1_star`
- `optimized_bound_decimal_prefix=0.672500703679...`
- `reviewed_paper_pdf_sha256=6792988e6cd0e17690621ce898abd5d534f98407741bc7cb14bbe7d07c77d72f`

The mathematical context and imported analytic inputs were checked against these versioned primary sources. Identity proves which bytes were reviewed, not that this note re-proved their theorems:

| Context source | Reviewed bytes | Role in the reviewed argument or priority boundary |
|---|---|---|
| Baluyot–Goldston–Suriajaya–Turnage-Butterbaugh, arXiv:2306.04799v1, exact PDF transport | 210,602-byte PDF; SHA-256 `133071c4d85c875fe9b1a7d4001f22d7174270ab8f1fa8e6cee6178d20276ee9` | Unconditional pair-correlation first/second-moment input; it neither supplies nor claims the new rank–trace readout. |
| Goldston–Suriajaya, arXiv:2501.14545v2, exact PDF transport | 421,275-byte PDF; SHA-256 `a615cac75b57445eb434881cde2e3ec4e016b5f977928bae2dafbdb906b1ae58` | Corrects a proof route in arXiv:2306.04799 while preserving the relevant pair-correlation result/applications; supplies thin/fixed-box conditional two-thirds and `0.6725` context. The new paper also rederives needed finite-family errors rather than importing this as an opaque black box. |
| Goldston–Suriajaya, arXiv:2511.20059v2, exact PDF transport | 345,194-byte PDF; SHA-256 `7b4f638cbd0438123b7a54869fc998fc3d4dee9b74572c04cef9da0463ed4c6f` | Records the open removal-of-box framing and a conditional diagonal route. |
| Goldston–Suriajaya, arXiv:2603.28104v1, exact PDF transport | 339,445-byte PDF; SHA-256 `67866b1ee3f9999c4673e78d4a0e81fcb97584a5e4221af8d4b02b10d3ce01c4` | A narrow-box route used for context, not an unconditional substitute. |
| Related pair-correlation context, arXiv:2503.15449v4, exact PDF transport | 339,013-byte PDF; SHA-256 `9b9f3c66c24a1a923e84150cd35e1e45613c59aa5ed894d7a71d84fc440e4c91` | PCC-conditional near-total/simple-on-line context, not an input that makes the new result unconditional. |
| Bombieri, "Remarks on Weil's quadratic functional in the theory of prime numbers" | 710,042-byte PDF; SHA-256 `20bd544fc5297766966630092aba4c1e10c6a7e663d14be89bbe1d0c8220b7fd`; exact reviewed PDF transport | Prior finite-matrix negative-index observation. The PDF transport was HTTP after its HTTPS certificate failed, so it receives reviewed-byte identity only: neither authenticated-source identity nor transport-authentication credit. |

One non-primary contextual reply by a third party, posted on X on 10 August 2026 at 19:08 UTC, observed that the constant and analytic ingredients had appeared under a closeness-to-line premise. The paper's Section 7.4 and the versioned predecessor sources above, not the reply, carry the mathematical support for that boundary.

## Mathematical method reconstructed from the paper

The claimed advance is unconditional and asymptotic. It does not prove the Riemann hypothesis and does not classify the uncertified remainder. In the paper's normalization, the argument is:

Write $N(T,2T)$ for all nontrivial zeros in the dyadic ordinate window, counted with multiplicity; $N_0^*(T,2T)$ for distinct zeros on the critical line; $N_0^s(T,2T)$ for simple zeros on the line; and $N_d(T,2T)$ for distinct zeros overall. Every proportion stated below uses the common multiplicity-counted denominator $N(T,2T)$, not the corresponding distinct population.

1. Compress Weil's Hermitian form to a finite critical-density Gabor test family and call the full matrix $G$. Split by zero ordinate as $G = A + E$, where $A$ is the contribution from the expanded central height window and $E$ is the tail. In the paper's fixed units, $\widehat{G} = G/(aL^2)$, $\widehat{A} = A/(aL^2)$, and $\widehat{E} = E/(aL^2)$.

2. Proposition 4.1(ii) gives $\widehat{A} = P + Q$. Critical-line zeros contribute positive-semidefinite rank-one terms to $P$. Before evaluation pullback, each functional-equation pair $\{\rho, 1-\overline{\rho}\}$ off the line supplies a hyperbolic block of signature $(1,1)$. The coefficient-space $Q$ is the pullback of their direct sum, so the load-bearing conclusion is $n_+(Q) \le p$, not that $Q$ retains exactly $p$ such blocks. The tail $E$ is controlled separately.

3. Use the new rank–trace inequality: if $P \succeq 0$, $\operatorname{rank} P \le r$, and the positive index $n_+(Q) \le b$, then for every $c > 0$,

$$\|P+Q\|_F^2 \ge c \operatorname{tr} P - \frac{c^2 r}{4} + 2c \operatorname{tr} Q - c^2 b.$$

   The proof separates the positive and negative spectral parts of $Q$, applies von Neumann's trace inequality, and completes scalar squares. Equality is attained by $P = (c/2)\Pi_1$ and $Q = c\Pi_2$ for orthogonal projections of ranks exactly $r$ and $b$, $\Pi_1\Pi_2 = 0$, when the ambient dimension permits $d \ge r + b$. This is a finite linear-algebra theorem; the number-theoretic content lies in the preceding block interpretation and following moment input.

4. Transfer the central-matrix conclusion to the full matrix with

$$|\operatorname{tr} \widehat{E}| \le \|\widehat{E}\|_1$$

   and

$$\|\widehat{A}\|_F \le \|\widehat{G}\|_F + \|\widehat{E}\|_1.$$

   On the prime side, for its specific kernel and truncation errors, the paper rederives the same unconditional Montgomery/BGSTB moment evaluation from the explicit formula, Chebyshev–Mertens estimates, and Montgomery–Vaughan. That analytic content is prior art for novelty purposes but is not imported as an opaque theorem in the paper/Lean route. For bandwidth $0 < \lambda \le 1$ it obtains $\operatorname{tr} \widehat{G} = N + o(N)$ and $\|\widehat{G}\|_F^2 = (1/\lambda + \lambda/3)N + o(N)$ after the fixed normalization.

5. Combine that transfer, the block bookkeeping, $c = 2$ specialization, and moments, then let $\lambda \uparrow 1$. This gives lower asymptotic proportions $2/3$ for distinct critical-line zeros and for simple critical-line zeros, and $5/6$ for distinct zeros. The optimized Montgomery–Taylor

window uses $c_1^* = \sqrt{2}\tan(1/\sqrt{2})/(1 + (1/\sqrt{2})\tan(1/\sqrt{2}))$ and gives $2 - 1/c_1^* =$ `0.6725007 03679...` for the first two proportions and $(3 - 1/c_1^*)/2 =$ `0.836250351839...` for the third. These are liminf/epsilon-form constants, not finite observed fractions. In particular, `0.6725008` is not the reviewed constant.

Theorem E gives the corresponding $H(\lambda)$, $H_d(\lambda)$, and $F(\lambda)$ bounds, including the two-thirds simple/on-line and five-sixths distinct consequences and the optimized Theorem D constants, for each fixed primitive Dirichlet $L(s, \chi)$ with fixed modulus $q$ and primitive $\chi \pmod q$ once $T \geq T_0(\lambda, q)$. This fixed-character theorem does not establish a hybrid result uniform in growing $q$, and no such extension is credited here.

For the simple/distinct strengthening, regroup $\widehat{A} = P_1 + Q'$ with $P_1$ the simple on-line terms and $Q'$ the multiple on-line plus off-line terms. Then $\operatorname{rank} P_1 \leq s_1$, $\operatorname{tr} P_1 \leq s_1$, and $n_+(Q') \leq s_2 + p$. The $c = 2$ lemma gives $3s_1 + 4s_2 + 4p \geq 4 \operatorname{tr} \widehat{A} - \|\widehat{A}\|_F^2$; combined with $N(I') \geq s_1 + 2s_2 + 2p$, this yields the simple-on-line and distinct bounds. This regrouping and the tail transfer are load-bearing; neither may be hidden inside a single informal $P + Q + E$ slogan.

The new ingredient is best described, subject to a complete priority review, as a new linear-algebraic readout of published analytic first- and second-moment input. It is not a new pair-correlation estimate, not the first matrix or inertia view of Weil's form, not the original discovery of the conditional `0.6725` constant, and not an RH proof. First-two-moment information alone would be insufficient; the zero-side signature and count interpretation is load-bearing.

The stated method has a real ceiling. A model with $2N/3$ orthogonal simple on-line eigenvalue-one contributions plus $N/6$ on-line doubles with eigenvalue two has trace $N$, squared Frobenius norm $4N/3$, simple fraction $2/3$, and distinct fraction $5/6$, saturating the simple/distinct bookkeeping. Replacing the doubles by shallow off-line pairs instead saturates the $2/3$ critical-line-distinct bound but not the total-distinct $5/6$ bound. The unconditional prime-side evaluation is scoped to $\lambda \leq 1$; extending the useful bandwidth would require prime-pair/Hardy–Littlewood-strength input not supplied here, and the higher moments currently available do not improve the useful bandwidth certificate. The conclusions are asymptotic liminf statements and the paper does not report a computed numerical value of $T_0(\lambda)$ or a finite observed proportion. An $o(N)$ off-line population can remain invisible, so the method supplies no RH route. It uses neither a new mollifier, zero-density theorem, nor zero-free region. Paper Remark 1.1 states that no configuration-by-configuration certificate using only mean density, bandwidth-one pair correlation, and multiplicity integrality can exceed about `0.68185` simple-on-line. The separately stated Lean certificate is $0.6818287 + 2.55 \times 10^{-6}(|r'(1)| + \int |r''|)$ under its displayed `EnclOK` grid-enclosure premise. This bandwidth-one ceiling is distinct from Proposition 7.4's dimension cap and separate from Theorems A–E; it does not enlarge the unconditional headline or transfer to PID. The paper's displayed simple/distinct regrouping uses the $c = 2$ route above, while the Lean repository also records a $c = 3$ multiplicity route to the same five-sixths conclusion. These are distinct proof routes and must not be described as line-by-line identical.

The direct pid-rs comparison target was the accepted bounded claim in `claims/SX-COUNT-ATOM-BRIDGE-001/claim-v2.md`: supplied natural counts on one fixed two-source finite categorical key space, with all 24 informative, misinformative, and signed-net cumulative/atom coordinates obtained from keyed event probabilities, logarithmic rational products, and the fixed four-node Möbius transform. That claim contains no Hermitian compression, rank, inertia, trace, or Frobenius semantics. It also excludes row-to-count refinement, Rust and binary64 behavior, population/sampling claims, continuous, quantized, and $I_{\min}$ routes, and higher-source lattices. This exact target boundary, rather than vocabulary similarity, is the basis of the no-mapping decision below.

## Discovery, verification, and communication methods

The institutional landing page describes two sessions, about 31 million output tokens, nearly 60 subagents, 2,400 shell commands, hundreds of Python scripts, thousands of numerical checks, and 54 downloaded papers. The model-authored appendix separately reports on the order of 1,000 short-lived workflow agents in the earlier session, 650 attempted ideas, 106 retained survivor-ledger entries, and 60 launches in the later campaign of which 58 actually ran. These are source-specific, non-interchangeable populations; none is an independently verified count and they must not be added or substituted for one another.

The initially circulated draft already contained the two-thirds bound for distinct critical-line zeros, but only one-half for simple critical-line zeros and three-quarters for total distinct zeros. Later model referees found the regrouping that raised the latter two to two thirds and five sixths. Draft circulation and polished exposition therefore were not final verification.

The process appendix's event-level attribution is more precise than a single-agent discovery story:

| Actor or route | Recorded contribution | Evidentiary boundary |
|---|---|---|
| Jarred Sumner | Set the direction, supplied encouragement, and asked whether the surviving one-half route could reach two thirds. | Human target selection and coordination are not proof steps, but they materially shaped the search. |
| Coordinator model | Framed objects and briefs, supplied an integer-template analogy, dispatched joint-specific reviewers, checked persisted files, and made the stop/handoff decision. | Coordinator validation shares model lineage and source dependencies with the generated routes. |
| E2 | Rejected the requested negative-index direction as vacuous and found the positive-index dual giving an initial one-half route. | Initial route, not the final two-thirds inequality. |
| A, B, and D review routes | A proposed a coefficient-coordinate/drop-the-mass-matrix repair; B separately detected the mass-matrix issue and checked the prime-side route; D made A's repair rigorous, including the exact Poisson and tail details. | Distinct assignments reduce local copying risk but are same-workflow evidence. |
| E2-pairs | Rejected stronger free-phase/free-mass/blockwise conjectures, proposed and proved the global rank–trace inequality, and supplied the simplified application bookkeeping. | Primary model route for the new lemma/application. |
| Y and later model referees | Y reconstructed the lemma and equality case; later model referees found the simple-on-line and distinct-zero regrouping absent from the circulated draft. | Reconstruction and strengthening are correlated model review, not institutional independence. |
| Numerical scripts | Generated conjectures and falsification pressure through random and optimization-guided examples. | They did not prove the matrix inequality or analytic estimates. |
| Paper-writing route | Organized the surviving argument and boundaries into a draft. | Exposition is not an additional verification route. |

| Actor or route | Recorded contribution | Evidentiary boundary |
|---|---|---|
| Eric Easley and formal routes | Orchestrated the Lean development and comparator architecture. | Formal acceptance remains conditional on statement correspondence and exact toolchain/kernel replay. |
| Levent Alpöge and Ralph Furman | Studied the result, placed it in context, and took responsibility for communication; Conrey and Goldston also read and commented on short notice. | Human expert reading is material review, but no retained external journal-referee report was observed. |

All model reviews remain correlated. The table records contributions without converting role labels into independent proof credit.

For this repository review, a separately tasked reconstruction derived the finite rank–trace lemma from spectral splitting, von Neumann's trace inequality, and scalar completion of squares, then substituted the exact equality case. Twenty thousand random complex-Hermitian trials were also used only as uncommitted scratch falsification pressure; they have no replay receipt and no proof credit. This reconstruction did not re-prove the analytic prime-side inputs or create an institutionally independent route.

| Phase | Retained method or process lesson | Transfer class | Boundary that remains |
|---|---|---|---|
| Epistemic contract | Require honest reporting, retained failures, controls, and the first unjustified step before search begins. Encouragement may license breadth; it cannot raise evidentiary weight. | PID-engineering/formal-workflow only | Confidence and effort are not proof. |
| Broad search | Assign distant approaches and preserve negative results. The successful route appeared after many routes correctly terminated at known barriers. | PID-engineering/formal-workflow only | Many attempts do not make the surviving attempt statistically validated or complete. |
| Sideways discovery | The productive agent was asked for one index bound, found that direction vacuous, and examined the nonvacuous dual. A later global rank–trace inequality emerged after stronger blockwise conjectures failed. | PID-engineering/formal-workflow only | A narrative about surprise is provenance, not a mathematical step. |
| Controls | Exercise candidate mechanisms on nearby objects where the analogous target is known false, and require the mechanism to under-certify rather than over-certify there. | PID-engineering/formal-workflow only | Zeta-specific controls do not become PID controls; each local claim needs its own adjacent worlds. |

| Phase | Retained method or process lesson | Transfer class | Boundary that remains |
|---|---|---|---|
| Joint-specific review | Give separate reviewers named failure joints: localization, hidden use of RH on the prime side, block/inertia algebra, technical approximation, and a "proves too much" route. Ask fresh reviewers to reconstruct key lemmas from statements rather than merely reread the claimant. | PID-engineering/formal-workflow only | Same-model and same-institution reviews are correlated and do not establish institutional independence. |
| Numerical work | Use random and optimization-guided matrices to discover or falsify inequalities, then replace successful experiments with an exact proof. | PID-engineering/formal-workflow only | Thousands of successful floating-point cases do not prove a universal inequality. |
| Failure recovery | E2-pairs' API run failed after the lemma/proof file was persisted but before its application was complete. The coordinator inspected the retained file, resumed the same agent with a seven-item checklist, and launched a separate critical X route and statement-blind Y reconstruction. | PID-engineering/formal-workflow only | Resume preserved continuity; it inherited the original route's dependencies and was not an independent proof. |
| Formal assurance | Separate trusted definitions/statements from untrusted proof implementation; compare exact theorem statements; audit axioms; replay with the primary kernel and, where available, a second kernel. | PID-engineering/formal-workflow only | Kernel acceptance proves the formal statement from its formal premises, not the paper correspondence, analytic source theorems, implementation refinement, or publication priority. |
| Saturation and handoff | Stop internal review when the remaining passes share the same information and escalate to domain specialists. Record who reviewed mathematics, context, communication, and formalization. | PID-engineering/formal-workflow only | Human reading, institutionally posted communication, arXiv deposit, journal review, and accepted publication are different states. |
| Communication | Publish the theorem, informal explanation, process appendix, selected transcripts, formal source, and nonclaims as separately typed artifacts. | PID-engineering/formal-workflow only | A screenshot, landing page, acknowledgement, or edited transcript is not an additional proof route. |

For pid-rs, every external-result intake must record the **mathematical technique**, the broader **method** that makes the technique applicable, the **discovery process**, the **verification process**, and the **publication state** separately. A useful workflow may transfer even when no mathematical

object transfers. Conversely, a shared mathematical phrase does not transfer a method or theorem.

### Thirty-four-lens PID transfer audit

Separately tasked, model-isolated reviews and a separate reconstruction found no direct mapping to the bound pid-rs claim above. They are correlated, not dependency-disjoint evidence. This is a disposition on the reviewed mappings, not a theorem that no future PID construction can exist; lens 6 remains `OPEN`.

| Lens | Disposition and reason | Transfer class |
|---|---|---|
| 1. Object identity | `NEGATIVE`: zeta zeros, analytic multiplicities, and test functions are not finite PMFs, local information values, or PID atoms. | Non-transferable |
| 2. Hypotheses | `NEGATIVE`: functional equation, Weil explicit formula, and prime-side pair-correlation asymptotics have no supplied PID counterpart. | Non-transferable |
| 3. Conclusion type | `NEGATIVE`: an asymptotic zero-count proportion is not a PID identity, estimator guarantee, error bound, or calibration statement. | Non-transferable |
| 4. Quantifiers and limits | `NEGATIVE`: $T \to \infty$ and bandwidth $\lambda$ do not map to sample size, KSG $k$, alphabet size, or quantizer refinement. | Non-transferable |
| 5. Units and normalization | `NEGATIVE`: spectral normalization and zero multiplicity do not map to nats, log-ratios, PMF mass, or an SxPID gauge. | Non-transferable |
| 6. Matrix construction | `OPEN`: no canonical PID Hermitian compression with the required semantic interpretation is known here. | PID-mathematical (only if items 1–9 below are proved) |
| 7. Block semantics | `NEGATIVE`: an on/off-critical-line functional-equation pair is not redundancy/unique/synergy or informative/misinformative pairing. | Non-transferable |
| 8. Positivity | `NEGATIVE`: signed-net SxPID atoms may legitimately be negative; positive index is not atom nonnegativity. | Non-transferable |
| 9. Rank interpretation | `NEGATIVE`: no proved PID property is counted or bounded by the rank of the proposed matrix. | Non-transferable |
| 10. Inertia interpretation | `NEGATIVE`: no positive-index theorem counts supported events, lattice nodes, atoms, or valid estimator cases. | Non-transferable |
| 11. Trace interpretation | `NEGATIVE`: no trace identity is proved equal to the named PID functional or estimator target. | Non-transferable |
| 12. Frobenius interpretation | `NEGATIVE`: first two matrix moments do not determine rank or inertia without the zeta-specific block theorem. | Non-transferable |
| 13. Lattice and source order | `NEGATIVE`: matrix rank/inertia is not the antichain poset or its zeta/Möbius transform. | Non-transferable |

| Lens | Disposition and reason | Transfer class |
|---|---|---|
| 14. Symmetry | `NEGATIVE`: functional-equation conjugate pairing is not a source permutation, target relabeling, or event-union invariance. | Non-transferable |
| 15. Population versus empirical | `NEGATIVE`: the zero theorem has no empirical-PMF/population-functional distinction and establishes neither route. | Non-transferable |
| 16. Sampling and uncertainty | `N/A`: the deterministic analytic theorem has no row law, split, confidence, or multiplicity-control premise; therefore it cannot calibrate one. | Non-transferable |
| 17. Formal correspondence | `NARROW`: trusted-statement comparison is useful architecture, but a formal theorem is not a paper-to-Lean or Lean-to-Rust refinement theorem. | PID-engineering/formal-workflow only |
| 18. Kernel and toolchain | `NARROW`: exact pins and axiom inventories transfer as controls. The external repository's release-candidate toolchain differed from the target worktree's pinned Lean 4.32.0 finite-convergence toolchain at source-review time. | PID-engineering/formal-workflow only |
| 19. Route diversity | `NARROW`: role separation and statement-blind reconstruction are useful; same-model reviewers share lineage and sources. | PID-engineering/formal-workflow only |
| 20. Counterexamples and controls | `NARROW`: the control-world pattern transfers, but the actual control objects must be PID-specific. | PID-engineering/formal-workflow only |
| 21. Numerical stability | `NEGATIVE`: exact Hermitian algebra and asymptotic estimates prove no binary64 log, summation, eigenvalue-gap, overflow, or platform property. | Non-transferable |
| 22. Resources and implementation | `OPEN`: no Rust algorithm, complexity bound, wrapper parity, or resource contract follows from the proof. | PID-engineering/formal-workflow only (after a separate specification) |
| 23. Citations and novelty | `NARROW`: source/application mapping and old-input/new-readout distinctions transfer; the priority claim remains bounded by the reviewed search. | PID-engineering/formal-workflow only |
| 24. Publication and authority | `NARROW`: discovered, drafted, model-reviewed, human-read, formally encoded, institutionally posted, peer-reviewed, and published are distinct states. | PID-engineering/formal-workflow only |
| 25. Local-to-global | `OPEN`: no common typed map or commuting diagram is supplied, so neither commutation nor noncommutation between the zeta operations and PID pointwise averaging/Möbius inversion is established; a PID-specific aggregation/transport theorem would be required. | PID-mathematical (only if items 1–9 below are proved) |

| Lens | Disposition and reason | Transfer class |
|---|---|---|
| 26. Exact versus numerical | `NARROW`: conjecture search and numerical falsification are useful only when replaced by exact or enclosed PID evidence. | PID-engineering/formal-workflow only |
| 27. Constructive content | `NEGATIVE`: the zeta theorem constructs no PID object, estimator, certificate, or executable algorithm. | Non-transferable |
| 28. Proof-assistant portability | `NARROW`: statement/solution separation and kernel replay can be reimplemented at pid-rs' pinned toolchain, but theorem semantics do not transfer with the code pattern. | PID-engineering/formal-workflow only |
| 29. Search space | `NARROW`: a retained route/failure ledger and constraint-ablation record improve search discipline; token or route counts do not prove completeness. | PID-engineering/formal-workflow only |
| 30. False analogy | `NEGATIVE`: shared words such as information, rank, lattice, positive, negative, and moment do not establish a typed correspondence. | Non-transferable |
| 31. Estimator versus functional | `NEGATIVE`: the analytic zero theorem is neither the finite categorical functional nor a sampling theorem for a pid-rs estimator. | Non-transferable |
| 32. Finite versus infinite support | `NEGATIVE`: an asymptotic zero multiset and finite compression do not map to finite PMF support or continuous-law support. | Non-transferable |
| 33. Categorical versus continuous | `NEGATIVE`: neither the MGW finite categorical construction nor the Ehrlich continuous construction appears in the zeta objects. | Non-transferable |
| 34. Actionable workflow | `NARROW`: the review-joint matrix, comparator architecture, failure resume, and handoff rule are actionable only as process controls; any mathematical use remains gated by items 1–9 below. | PID-engineering/formal-workflow only |

Three exact countercontrols explain why a lexical matrix analogy is insufficient:

- Let $S_1, S_2, T$ be independent fair Rademacher variables, and compare this with $T = S_1 S_2$. The covariance matrix is the identity in both cases, while $I((S_1, S_2); T)$ is respectively $0$ and $\ln 2$. This covariance matrix, and any linear Gram representation determined only by those same first and second moments, cannot recover even total mutual information here, much less a PID. This does not claim that every nonlinear characteristic-kernel sample Gram loses the full distribution.
- The Hermitian matrices $\mathrm{diag}(3, 4, -3, -4)$ and $\mathrm{diag}(5, -5, 0, 0)$ both have trace zero and squared Frobenius norm fifty, but different rank and inertia. First and second moments cannot replace a block-to-count theorem.
- Congruence by $\mathrm{diag}(2, 1)$ sends $\mathrm{diag}(1, -1)$ to $\mathrm{diag}(4, -1)$. Inertia is preserved while trace and Frobenius norm change, so a signature analogy without a fixed coordinate and normalization theorem is insufficient.

A future PID use must close every item below for one named construction and conclusion:

1. `M1_domain_to_hermitian`: define $F : D \to \mathrm{Herm}(d(x))$ on the exact PID domain $D$;
2. `M2_decomposition`: prove $F(x) = P(x) + Q(x) + E(x)$ with a bounded error term;
3. `M3_positive_semidefinite_part`: prove $P(x) \succeq 0$;
4. `M4_rank_semantics`: prove the rank bound and a noncircular implication from that rank to the named PID conclusion;
5. `M5_positive_index_semantics`: prove a positive-index bound for $Q(x)$ with explicit PID complement semantics;
6. `M6_coordinates_scale_units`: fix coordinates, scale, units, source order, event semantics, and Möbius convention;
7. `M7_trace_frobenius_total_relation`: prove PID-specific trace and Frobenius bounds plus the required total-count relation;
8. `M8_error_budget`: enclose every tail, approximation, representation, and numerical error below the strict margin;
9. `M9_transport_to_claimed_pid_object`: transport the matrix conclusion back to the exact functional, estimator, and implementation claimed.

The corresponding executable negative-control plan is deliberately PID-specific and carries no positive validation credit:

| Gate or mutation family | Concrete check | Evidentiary class and required failure |
|---|---|---|
| `scripts/check-zeta-pid-transfer-firewall.py` | Pure-standard-library exact checks for independent-versus-XOR equal covariance with different mutual information, equal trace/Frobenius matrices with different rank/inertia, and congruence-preserved inertia with changed moments. | PID-engineering/formal-workflow only; any accidental PID inference must fail. |
| `scripts/check-zeta-pid-transfer-firewall-self-test.py` mapping mutations | Omit or weaken each of M1–M9, substitute $\lambda$ for KSG $k$, and insert the circular diagonal embedding of already computed atoms. | PID-engineering/formal-workflow only; every incomplete or circular mapping must be rejected at its registered causal code. |
| Reviewed-source-record mutations | Change the recorded quantifier scope, multiplicity-counted denominator, exact symbolic constant definition, decimal display prefix, or reviewed paper digest. | PID-engineering/formal-workflow only; this checks a local source record, not an external comparator replay or trusted-statement digest. |
| Workflow-PDF semantic sentinels | Require the typed chain $G = A + E$, $\widehat{A} = P + Q$, the `0.672500703679...` constant, and the M1–M9 abstention rule; reject the old conflated $P + Q + E$ shorthand as a complete proof description. | PID-engineering/formal-workflow only; publication must not erase the transfer firewall. |

The publication firewall therefore retains the literal source/PDF sentinels `G=A+E`, `Ahat=P+Q`, `0.672500703679...`, and `M1-M9-incomplete=>abstain`.

Absent any one item, the route must abstain. Diagonalizing already computed atom values would be circular and would define at most a project diagnostic, not a theorem about how the atoms

arise. Accordingly this review adds no PID method, estimator, theorem, numerical-stability result, or validation row. Its net-new workflow controls are: joint-specific review assignments; an explicit internal-review saturation and specialist-handoff rule; trusted-statement/solution separation with statement comparison and optional second-kernel replay; raw-versus-edited transcript provenance; and event-level idea-attribution corrections. Existing failure ledgers, critical-cut accounting, same-lineage warnings, and durable resume rules remain authoritative rather than being duplicated under new names.

## 3.5 AI model operating protocol

### 3.5.1 Applicability and risk

A **major claim** is any claim that changes a public method definition, theorem, estimator result, validated-status statement, release gate, or downstream readiness decision. A **high-consequence claim** is a major claim that could materially affect scientific inference, safety monitoring, mission or authorization policy, or a silent false numerical result. Treat an uncertain classification as the higher class until the claim packet justifies a downgrade.

Major claims require recorded role overlap, a claim packet, a counterexample route, the 20-row scientific baseline below, at least 30 explicitly named object-specific sub-lenses, and an explicit independence vector for every route pair. Do not compress functional/mechanism, epistemic/dependency, and institutional/custody independence into the word "independent." High-consequence claims additionally require a dependency-disjoint epistemic route at every load-bearing shared bridge, specialist human review, and an explicit consumer no-go condition while any applicable gate is open.

### 3.5.2 Candidate/judge isolation and research-integrity controls

This protocol applies prospectively to research/scientific or claim-adjudicating attempts opened after this policy revision. Historical claims receive no retroactive compliance, custody, or independence credit merely because their artifacts are later described in this vocabulary. Deterministic execution of an already-fixed operational custody contract follows that contract's own pre-bound checker and receives no scientific credit; routine receipt generation is not a new scientific candidate attempt.

Before any proof, certificate, implementation, benchmark, or model candidate sees an attempt, seal and record a versioned task packet–publishing only its permitted projection or commitment–containing the exact claim/statement, definitions, imports, permitted axioms and trust routes, source and toolchain identities, candidate write scope, acceptance predicate, resource budget, stopping rule, and selection rule. The packet is frozen for that attempt, not immutable forever. Any semantic or acceptance change creates a new revision and leaves the earlier attempt at its original disposition.

Any output produced before its attempt is registered and its packet is sealed permanently loses preregistered, selection-unbiased, and strict-confirmatory credit. Preserve it as retrospective exploratory evidence rather than erasing it. Naming or registering the output after selection does not cure the missing boundary, and rerunning it against the same exposed data or judge does not cure adaptive selection. A deterministic formal or exact artifact may later receive credit only for the separately checked predicate; an empirical promotion needs a candidate-inaccessible confirmation route or a declared selection-aware design.

The candidate may write only its declared payload. A separately instantiated judge, built from pre-bound bytes outside candidate authority, must reject any change to the claim, definitions, imports, axiom policy, verifier, tests, fixtures, reference values, or trust surface. It must also

reject undeclared `sorryAx`/axioms, unsafe or partial routes, code generation or external execution outside the packet, network or secret access outside the frozen allowlist, and source/toolchain drift. Compilation and proof checking run in a fresh confined environment; imported modules are replayed when import replay is claimed. An unavailable, crashed, timed-out, or incomplete judge is `not-run` or `failed`, never a degraded pass. A colocated hash detects drift but does not prove that the packet predated candidate access or that a separate institution controlled it.

Retain an append-only attempt ledger. Every attempt receives a durable minimum entry: task revision, purpose and route, candidate and judge identities, model/system/tool identity when knowable, terminal status, selection or rejection decision, reopen condition, and an output locator/identity or explicit omission. Retain the full permitted input or privacy-preserving projection, seed when exposed, decisive output/diff, resource use, and verifier receipt for every promoted, adjudicating, or load-bearing failed or withdrawn attempt. A valid unselected attempt that entered a best-of-$N$ or any other selection decision is load-bearing: retain its decisive permitted output and verifier receipt. If that record is unavailable, mark the selection route incomplete and grant it no strict selection credit. Other routine outputs are retained only when useful and permitted; record bounded omission or redaction for privacy, licensing, security, provider, or volume reasons instead of claiming completeness or replay. Retain failures, withdrawals, and unselected valid candidates; append corrections rather than rewriting them.

### 3.5.3 Problem-specific autoresearch and durable publication

pid-rs uses an **autoresearch-like** loop only in a bounded methodological sense: propose a typed hypothesis, run a predeclared falsification or evaluation route, retain the result, update a small beam of live hypotheses, and let a human decide what to revise, promote, or stop. Sankalp's GPU-kernel worklog reports that a problem-specific harness, an attempt log, profile-driven questions, and a three-to-five-family candidate beam accompanied its move beyond a single-incumbent local maximum (Sankalp, "Auto-research with codex," 8 July 2026, inspected 15 August 2026). The retrospective account does not isolate the causal contribution of the beam, adviser, agent, or harness. It is an attributed engineering case study, not authority for PID validity, reproducibility, or optimal research design; pid-rs adopts only the lessons that survive its own claim analysis. The URL and inspection date are source locators only; this repository retains no authenticated provider record or archival copy of the article bytes.

The analogy stops where a scientific claim ceases to be a fixed-score optimization problem. A leaderboard can rank one accepted output by one scalar. PID research may simultaneously change a functional, estimator, support class, numerical representation, proof obligation, benchmark, selection rule, and downstream interpretation. Therefore no single "best score," passing test, formal proof, model vote, or publication status may choose across those objects. The scientific loop uses a typed obligation-and-evidence vector and freezes the task and judge before each scored attempt.

Repeated adaptive evaluation also creates a selection problem. Reusing the same samples, simulations, benchmark cases, prompts, or visible verifier failures to choose among many candidates makes that route development evidence. A promoted empirical claim therefore needs either (a) candidate-inaccessible confirmation–for example untouched data or sealed cases or seeds sampled from a predeclared generator–or (b) a declared selection-robust route–for example exhaustive validation over the entire declared finite domain or valid sequential/multiplicity-adjusted inference. Every prior exposure and shared dependency must be recorded. Deterministic theorem checking does not create a sampling $p$-value, but repeated feedback can still optimize against a weak or candidate-influenced statement and judge; pre-candidate target and verifier custody remains mandatory.

| Lens | Transfer from the engineering case study | pid-rs boundary |
|---|---|---|
| Objective | Keep the objective quantitative where possible. | Use an obligation vector, not a scalar that trades correctness or correspondence for speed or convenience. |
| Search | Maintain exploit, near-miss, and structural/high-risk beams. | Each beam names one exact PID/method object and revision; it cannot silently change the estimand or judge. The reported three-to-five size is anecdotal, not a universal optimum. A near-miss has a localized diagnosed failed obligation or robust partial property, not merely a score or significance value close to a threshold after adaptive evaluation. |
| Feedback | Prefer completed evaluator output to intuition. | Evaluator output is evidence only for its frozen predicate and cannot validate the predicate itself. |
| Failure memory | Log failed and inconclusive attempts. | Retain load-bearing failures and all selection-bearing valid alternatives; distinguish reject, timeout, crash, and not-run. |
| Diversity | Seek structurally different ideas when local tuning saturates. | Record shared premises and dependency overlap; model/session diversity is not epistemic independence. |
| Human steering | Use domain knowledge to identify missing questions. | Human judgment is a named disposition with scope, not an unexplained override or retrospective target change. |
| Selection | Promote only after comparable evaluation. | Freeze the candidate-admission and adaptation policy, budget, stopping rule, and selection rule; log every adaptation and use fresh confirmation or sequentially valid inference. |
| Generalization | Recheck across shapes and input regimes. | Recheck across declared alphabets/supports, sample roles, gauges, boundary cases, and implementation representations; do not infer population validity from fixtures. |
| Security and integrity | Keep the evaluator outside the candidate's write scope. | Pre-bind statements, definitions, imports, axioms, tests, reference values, and judge bytes; reject undeclared trust expansion. |
| Publication and durability | Keep attempt logs and promoted candidates. | Make each coherent accepted milestone reachable from the reviewed remote main branch, then bind publication artifacts and archival locators separately. |

The research loop is therefore:

1. freeze one typed claim packet and its judge;
2. maintain a small, genuinely diverse beam of hypotheses for that same object;
3. run the cheapest high-information falsification check, negative control, or counterexample search before expensive proof, certification, or hosted evaluation;

4. record every terminal disposition and every candidate that affected selection;
5. promote only a coherent packet whose evidence classes remain separate;
6. commit and push the packet, result, disposition, and synchronized publication closure in small reviewable milestones; and
7. reopen the science only under a new revision with an explicit reason.

### 3.5.4 Why branches and worktrees exist, and why they create a closure obligation

Git supports multiple linked worktrees so that more than one branch or detached commit can be checked out at the same time. Git presents this as a way to perform experimental or urgent work without disturbing an existing checkout (Git worktree documentation). In this project, that isolation has five legitimate uses:

| Reason for a separate lane | What the lane protects | What the lane does not prove |
|---|---|---|
| Explore a theorem, estimator, counterexample, or document without changing stable code. | `main` remains reviewable while the result is uncertain. | The experiment is correct, useful, or ready to merge. |
| Freeze one claim revision and its exact inputs while other work continues. | A proof, checker, and evidence packet can refer to one stable source tree. | The formal statement matches the paper or the intended PID object. |
| Give parallel actors separate write scopes. | Two tasks do not overwrite the same working files by accident. | The actors are scientifically independent. They can still share prompts, models, sources, code, and mistakes. |
| Reproduce a result from a clean checkout. | Hidden tracked edits and many ambient-state errors become visible. | The run is hermetic, portable, statistically valid, or independent of caches and external tools. |
| Preserve a rejected or restricted route outside the accepted surface. | Negative evidence can remain recoverable without entering executable APIs. | A side branch is accepted, public, permanent, or reachable from remote `main`. |

A branch is a mutable reference to a commit. Each linked worktree has its own checkout, index, and `HEAD`. Linked worktrees share the common object database and most refs; pseudorefs and some per-worktree refs remain separate. A separate build root and bounded write scope exist only when project convention or configuration supplies them. Worktrees are therefore isolation tools, not independent repositories and not durable storage by themselves. A clean committed tree does not include uncommitted tracked edits, untracked or ignored files, the index, stashes, hooks, or repository configuration. Git's bundle documentation states the same boundary: a bundle transfers selected refs and reachable objects, but not this other local state (Git bundle documentation).

#### Failure-to-repair map

| Failure mode | Detection | Required correction |
|---|---|---|
| **Split truth:** useful work is distributed across several branches, worktrees, or dirty paths. | Inventory every common Git directory, worktree, ref, stash, alternate index, dirty path, untracked path, relevant ignored path, and active writer. Compare paths and blobs, not only branch tips. | Classify each useful fragment. Integrate the current form into one coherent claim packet, or retain it as labelled negative/recovery evidence. Never merge the whole stale branch merely because one file is useful. |

| Failure mode | Detection | Required correction |
|---|---|---|
| **Revision drift:** a checker names one theorem or artifact revision while the checked files belong to another. | Bind the checker source, artifact paths, theorem roster, toolchain, and digests. Mutate each revision pin and require failure. | Commit the mutually consistent checker and artifacts together. Re-run baseline and hostile self-tests. Do not weaken the pin or treat a passing theorem from another revision as equivalent evidence. |
| **Uncommitted-state loss:** a bundle or branch omits dirty, untracked, or ignored research bytes. | Compare the worktree and index with `HEAD`; enumerate untracked and relevant ignored paths; verify every manifest entry by mode, size, and digest. | Commit permitted bytes to an explicit archive or successor ref, or place protected bytes in an approved restricted store. A digest without a retrievable preimage is not recovery. |
| **Stale integration:** a bulk merge or directory copy restores an older formula, test, explanation, or generated artifact over a corrected version. | Compare each candidate path with the current mainline object and its semantic role. Re-run source-state, method-catalog, math, and document checks after reconciliation. | Port only the accepted path or coherent packet. Regenerate self-referential manifests and derived PDFs last. Preserve the older route only when it has historical or negative-result value. |
| **Concurrent writers:** two actors edit a shared source or generated output. | Record an owner for each mutable path and inspect live processes before freezing. Treat unexpected changes as a new source revision. | Stop writers, discard no bytes, reconcile both variants, and restart review from the new frozen tree. A council does not grant shared write authority. |
| **Cache or toolchain inheritance:** a clean worktree succeeds only because it sees an undeclared cache, binary, import, or object store. | Pin the interpreter, compiler, proof assistant, solver, dependencies, and relevant environment. Rebuild in a fresh route and use hostile absence/drift controls. | Declare the dependency or remove it. Repeat the exact gate in a clean local checkout and hosted CI. Never call a warm-cache run an independent reconstruction. |
| **Local/remote divergence:** a result is committed locally or pushed to a side branch but not accepted on remote `main`. | Query the remote ref directly. Record the observed old and candidate object IDs and prove the update is a fast-forward. | Push the exact candidate with an explicit expected-old lease, then query the remote again. The explicit lease fails if another writer advanced the ref (Git push documentation). |
| **Premature cleanup:** a worktree is deleted because it appears old, clean, or mostly superseded. | Require a process-ownership check, dirty/untracked/ignored disposition, path-level semantic comparison, verified bundle or successor, remote reachability, and a post-removal object inventory. | Retain on ambiguity. Remove a linked worktree with Git's worktree-removal command; remove its branch only after all retirement predicates pass. Delete reproducible caches last. |
| **Formal-scope inflation:** a checked algebraic theorem is described as probability, estimator, or application assurance. | Read the exact theorem statement, imports, axiom roster, source map, and nonclaims. Ask which random variables, laws, quantifiers, limits, and executable representations actually occur in the formal object. | Narrow the wording. Open separate obligations for paper correspondence, probability, estimator calibration, executable refinement, numerical stability, and consumer qualification. |

The revision-pin failure has a bounded repository instance. Archived primary-worktree commit `86faa9a0850ca416f54a467230106b01d4162687` preserves two untracked `KSG-INTEGER-HARMONIC-001` checkers that select a missing Lean `v3` path and the absent `ksg-local-bound-v3.smt2`, while the same snapshot contains the revision-4 Lean source and `ksg-local-bound-v4.smt2`. The exact hashes and custody boundary are recorded in `audit/evidence/worktree-and-branch-preservation-2026-08-27.md`. That mixed packet must fail. Pin drift alone does not show that the theorem is false or true; it shows that the named replay cannot establish the claim from that checkout. First adjudicate which revision is intended. The valid routes are to restore one complete revision-3 packet, migrate the complete packet to revision 4, or retire the lane as obsolete while preserving its evidence. The accepted source tree uses the coherent revision-4 route: `scripts/check-lean-ksg-integer-harmonic.py` selects `audit/formal/lean-ksg-harmonic/v4/PidKsgIntegerHarmonic.lean`, and `scripts/check-z3-ksg-integer-`

`harmonic.py` selects `ksg-local-bound-v4.smt2`. Checker pins, artifacts, theorem roster, and hostile controls move together and are replayed in ordinary and optimized modes. It is not correct to remove the revision check, accept whichever file exists, or edit the expected digest until the test passes. This is a certification defect, not automatically a mathematical defect.

### Exact closure sequence

Apply the following sequence to each lane. Do not skip directly from a green test to deletion.

1. **Name the lane.** Record its repository, common Git directory, worktree path, branch or detached `HEAD`, base commit, current commit and tree, owner, purpose, and allowed writers.
2. **Freeze it.** Stop writers and record tracked, staged, untracked, relevant ignored, conflicted, assume-unchanged, skip-worktree, sparse-checkout, alternate-index, submodule, and symbolic-link state. Record active processes that hold the path.
3. **Enumerate history and bytes.** List local and remote refs, stashes, branch-only commits, changed paths, modes, sizes, and digests. Inspect every partially useful branch at path and semantic level. An old branch can contain one current counterexample or proof witness.
4. **Classify every variant.** Use accepted science, open candidate, negative result, superseded-but-needed recovery state, restricted/local-only material, reproducible cache, or discardable temporary output. State why the classification is correct.
5. **Preserve before integration.** Give every unique or uncertain permitted payload a reachable commit/ref and a second durable copy. For a Git bundle, record its size and SHA-256, run Git's bundle-verification command, list its advertised heads, and perform a recovery drill in a clean repository. Preserve non-Git state separately because a bundle cannot contain it.
6. **Integrate from fresh `main`.** Port explicit paths or one coherent claim packet. Never use a wholesale stale-tree overlay. Reconcile formulas, assumptions, tests, docs, citations, and generated artifacts together.
7. **Regenerate derived state.** Synchronize canonical Markdown and TeX, rebuild PDFs, inspect rendered pages, regenerate source-state manifests and receipts, and re-run portability checks. Derived bytes must not become a competing editable source.
8. **Run the evidence layers.** Run mathematical, formal, exact-arithmetic, numerical, estimator, implementation, mutation, build, lint, documentation, PDF, and consumer gates that apply. Mark an inapplicable layer and state the reason. A skipped layer is not a pass.
9. **Commit and publish one coherent milestone.** Keep source, checker, schema, hostile controls, human narrative, PDF, and binding receipts together when splitting them would create a false intermediate state. Use unsigned, human-authored commits with no agent attribution.
10. **Advance `main` with concurrency protection.** Re-read the remote old object ID, prove the candidate is its descendant, and use an explicit expected-old lease. Fetch the result, update local `main` only from the observed remote object, and require local and remote IDs to agree.
11. **Verify the mainline event.** Run or observe the required hosted workflows on the exact `main` commit. A prior side-branch run is useful preflight evidence but is not the mainline publication event.

12. **Retire conservatively.** Delete a ref or worktree only after every accepted byte in that lane is reachable from remote `main`, every remaining adjudicating or selection/load-bearing byte (including a rejected route) remains retrievable under a typed negative/archive disposition, and every byte selected for destruction is separately established as non-load-bearing and outside the required retention scope with its rejection reason recorded. A rejection label or digest without a retrievable preimage never satisfies a load-bearing retention obligation. Re-run reachability, bundle, branch, worktree, process, disk, and cleanliness inventories after removal.

### Formal verification inside an isolated lane

An isolated checkout supports formal verification only when the evidence chain remains explicit. First, freeze the mathematical proposition with all variables, domains, distributions, support conditions, units, quantifiers, and boundary cases. Second, map each paper definition or repository-defined construction to the formal symbol that represents it. Third, pin the proof assistant or solver, dependency commits, imports, declarations, axioms, options, and checker bytes. Fourth, run the kernel or solver and inventory the theorem and axiom output. Fifth, preregister a suite of well-formed fault mutations. Name the obligation that each mutation violates and the causal rejection that is expected; require those fault mutations to fail for the named reason. Equivalent or stronger statements and irrelevant import changes are not fault mutations and need not fail. Sixth, compare with a separately developed route when feasible, and record the shared cuts between the routes. Seventh, test executable refinement, binary64 behavior, estimator calibration, and application behavior as separate obligations. Finally, bind the exact source tree and result to a hosted run and a durable mainline commit.

No step permits a tool to choose the theorem after seeing what it can prove. No successful kernel run proves an unstated probability law. No finite exhaustive census proves a population limit outside its declared finite domain. No Z3 real-arithmetic proof establishes binary64 execution. No Rust test establishes paper correspondence unless the correspondence itself is an explicit, checked bridge. Formal tools support judgment; they do not replace it.

The deterministic workflow-PDF builder fixes `SOURCE_DATE_EPOCH` at `2026-08-15T00:00:00Z`. That value is a reproducibility epoch for stable metadata bytes, not the date of every narrative revision, visual inspection, or scientific observation. Those events keep their own explicit dates and object identities.

The dated worktree and branch preservation receipt is a bounded case record for this repository. It documents an actual temporary-state loss, reconstruction boundary, dirty-state capture, divergent-tip comparison, verified bundle roster, and cleanup preconditions. The machine-readable retirement ledger binds the dated 1 September 2026 primary-common-Git-directory snapshot to a strict schema and fail-closed checker. It explicitly excludes sibling repositories and authorizes no deletion. Neither record replaces this general protocol, promotes an archived draft, or proves a retained mathematical claim.

The separate sibling-registry retirement ledger records a bounded read-only observation of 10 common Git directories and 12 worktrees, including all four C12-family worktrees. It distinguishes the documentation candidate from the then-live remote `main` in every target-specific C12 comparison; records one configured `blob:none` promisor registry without turning a zero-missing observed-head traversal into global object completeness; and binds three live-ref-unreachable C12 commits to their exact public-receipt bundle custody. Its checker validates the recorded snapshot. Its companion self-test demonstrates rejection of 75 hostile mutations and three path-custody attacks. Neither script revisits those registries, bundles, remotes, or processes. The ledger authorizes no ref, worktree, registry, cache, or garbage-collection deletion. It excludes

the primary common Git directory and unregistered non-Git copies, so the two ledgers together still do not constitute a live global inventory.

### Agent and council ownership

Each agent owns a bounded task and a declared write set. The coordinator owns integration and the final disposition. Council members first produce independent critiques, then expose shared dependencies and attack the strongest surviving design. The ledger retains concrete dissent, counterexamples, failed routes, and conditions that would reverse the decision. A model vote, confidence score, or repeated wording never closes an obligation. After a terminal reset or context compaction, the next agent treats the continuity record as an index and verifies the live repository, processes, exact source bytes, and remote refs before it acts.

Durability has layers. A scratch file, temporary worktree, local commit, remote side branch, remote-`main` commit, release artifact, and scholarly archive are not interchangeable. Scratch and local-only state can disappear and carry no durable-evidence credit. Under the recorded repository retention and no-history-rewrite assumptions, a verified remote-`main` commit is the minimum operational recovery anchor for accepted work, not immutable or scholarly preservation. Record the remote URL, ref, observed OID, observation time, and whether the milestone is the tip or an ancestor of a later remote tip. A versioned release or an identified scholarly archive under a declared retention policy is required when the publication packet claims longer-term preservation. Record the archive's versioned locator or DOI when one is supplied; a DOI is an identifier and resolution route, not byte custody by itself.

Each ledger-declared load-bearing artifact needs an exact path or object name, SHA-256, byte size and format, claim/campaign/attempt provenance, a retrievable versioned locator with access, license, and retention limits, and a clean retrieval or rebuild check. A digest without a retrievable preimage is only a commitment or omission record. If protected or oversized material cannot enter Git, use an approved restricted durable store and publish only a safe commitment/manifest until blindness or embargo ends. Before abandoning a worktree or branch, verify that every accepted artifact is reachable from remote `main`; that every adjudicating or selection/load-bearing artifact, including a rejected route, remains retrievable from remote `main` or a checked durable locator under its typed disposition; and that every artifact selected for destruction has been established as non-load-bearing and outside the required retention scope with its rejection reason recorded.

This is deliberately problem-specific. The project does not build a generic self-improving harness or reward submission volume. It uses automation to widen and test the hypothesis space while preserving the right to leave an obligation open, a claim `blocked`, a route abstained, or a search without improvement; each uses its canonical gate, route, or claim namespace. A claim about research-process quality is eligible for a scoped methods report only if the protocol, attempt population, adaptive exposures, selection decisions, failures, inconclusive outcomes, deviations, resource use, model/tool/AI roles, and limitations are reported. Such a report should additionally freeze its task suite and resource accounting, compare or ablate the claimed loop components, report missing/redacted attempts, separate exploratory from confirmatory outcomes, and measure both useful yield and false acceptance/semantic-drift rates. A successful PID result can motivate such a study; it cannot by itself establish that the research process caused the success or generalizes to another problem. These are necessary publication conditions, not a guarantee of novelty, correctness, peer-review acceptance, or external reproducibility.

### 3.5.5 Council protocol

Use a council for every named scientific claim, method adoption or transfer, completion/disposition, and research-process or publication milestone. Routine mechanical work inherits the last applicable council disposition only while it changes no semantics, evidence, or public claim. A council is a structured adversarial review mechanism, not a vote and not an independence claim.

1. Freeze the object under review and ask each member to write a memo before seeing any other member's conclusions. Record exactly what source material each member could see.
2. Cover the applicable source/semantics, mathematics, numerical/statistical, implementation, applicability/transfer, adversarial, publication, and custody lenses. Record role overlap and shared model, prompt, data, code, institutional, and tool dependencies.
3. Ask for the strongest counterexample, alternative explanation, and scope-narrowing proposal, not merely a confidence score or summary.
4. Retain each permitted minority report verbatim or as a labelled faithful summary and bind the retained content by digest. If its content cannot be retained, record the omission/redaction and a durable restricted locator; a bare digest supplies no substantive dissent-review credit. Consensus does not erase a falsifier, and majority agreement supplies no scientific evidence.
5. The named integrator adjudicates each finding against primary sources and executable evidence, states what changed, and owns the final decision. A council recommendation is advisory unless a frozen claim packet explicitly makes a particular human or institutional review dispositive.
6. If council review changes the task, definitions, acceptance predicate, judge, or selection rule, open a new revision; never repair the already-scored attempt retrospectively. A persistent council may cover child attempts under one unchanged campaign packet; reconvene at promotion, publication, or any change to a frozen field.

Use the following coexisting evidence tags; several may apply simultaneously, none is an assurance ladder, and they must never be compressed to one green Boolean:

| Status | Exact meaning |
|---|---|
| `exploratory` | Useful for search; no closure credit. |
| `system-tested` | Passed the named internal protocol; no human-verification or independence claim. |
| `kernel-checked` | The exact formal statement was accepted under the inventoried kernel/import/axiom surface. |
| `numerically-certified` | The exact finite arithmetic predicate was enclosed or checked under its declared representation and coverage proof. |
| `human-reviewed` | A named human issued the recorded disposition on the named object and scope; this is not "verified" unless a verification protocol is separately named. |
| `separately-reproduced` | A separately controlled route reproduced the exact scoped result and recorded its dependency vector; the label alone grants no independence dimension. |

| Status | Exact meaning |
|---|---|
| `reproduced-with-independent-dimension` | A reproduction has at least one named, evidenced independent dimension; the label names that dimension and leaves every shared or unknown semantic, data, implementation, epistemic, institutional, and custody dimension explicit. |
| `published` | An exact version was publicly disseminated; this supplies no peer-review or correctness claim. |
| `peer-reviewed` | An exact version received the named peer-review disposition; this does not retroactively supply missing proof, implementation, or application edges. |

These are subject-bound **derived display tags**, not stored attempt states, evidence classes, gate states, or claim dispositions.

`kernel-checked` derives only from a matching `FORMALLY-CHECKED` artifact-verification record.

`numerically-certified` derives only from a matching `CERTIFIED-NUMERICALLY` artifact-verification record. Review, reproduction, dissemination, and peer-review tags derive from their separate metadata records and do not create an accepted-result evidence class. The canonical accepted evidence classes remain the six classes in the acceptance rule; claim disposition remains exactly `proposed`, `active`, `blocked`, `falsified`, or `complete`; route and gate state remain in their own namespaces. A display may show multiple tags without changing any underlying record.

A model-labelled proposer or judge remains advisory. Different prompts, sessions, roles, or vendors do not establish semantic, data, implementation, epistemic, institutional, or custody independence. Likewise, a kernel proof establishes only the exact formal statement under the inventoried trust surface, and a numerical certificate establishes only its finite arithmetic predicate. Neither closes paper-to-definition correspondence, binary64 refinement, estimator calibration or consistency, PID-method identity, or an application edge.

### 3.5.6 Separate roles

Use explicit roles and deliverables for a major claim. Do not ask one undifferentiated run to set the problem, prove it, design its test, and adjudicate its own work.

| Role | Required output | Invalid shortcut |
|---|---|---|
| Specification editor | Frozen claim packet and ambiguity table | Solving before the claim is fixed |
| Source checker | Version-pinned source and known-result map | Trusting the prompt premise |
| Proof route | Named subclaims and dependency graph | Confidence language as proof |
| Counterexample route | Exact falsifiers and boundary cases | Only random examples |
| Formalization route | Prose-to-formal object map | Silent weaker surrogate |
| Certificate route | Exact or interval certificate format | Opaque decimal oracle |
| Implementation route | Specification-to-code correspondence | Unit tests as refinement proof |
| Statistical route | Frozen calibration or holdout protocol | Development data as holdout data |
| Adversarial auditor | Attack log and unresolved objections | Restating the candidate proof |
| Integrator | Evidence matrix and scoped decision | Majority vote among models |

One worker can fill more than one role only when the overlap is recorded. A final auditor must reconstruct the claim from the frozen packet and evidence. It must not rely only on the proof

author's summary. Role labels, separate contexts, fresh sessions, and adversarial prompts provide structured review only; they do not create institutional or model-lineage independence.

### 3.5.7 Review-joint matrix and saturation handoff

"Check the proof" is not a sufficient review assignment for a major claim. Before review, derive the load-bearing joints from the obligation graph and assign at least one named falsification task to each. A review-joint record contains:

```
joint_id:
claim_edge:
failure_conjecture:
smallest adjacent control world:
reviewer role and dependency overlap:
source material visible to the reviewer:
challenge actually run:
result and retained artifact:
residual uncertainty:
```

Examples of joints are a localization/tail bound, a hidden strengthening of a cited premise, a block decomposition, a representation change, an optimizer-existence step, or a numerical error margin. Reviewers should receive the minimum material needed for their assigned joint. After a joint-specific pass, give the statement alone to a fresh route and request reconstruction. A source-blind reconstruction can expose copying and exposition errors, but it remains correlated when it uses the same theorem or oracle.

Internal review has **saturated** only when all applicable joints have retained challenges, new passes share the same load-bearing dependencies and produce no new objections or evidence, the open uncertainty is explicitly listed, and the integrator can explain why more same-lineage review would add little information. Saturation is not acceptance. It triggers handoff to a specialist, custodian, implementation route, or empirical route that supplies the required recorded independence dimension. Record the stopping decision and the external evidence still required; never use compute or token exhaustion as the reason a claim became true.

### 3.5.8 Independence vector

For each pair of evidence routes record the three-component vector (functional/mechanism, epistemic/dependency, institutional/custody) rather than one Boolean:

| Component | Positive evidence | What does not establish it |
|---|---|---|
| Functional/mechanism | Different languages, arithmetic libraries, algorithms, data structures, or independently specified encodings that can expose route-specific defects | A renamed wrapper or the same generated table read twice |
| Epistemic/dependency | Materially different mathematical ideas with no shared unproved bridge, source ambiguity, oracle, formalization seam, or selected stopping rule at the claimed cut; a different model lineage can improve diversity but does not guarantee this | Fresh prose, a new prompt, or two derivations sharing the same imported crux |

| Component | Positive evidence | What does not establish it |
|---|---|---|
| Institutional/custody | Separately controlled authority, access, and refusal capability: a person, organization, protected system, or custodian that cannot silently revise the original evidence or acceptance record | Different model lineage alone, one Codex-like agent in separate contexts, role labels, local hashes, or same-lineage review |

A same-agent Rust/MPFR producer and Python exact-arithmetic checker can be functionally diverse and valuable. A same-lineage source-blind derivation can reduce one epistemic dependency. Neither is institutionally independent merely because it ran separately. State unknown or absent components instead of promoting a partial vector to "independent verification."

### 3.5.9 Separately developed routes and independence accounting

For each major claim:

1. Give at least two routes the same frozen claim packet without the other route's answer.
2. Require each route to state its starting point, assumptions, strongest result, exceptional cases, and likely failure mode.
3. Record a route memo before routes exchange results.
4. Share route artifacts and stated implications, not hidden reasoning traces.
5. Identify shared unproved lemmas, source material, generated data, and oracles.
6. Count routes that use the same unproved bridge as one route for confidence purposes.
7. Preserve every materially distinct valid proof or solution as a separately named route, even after selecting a preferred exposition. Do not overwrite a special-case proof with a later generalization or merge two mechanisms merely because they reach the same conclusion.

Prefer method diversity to model-name diversity. Examples are a coupling proof and exact enumeration, a real-analysis proof and a formal kernel, or a generator and a separately written checker whose dependency vector is recorded. Repeated passes by the same model against the same prompt, source text, imported theorem, or proposed proof are correlated evidence, even when they occur in fresh sessions or use harder reasoning settings. They can reveal instability and generate attacks, but they do not satisfy the epistemic route requirement unless their load-bearing dependencies are genuinely disjoint, and they never establish institutional/model-lineage independence by session or role label alone.

### 3.5.10 Critical-cut-set accounting

Count independence at the level of proof dependencies, not prose narratives. Represent obligations as an AND/OR acyclic hypergraph: every tail of an AND-hyperedge is required, while alternative incoming hyperedges are OR-routes; logical alternativity alone does not make them dependency-disjoint. A node closes only when one permitted incoming hyperedge has all required premises closed and its implication checked. Define each route as a set of dependency nodes in a frozen admissible vertex universe. Unless a deliberately trivial cut is reported separately, that universe excludes the common claim/goal node and synthetic route or AND/OR aggregator nodes. Record every inclusion-minimal shared cut set: each inclusion-minimal admissible node set that intersects

every route in the declared route family. State whether the family is candidate, permitted, or accepted, and tag each cut node as open, closed, or a known common cause. An accepted route cannot contain an open node; a cut over accepted routes may nevertheless expose a closed shared dependency.

For the worked two-route figure, freeze $U = \{A1, A2, B1, C\}$, $R_A = \{A1, A2, C\}$, and $R_B = \{B1, C\}$. A cut is a set $H \subseteq U$ with $H \cap R_A \neq \emptyset$ and $H \cap R_B \neq \emptyset$. The complete inclusion-minimal cut family is $\{\{C\}, \{A1, B1\}, \{A2, B1\}\}$. This follows without trusting the drawing: if $C \in H$, minimality forces $H = \{C\}$; if $C \notin H$, hitting $R_B$ forces $B1 \in H$, and hitting $R_A$ then forces exactly one of $A1$ or $A2$. The figure metadata records these same sets, and the publication checker separately enumerates their finite transversal family. Omitting $\{A2, B1\}$, or retaining a nonminimal superset, is a failing semantic mutation.

Two derivations that use different case splits but obtain their power from the same imported theorem, unproved lemma, generated table, numerical oracle, or implementation are correlated at that cut set. They can cross-check local algebra outside the cut set, but they do not independently close it. A route summary must therefore include:

- the critical cut set;
- all other routes that share each cut-set node;
- the evidence type that closes each node;
- whether the node was re-derived, checked against a primary source, assumed, or only restated;
- one falsification attempt directed at each shared node.

Use a dependency-disjoint route, not another paraphrase, when a high-consequence claim depends on one shared bridge. Examples include an exact enumerator versus a symbolic proof, a Python integer/Fraction checker versus a Rust MPFR producer, or a formal theorem versus a numerical fixture.

### 3.5.11 Frozen run context

Record this full context for each model run promoted to evidence or used to adjudicate a claim:

- claim ID and revision;
- `pid-rs` commit;
- paper, generated document, and formal artifact revisions;
- proof and imported-library toolchains;
- Rust compiler, `Cargo.lock`, target, and feature set when code is in scope;
- generated table and lattice digests;
- exact definitions, conventions, assumptions, and non-solutions;
- pre-candidate task-packet identity, candidate write scope, judge/verifier identity, and evidence plus an enforcement mechanism showing that the packet and judge were outside that candidate's permitted mutations;

- ex-ante intended closure and falsification routes, the initial accepted-result label, and completion checks, resource/stopping rule, and selection rule;
- prompt text or an exact permitted projection, model/system/tool identity where knowable, run date, seed when exposed, output path and digest, terminal disposition, and retained redaction/omission record.

For a non-exploratory run that is not promoted or adjudicating, retain only enough identity, purpose, disposition, and reopen condition to prevent accidental reuse or double counting. Retain the full context above and decisive output for every evidence-promoted or adjudicating run. Summarize exploratory failures with their route, falsifier, and reopen condition; do not mistake a digest for proof or archive every routine prompt merely to increase evidence volume. Hidden reasoning traces are neither required nor an accepted evidence artifact.

Without this context, treat the output as exploratory. It cannot change a claim disposition.

### 3.5.12 Freeze purpose and identity boundary

Do not use "frozen" without naming its object and purpose. For every freeze record: the exact object; the threatened error; the checker, CI gate, release system, or custodian that refuses to proceed on mismatch; whether the same actor can revise both object and expected record; and whether the theorem or statistical design actually requires fixedness.

| Freeze class | Object and purpose | Required refusal point and limitation |
|---|---|---|
| Mathematical claim | Definitions, types, quantifiers, premises, conclusion, and non-solutions; prevents proof of a nearby statement | Claim revision and review gate refuse silent semantic drift. A same-author revision is visible, not independently authenticated. |
| Exact bytes | A named raw file, canonical payload, source tree, certificate, release asset, or external source; prevents stale or substituted instance use | The consuming checker or release gate recomputes identity and refuses mismatch. A colocated digest detects drift but is not external authentication. |
| Fitted transform | Parameters and behavior of a measurable transform trained only from declared fit information; prevents evaluation leakage or adaptive remapping | Evaluation code refuses an unbound/refitted transform. Fixedness may be a theorem premise; its digest only identifies the represented transform. |
| Analysis/holdout plan | Metrics, tolerances, multiplicity, failures, seeds/target custody, stopping, and acceptance rules; reduces result-dependent tuning | Adjudication refuses deviations or records a new version. Call it blind only when inaccessibility is enforced; otherwise it is controlled. |

Git-tree identity, SHA-256 of raw file bytes, canonical mathematical-object identity, and external authentication answer different questions. A Git commit binds a tree and paths; a raw-file digest binds one encoding; a domain-separated canonical digest binds the declared mathematical serialization; a signature, protected log, or separately controlled custodian with a recorded custody dimension can authenticate or anchor an identity. None proves the object's mathematical correctness. Retain extra digests where they bind cross-artifact, release, package, certificate/input, or external-source bytes, or where mechanism diversity checks an encoding. Do not multiply routine prose digests and call the count evidence.

### 3.5.13 Durable agent continuity

Before a long-running command, after every consequential result, at every observable checkpoint before a possible context boundary, and immediately after detecting compaction or resume, write or update a durable continuity record outside secret material. Current Codex surfaces do not expose a guaranteed pre-compaction hook; a future harness may add one, but this protocol must not claim that an unobservable checkpoint was recorded. The record must contain:

- objective plus the exact permitted normative user prompt and instruction locators; when the prompt contains protected spans, store a redacted canonical projection plus immutable locators and an identity of the permitted projection, while representing protected spans only by custody locators or permitted opaque identities;
- exact in-scope work and explicit non-solutions;
- repository path, remote URL, HEAD commit, tree identity, worktree path, clean/dirty state, and any alternate-index identity;
- owner for every agent, worktree, mutable path, immutable path, and external evidence location;
- each running command with session identifier; PID and process-group ID only when the tool exposes them or they are reliably observed, otherwise explicit `unavailable`; evidence directory, expected outputs, and a safe non-destructive poll rule;
- primary-source and citation identities, versions, locators, and locally checked byte digests when the downloaded bytes matter;
- every closed gate with its exact evidence and explicit non-implications;
- open obligations, blockers, dependencies, caveats, contradictions, and every invalidated, withdrawn, unselected, or zero-credit run with cause;
- user decisions, permissions, and forbidden actions;
- the next safe read-only or in-scope action; and
- record schema version and UTC update time as observation metadata, never as preregistration or time authority; when a continuity transport needs byte identity, place raw-byte SHA-256 in an adjacent handoff/sidecar or the next resume record, or define an explicit canonical projection that excludes the identity field, and state which method was used. A matching colocated record and digest provides drift detection, not authentication.

Resume protocol:

1. Read the normative prompt, applicable instructions, and continuity record fully.
2. Verify live Git/remote/worktree state, processes, and named files with read-only checks.
3. Reconcile every drift. The continuity record is a memory aid, not authority, attestation, or theorem evidence; a colocated digest does not authenticate it.
4. Do not restart closed milestones, double-count prior evidence, transfer credit across a changed claim/fixture/toolchain, or silently reuse a zero-credit run.
5. Revalidate external facts that are missing, stale, mutable, or outside the pinned source scope.

6. Update the record at the next observable checkpoint before another possible context boundary or long run, immediately after a consequential pass, failure, invalidation, permission change, or scope change, and immediately after detecting compaction or resume.

Never store credentials, tokens, decryption keys, secret holdout rows/seeds/targets, hidden reasoning traces, or other protected material in the continuity record. Store only custody locators and permitted opaque identities for inaccessible assets.

Any active summary must be a derived navigation view over these append-only facts. It may prioritize or compress, but it must carry stable references to every still-load-bearing caveat, contradiction, open obligation, and withdrawal. A summary rewrite cannot delete or upgrade the underlying record; if the summary and ledger disagree, adjudication stops until the conflict is reconciled against the original artifacts. This prevents a polished retelling from silently converting an exploratory value into an accepted result.

Compact copyable template:

```
CONTINUITY v1 | updated_utc: <observation metadata, not time authority>
record_identity_method: <sidecar/next-record raw SHA-256, or projection excluding this field>
objective: <exact objective>
normative_prompt: <exact permitted prompt, or redacted canonical projection>
normative_locators: <prompt/instruction paths or immutable message IDs; protected custody
↪ locators>
scope: <in scope> | non_solutions: <explicit exclusions>
repo: <path> | remote: <url> | HEAD: <commit> | tree: <tree>
worktree: <path> | dirty: <exact status> | alternate_index: <none or identity>
ownership: <agent/worktree/path/evidence owners; immutable paths>
running: <command; session; PID/PGID or explicit unavailable; evidence_dir; safe_poll; or
↪ none>
sources: <citation/version/locator/raw-byte identity as applicable>
closed: <gate -> exact evidence -> non-implications>
open: <obligation/blocker/dependency>
invalid_zero_credit: <run/artifact -> cause>
decisions_permissions_forbidden: <user decisions; allowed and forbidden actions>
next_safe_action: <one exact action>
```

### 3.5.14 Codex goal, plan, and tool lifecycle

A long-running Codex execution has three distinct state layers. The **scientific claim state** is the normative record of the mathematical disposition; the **goal state** records the user's unfinished objective; and the **plan state** is only a mutable scheduling aid. A token count, elapsed time, model confidence, completed plan step, or the mere success status of a tool call is not evidence of scientific progress. Validated and retained tool output may supply evidence for the exact obligation it checks.

Under the current Codex function contract, apply this lifecycle:

1. Create a durable goal only when the user explicitly requests one. Use the exact objective, and set a token budget only when the user explicitly supplied a budget. Do not replace an unfinished goal with a narrower convenience goal.
2. Call `get_goal` after a resume or compaction, before a progress/completion report, and before a terminal goal transition. Reconcile it with the continuity record and live repository evidence; neither source silently overrides the other.
3. Use `update_plan` to expose sequencing, with at most one step in progress. A plan item closes only when its stated work is done, but this still gives no evidence credit unless its claim gate also closes.

4. Use `update_goal(status="complete")` only when the exact objective is achieved and no required work remains. Near-exhausted compute or a convenient stopping point is not completion. Use `blocked` only after the same impasse has recurred for at least three consecutive goal turns, counting the original user-triggered turn, and no safe in-scope progress remains; record the blocker and each attempted alternative. When a previously blocked goal resumes, begin a fresh three-turn blocked audit. A hard, slow, or partially complete goal stays active: difficulty, slowness, or partial progress alone is not a pause request. Pause only at the user's explicit request; then call `update_goal` with status paused when that tool is available, report its returned state, and stop goal work. Use plan or goal tools only when they are actually available, and never invent a successful call.
5. Derive progress from a versioned obligation registry, not from tool usage. Publish the weighting rule, count only closed applicable obligations, retain failed and unknown states separately, and report a range when the denominator or scope can still expand.

Some Codex surfaces may spell goal creation as `create_goal` and a user may call the action "set goal." A harness adapter must bind the semantic operation and verify the returned goal identity/status; it must not pretend an unavailable call succeeded. Goal-service state is coordination metadata, not proof, archive custody, or authentication.

### 3.5.15 Tool-call and agent receipts

For every consequential tool action, classify it before execution as read-only, reversible in-scope mutation, external state change, or destructive. Record the exact subject and authority. A replayable receipt contains, as applicable:

- tool/function name and contract revision or client version;
- domain-separated operation ID, goal ID, claim ID/revision, and dependency IDs;
- exact repository root, worktree, HEAD, tree, dirty-state projection, and alternate index;
- normalized argument projection, working directory, relevant environment projection, toolchain, platform, and input artifact identities;
- start/end observations, exit status, stdout/stderr identities, produced artifact identities, and whether output was truncated;
- expected-versus-observed predicate and explicit non-implications; and
- failure disposition, retry policy, cancellation state, and safe resume/poll identifier.

Do not record secrets or hidden reasoning. A timestamp orders observations only when its clock and authority are declared; it is not a cryptographic time proof. A receipt produced and checked by the same mutable authority detects accidental drift but is not independent attestation.

For a long command, retain the returned session/cell identifier and poll with the corresponding non-destructive wait function; do not launch a duplicate merely because no new output arrived. Before retrying, determine whether the prior process is running, terminal, or externally unobservable. Bind the final result to the original command rather than reporting only the last poll.

Delegate only bounded tasks with an explicit input subject, mutable-path ownership, forbidden actions, deliverable, and completion test. Two agents must not edit the same mutable path concurrently. Use read-only hostile reviewers for frozen candidates, and record whether their starting assumptions, implementations, source access, model lineage, and custodians actually differ. Codex subagents can add functional or epistemic diversity; being subagents does not by itself add institutional independence.

### 3.5.16 Publication-harness state machine

A future executable harness should make the workflow below a typed acyclic graph, not infer it from prose. Each frozen, append-preserved claim revision owns:

- a premise registry and positive mapping obligations;
- artifact nodes for source, formal statement, certificate, generator, checker, executable, data, statistical plan/result, rendered document, package, and hosted receipt;
- gate nodes with `open`, `passed`, `failed`, `blocked`, or reasoned `not_applicable` state;
- directed edges labelled `AND`, `OR`, `derives`, `checks`, `renders`, `packages`, or `observes`, with every OR branch preserving its own premises;
- exact evidence identities, route-independence vectors, negative controls, limitations, and downstream non-implications; and
- a decision node that refuses completion while an applicable required gate is not passed.

Orient every dependency-bearing edge from prerequisite to dependent. A semantic change to a premise, definition, generator, checker, compiler/toolchain assumption, fixture, threshold, or mapping theorem invalidates every semantically dependent result reachable from that object. A raw byte change separately reopens every exact-byte-dependent gate that binds those bytes. It does not, by itself, falsify a mathematical theorem whose dependency-disjoint route is unchanged. Such a route may remain passed only when the graph contains a separately checked impact/equivalence map showing that no changed semantic prerequisite can reach any node used by the route. Formatting or serialization changes therefore still require artifact replay, while mathematical credit survives only through an explicit semantic-equivalence boundary. The harness moves affected descendants back to `open` or `blocked`, retains the old receipt as historical evidence, and requires the appropriate replay. This is the mechanism for correcting earlier work when a later finding truly invalidates it without pretending that every changed byte changes every theorem.

Hosted execution and self-binding artifacts require an acyclic commit protocol: commit and push the subject; wait for its exact hosted run to become terminal; then commit a descendant receipt that binds the subject and run. The receipt commit cannot contain evidence about its own future hosted run. If that receipt itself needs hosted observation, keep it external or bind it in a later, explicitly scoped descendant - never claim a self-hash or infinite receipt chain.

For publication, the harness must require exact agreement among canonical Markdown, any embedded typesetting source, generated tables/figures, and the committed PDF; deterministic rebuild where claimed; warning/font/link/metadata checks; extracted-text and page-geometry comparison across the declared toolchains; and page-by-page rendered visual review. A PDF that is newer in prose but stale in bytes, or byte-current but visually defective, fails the publication gate. Machine and human artifacts must expose the same claim revision, evidence state, limitations, and open obligations.

This workflow publication uses a repository-local adaptation of the reviewed publication design language: ivory paper; lapis title structure; turquoise subordinate headings; restrained bronze body accents and saffron structural accents; real searchable text; and low-contrast vector grain and girih/rosette fields. The title pattern fades in and out through five opacity zones. It is decorative: it does not encode evidence status, theorem strength, probability, or graph magnitude. The visual gate reviews hierarchy, typography, grid, spacing, narrative order, motif provenance and coherence, restraint, palette identity, data semantics, color-redundant labels, grayscale,

text extraction, print, A4/PDF profile, fonts, link/action safety, deterministic reproduction, source/derived separation, portable dependencies, and normal-size plus high-resolution renders. Passing that visual gate does not establish mathematical correctness, accessibility conformance, or the availability of an external link.

Compact harness transition rule:

```
local_closure(node) :=
  (leaf(node) AND accepted_evidence_of_declared_class(node))
  OR
  (internal(node)
   AND exists permitted incoming hyperedge e -> node
   AND every tail node of e is accepted
   AND implication_of(e) is discharged)

accept(node) :=
  schema_valid(node)
  AND exact_subject_identity(node)
  AND local_closure(node)
  AND every_positive_mapping_edge_discharged(node)
  AND negative_controls_fail_closed(node)
  AND limitations_and_non_implications_present(node)
  AND no_reachable_invalidated_dependency(node)

publish(claim_revision) :=
  all_applicable_required_gates_passed
  AND decision_receipt_acyclic
  AND machine_human_artifacts_coherent
  AND release_subject_matches_reviewed_subject
```

These predicates specify refusal behavior; they do not prove the mathematical predicates embedded inside a node. Each mathematical premise still needs its named proof, certified computation, accepted empirical design, or explicit assumption status.

### 3.5.17 Evidence labels and route memos

The following are **artifact-verification labels** for substantive model statements. They are not the accepted evidence classes or the claim dispositions defined in the acceptance rule. Use one:

- `UNVERIFIED-MODEL-OUTPUT`;
- `CHECKED-SYMBOLICALLY`;
- `CHECKED-EXACTLY`;
- `FORMALLY-CHECKED`;
- `CERTIFIED-NUMERICALLY`;
- `IMPLEMENTATION-REFINED`;
- `EMPIRICALLY-CALIBRATED`;
- `CONSUMER-QUALIFIED`; or
- `REJECTED-BY-COUNTEREXAMPLE`.

Each route memo must record:

```
Route ID:
Claim revision:
Mathematical family:
Starting point and independence vector:
Current obligation:
Strongest established result:
Exact evidence:
Counterexamples attempted:
Exceptional cases:
Missing lemma or bridge:
State:
Reopen condition:
Artifact paths and digests:
```

A route that reduces the target to an unproved claim of comparable strength is blocked, not complete.

Keep three fields separate in every claim schema:

| Field | When fixed or appended | Meaning |
|---|---|---|
| Artifact-verification label | Appended to one reviewed artifact | What kind of check that artifact actually survived; it is not the claim's evidence state |
| Intended closure/falsification routes | Frozen ex ante in the claim packet | Which premise-explicit proof, certificate, test, holdout, or counterexample routes are permitted to close or falsify each obligation |
| Current accepted-result evidence records | Initialized as an empty list rendered as the literal text "no accepted evidence"; append-only after adjudication | Zero or more records, each with exactly one evidence class, scope, assumptions, artifact identities, adjudication, and invalidation/supersession state; the claim disposition remains separate |

A diagnostic may create an obligation or motivate a falsifier, but it does not silently become a closure route. A counterexample closes only the exact negative implication or disproof whose quantifiers it instantiates. Do not rewrite the ex-ante field after seeing a favorable result; create a claim revision instead.

## 3.6 pid-rs protocol

The rest of this note defines a repository protocol. It is an adaptation, not a source claim.

### 3.6.1 1. Write an exact-claim packet

Create one packet before work starts. Give the packet a stable claim ID. Include these fields:

| Field | Required content |
|---|---|
| Claim kind | Exactly one primary kind: definition/semantic identity; mathematical theorem; statistical/estimator performance; formal correspondence or certificate; executable conformance; custody/release qualification; consumer/downstream readiness; or another precisely defined kind. Dependencies on another kind become separate typed obligations or claim IDs. |

| Field | Required content |
|---|---|
| Claim | One versioned, type-specific proposition or specification-conformance statement with one disposition. Split conjunctions whenever components can close, fail, expire, or be invalidated separately. |
| Objects | Every exact object used by the selected kind: mathematical domains, codomains, alphabets, measures, lattices, sample spaces, and source/target roles; formal statements and encodings; executable APIs, artifacts, builds, and platforms; custody/release subjects; and consumer system, use case, authority, and decision boundary. Mark an inapplicable object class with a typed reason. |
| Support/reference measure | Population support, sigma-algebras, dominating/reference measures, densities or RN derivatives, boundary and singular cases |
| Lattice/source count | Exact source count, finite antichain carrier, typed redundancy order, event map, and zeta/Möbius direction |
| Quantifiers | Their full order, including every uniformity requirement |
| Sampling | Row law, independence/dependence class, conditioning sigma-field, stationarity/ergodicity or named coefficients/rates, splits, and sample size |
| Assumptions | A typed premise ledger assigning every premise to the exact object and downstream edge that uses it |
| Units/gauge | Logarithm base, units, metric, scale, preprocessing/measurement gauge, and which changes alter the estimand |
| Mapping obligations | Every cross-definition, discrete/continuous, transformed/untransformed, paper/code, or formal/prose transfer and the positive theorem needed to justify it |
| Representation | Exact-real, symbolic, interval, high-precision reference, binary64, serialized, compiled, and wrapper claims kept distinct |
| Conclusion | The exact versioned definition; equality, inequality, quantified theorem, limit, coverage, calibration, error, or abstention statement; formal-correspondence claim; executable refinement/conformance property; archive/release predicate; or bounded consumer readiness/no-go decision appropriate to the selected kind, including scope and non-implications |
| Non-solutions | Weaker statements that do not complete the claim |
| Falsifiers | Boundary cases and counterexamples that can refute the claim |
| Attempt and judge boundary | Stable attempt/revision ID; exact pre-candidate packet identity; candidate-readable inputs and writable payload paths; prohibited mutations and trust expansions; pre-bound judge/verifier/test/reference identities outside candidate authority; confinement and network/secret policy; and fail-closed behavior when any required judge route is unavailable |
| Intended closure/falsification routes | Frozen ex ante for each obligation: premise-explicit mathematical proof, machine-checked proof, certified computation, scoped test or holdout, and a counterexample route where falsification is possible |
| Resource, stopping, and selection rule | Ex-ante or append-only rule for budget, concurrency, retries, termination, route comparison, candidate promotion, and retention of valid unselected, failed, and withdrawn attempts |

| Field | Required content |
|---|---|
| Initial accepted-result evidence records | Empty, rendered as the literal text “no accepted evidence”; later adjudicated records are appended one class per record and never substituted for the intended-route field |
| Completion predicate and adjudicator | A fail-closed predicate, mechanical where possible, plus the named human authority where required, that checks whether every applicable typed obligation is already closed by current accepted evidence and whether no unresolved falsifier, contradiction, expiry, or invalidation remains. It may adjudicate the disposition; it cannot create, replace, or upgrade missing evidence. |

Do not overwrite a claim packet after you inspect a result. To correct or change it, create a new revision and retain the old revision.

Retain the full packet schema for every claim kind. Mark a field `not_applicable` only with a typed reason showing why the claim has no corresponding mathematical, statistical, formal, executable, release, or consumer obligation. A reasoned non-applicability record closes only that applicability question; it supplies no evidence for any other field.

Completeness is a disposition, not an evidence class. A green completion checker is evidence only for the exact checker-conformance claim it exercised. It does not by itself close a mathematical, statistical, formal, executable, release, custody, or consumer claim. Each typed obligation still needs an accepted record from one of its frozen permitted routes, with every mapping edge closed.

### Why scientific results must be typed

Many consequential failures are object-identity failures rather than arithmetic failures. A bare number cannot say whether it is a functional or estimator output, population or sample object, training-fitted or same-sample result, exact-real or binary64 value, nominal permutation statistic or surrogate-tail diagnostic, or whether its support and units satisfy the route that consumes it. In PID specifically, the same numeric representation can otherwise conceal transfers among KSG MI; categorical MGW shared-exclusions SxPID; the unimplemented Schick–Poland general measure-theoretic construction; Ehrlich continuous shared-exclusions redundancy and PID2; Williams–Beer $I_{\min}$; incomplete versus research-only mixed-dimensional PID3; fitted quantized SxPID for a quantized estimand; and project-defined diagnostics.

Where applicable, a result object should therefore carry: estimand and method identity; domain/support and reference measure; units/gauge; source/target, sample, fitting, and evaluation roles; method/source revision; assumptions; representation and numeric semantics; artifact verification/invalidation state and links to separate evidence and claim records; warnings or abstention; resource budget; and admissible interpretations. Typed constructors and composition edges reject incompatible composition; the claim harness, not the result object, reopens dependent gates when a field is missing, incompatible, invalidated, or changed.

This has three distinct potential values. It is intended to improve **scientific error prevention** by making silent object substitution explicit or unrepresentable. It can improve **engineering and ecosystem composition** because Rust, Python, run-log, resampling, and downstream systems can exchange structured results without guessing what an unlabeled scalar means. Neither benefit is a demonstrated outcome until prevented counterexamples, usability, and cross-package behavior are measured. It provides **no automatic scientific novelty**: a type system becomes a potential methods/software contribution only when its composition laws and those empirical benefits are actually evidenced.

The pattern generalizes to causal estimands, posterior summaries, uncertainty intervals, surrogate tests, imaging transforms, and action-conditioned forecasts. Its transferable principle is: **an unlabeled number is not a self-interpreting or safely transportable quantitative scientific result; its object identity, assumptions, units, production route, and admissible interpretations must travel with it.** Types still cannot establish paper-to-definition correspondence, estimator consistency or calibration, numerical correctness, kernel soundness, or application validity. Those remain separate proof, certificate, experiment, and review obligations.

### Historical pid-rs/Wibral-program inventory snapshot (15 August 2026)

The following is a **historical**, non-additive status snapshot, not a current status report and not a novelty score. It is bound to base commit `bc3aa80fb6025e709c2906a08bce25a4fac40578`, `method-catalog.json` (401,485 bytes; SHA-256 `6e8fb1143019b3f2bffe982636586705d5e99cca5decfe17a810d694b50ed8aa`), and its generated `METHODS.md` (488,874 bytes; SHA-256 `a24a7c9a789902bd92dadcae442bdcf96e3d47de1632764fdcc0d67c9c125af4`). The landed inventory rows below derive from those base-commit objects. The mutable `0/7`, `0/5`, and `0/108` coordination rows are separately derived on 15 August 2026 from the same base commit's `audit/evidence/wibral-pid-program-active-plan-2026-08-12.md` (17,112 bytes; SHA-256 `4940532a46dcb345f5992bb3aa148f3262d1fef11f0b8e3114c8f78c52525417`) and the absence of completed claim packets; they are historical coordination observations, not base-tree scientific results and not instructions for the present tree.

| Object class | Landed or claimed count | Exact boundary |
|---|---|---|
| Globally novel PID theories/functionals claimed | 0 | The current catalog makes no scientific-priority claim. |
| Published PID theory/functional lineages with an implemented decomposition route | 3 | Williams–Beer $I_{\min}$; MGW categorical shared exclusions; Ehrlich continuous shared exclusions. Schick–Poland's separately represented general construction has no implementation here and is excluded. This is a lineage count, not a novelty claim. |
| Implemented PID decomposition method routes | 4 | Williams–Beer categorical $I_{\min}$; categorical MGW shared-exclusions SxPID; Ehrlich continuous PID2; research-only mixed-dimensional continuous PID3. PID3 is a distinct 18-node mixed-dimensional implementation route within the Ehrlich lineage, not a fourth theory lineage or a new PID. |
| Separately cataloged PID-defining redundancy dependencies | 1 | Ehrlich continuous shared-exclusions redundancy; it is not counted again as a decomposition or as another theory lineage. |
| Project-defined PID workflow/composition extensions | 10 | Typed compositions/adapters and selection/UQ workflows; none is an alias for its input paper functional. |
| Cataloged PID-facing validation/assurance extensions | 6 | Formal, exact, convergence, continuity, concentration, or audit packages; not six PIDs. |
| PID/KSG-relevant source-written Lean theorems | 275 | Scope-specific formal statements; theorem count is not scientific completeness. |

| Object class | Landed or claimed count | Exact boundary |
|---|---|---|
| Exact solver obligations | 9 | Five PID-algebra and four conditional KSG obligations; solver results are not kernel proof objects. |
| Named core-science units closed at this snapshot | 0/7 | KSG M1c (1), PID2 revision 4 (1), and categorical MGW SxPID3 Programs A–E (5) were open at the stated snapshot. |
| Categorical MGW SxPID3 assurance at this snapshot | 0/5 programs; 0/108 coordination entries | Stable code computed the 18-antichain lattice, but the stronger assurance program had not landed at the stated snapshot. |

The three-lineage count groups objects by their published defining theory/functional identity. The four-route count instead records distinct implemented decomposition methods: Ehrlich PID2 and the 18-node mixed-dimensional PID3 are separate algorithmic routes inside the same Ehrlich lineage. Raw/report variants, source-count variants, language bindings, wrappers, and workflows add neither lineages nor routes; a separately cataloged redundancy dependency is listed without double-counting. KSG MI, co-/O-information, target-free analogues, support diagnostics, resampling, and custody infrastructure remain outside both counts. Publishing this inventory adds no scientific credit; every future commit must be counted from its actual tree/diff. Two explicit estimator gaps remain: hyperbolic shared-exclusions PID and general mixed-support continuous shared exclusions.

**Later bounded SxPID3 evidence (26 August 2026).** The current evidence-adjudication index selects decision record 2. On exactly three ordered binary sources, one binary target, and every labelled 16-cell nonnegative-integer count vector with $1 \leq N \leq 5$, two implementation-disjoint-under-shared-semantics Python routes recomputed all 108 keyed scalar audit expressions per table and matched in the retained neutral-v2 digest and the complete six-block exact sign/zero census. Here 108 means 18 antichain positions $\times$ two representation stages $\times$ three components; it does not mean 108 PID atoms, lattice positions, independent degrees of freedom, or certified coordinates. This later result therefore supersedes the historical "nothing landed for the 108-entry audit" observation, but it does **not** convert the row to "108/108 certified": the complete claim remains proposed/open, all Programs A–E remain open, and paper correspondence, concrete dual-formal semantics, directed logarithm enclosure, compiled keyed Rust refinement, and complete replay and custody remain unresolved. The same decision separately accepts the finite-law informative source-marginal factorization under its stated assumptions; that theorem is not evidence for the remaining certificate cuts.

Add a semantic pin for every ambiguity that changes the mathematical object. Quote or transcribe the controlling primary-source statement, record the competing readings, choose one reading with a reason, and state which downstream obligations depend on it. An implementation test cannot repair a proof about the wrong object.

No similarity, limiting intuition, unit agreement, shared atom names, or successful API call is a mapping theorem. Before transferring a result, state a positive theorem whose domain, codomain, premises, map, preserved property, and conclusion match the claim packet. If that bridge is absent, mark the edge `OPEN` or `BLOCKED` and abstain from the transfer. Keep empirical-versus-population and exact-real-versus-binary64 obligations on separate nodes even when the formulas look identical.

### 3.6.2 2. Build an obligation graph

Split the claim into named obligations. Each edge must state why one obligation implies its destination obligation. Use separate nodes for these classes:

- definition and semantics;
- mathematical reduction;
- finite algebra;
- analytic inequality or limit;
- numerical certificate;
- estimator behavior;
- software conformance;
- statistical validation;
- downstream acceptance.

If an edge depends on an unproved lemma, create a separate obligation node. Mark it as open until evidence closes it.

#### Optional explicit mapping-check rule

Use the typed graph directly unless category-theoretic structure closes a named obligation. The ordinary project phrase is **explicit mapping check**; "commuting square" is only the optional mathematical notation for a proved path-equality claim. When that notation genuinely applies, treat the obligation graph as freely generating paths: composing accepted edges creates a candidate route, but parallel paths are not equal until an exact equality witness is accepted. For an approximate or statistical comparison, replace equality only with a declared common codomain, metric/divergence, domain, probability statement, and aggregate error bound. Never infer an edge from shared names, isomorphic-looking diagrams, matching units, a common implementation, or a limiting intuition.

Every category-theoretic claim, as distinct from a claim about finite category-valued data, must answer five questions in its claim packet:

1. What are the exact objects and admissible morphisms, and which PID/method identity does each inhabit?
2. What equality or equivalence is used, and is it exact, almost-sure, distributional, numerical, observational, or only heuristic?
3. Which identity, associativity, composition-preservation, naturality, monotonicity, or data-processing laws are actually proved?
4. What counterexample would show that a square does not commute or a proposed functor does not preserve the named structure?
5. What does the categorical formulation *not* establish about support, estimation, calibration, binary64 refinement, computation, custody, or application?

Keep MGW categorical shared exclusions, Schick–Poland general shared exclusions, Ehrlich continuous shared exclusions, Williams–Beer `I_min`, Blackwell-order proposals, PID2, PID3, and project diagnostics on distinct nodes. “Categorical” in MGW’s data domain remains unrelated to category theory. Do not add a Cargo dependency, plugin layer, schema abstraction, or CI mechanism for this notation unless a later object-specific proposal demonstrates a smaller trusted surface or a proof obligation that the existing typed graph cannot express adequately.

### 3.6.3 3. Keep a separate-approach registry

Use one row for each mathematical idea. Do not use one row for each worker or prompt.

| Field | Meaning |
|---|---|
| Approach ID | Stable identifier |
| Family | Main mathematical mechanism |
| Starting inputs and independence vector | Ideas not copied from another route; functional, dependency, and custody overlaps stated explicitly |
| Current obligation | The exact node under study |
| Best result | Strongest proved statement |
| Counterexample search | Cases that were tested |
| Missing lemma | Exact open step |
| State | Proposed, active, blocked, falsified, merged, or complete |
| Reopen condition | New fact that can justify more work |
| Artifact links | Notes, code, proof files, fixtures, and logs |

For PID work, start with different families when they apply. Useful families include Möbius and lattice algebra, measure-theoretic analysis, concentration inequalities, information geometry, optimization, exact finite enumeration, and certified interval analysis.

Do not give every route the current preferred answer. First, let each route state its own assumptions and failure modes. Combine routes only after this step.

### 3.6.4 4. Retain blockers and failed routes

Record every failed route. Keep its exact statement and its failure reason. Use one of these failure classes:

- a concrete counterexample;
- a false intermediate lemma;
- a missing assumption;
- a quantifier error;
- a reduction to a claim of comparable strength;
- a numerical certificate that does not cover the full domain;
- a machine-checked proof that checks only a weaker statement;
- a statistical design that reuses development data.

Do not delete an invalidated derivation. Mark it clearly as invalid. Add a small counterexample when one is available. State why it falsifies the route. A route can reopen only when new retained evidence resolves its recorded failure without silently weakening the claim. That evidence may be a corrected lemma, completed bridge, repaired certificate or implementation, a new mechanism, or a stronger premise that the claim packet permits.

### 3.6.5 Load-bearing correction ledger

When an audit finds an error, do not classify it informally as “minor” or “fatal.” Trace it through the obligation graph. Record:

| Field | Required content |
|---|---|
| Error | The exact false statement, code path, or certificate field |
| Detector | Counterexample, invariant, proof checker, mutation, or source comparison |
| Smallest witness | Minimal retained input that exposes the error |
| Dependency reach | Every claim node reachable from the false node |
| Load-bearing status | Whether any permitted conclusion depends on that node |
| Correction | New statement or implementation and why it is valid |
| Regression | A test, theorem, or mutation that fails before and passes after the correction |
| Residual boundary | What the correction still does not establish |
| Artifact revisions | Old and new claim, proof, source, and evidence digests |

A non-load-bearing error still remains in the ledger. “The conclusion survives” is acceptable only after the dependency reach is explicit and a separately checked, dependency-disjoint accepted path establishes the conclusion without the false node.

Likewise, a local counterexample refutes only the quantified statement or implication edge it instantiates. It does not refute a downstream theorem when an alternative proof route remains open; trace the OR/AND dependency graph, invalidate every route that uses the false edge, and leave the target `OPEN` or `BLOCKED` until another route is either proved or refuted. Do not promote “this proof fails” to “the theorem is false.”

If a result expands from a special case to a uniform or higher-dimensional theorem, create a new claim revision. Preserve the narrower proof and its evidence. Re-open all obligations involving new quantifiers, uniform constants, exceptional families, or imported theorems. Agreement on the old special case does not validate the generalization.

### 3.6.6 5. Convert computation into certificates

Use numerical work to find structure. Do not treat a plot or a floating-point match as a proof. Use this conversion path when it applies:

```
exploratory computation
    -> exact reduction
    -> rational or outward-rounded interval data
    -> finite certificate statement
    -> machine check
    -> separately implemented replay with a recorded independence vector
```

Keep the generator, its inputs, the generated certificate, the checker, and their digests. Add documented negative mutations. The checker must reject each mutation.

The accepted checker must re-derive the finite predicate from the certificate’s primitive fields; it must not trust a producer-written verdict, copied score, or shared hidden state. State every applicable item among the exact covered domain, partitions and overlaps, tail/remainder theorem, outward-rounding mode, precision, strict margin, aggregate error budget, and unsupported region; record a reasoned typed `not_applicable` for the rest. An exact integer or rational-equality certificate must not invent an interval partition, rounding mode, or positive margin it does not use. Check running coverage as well as final counts when coverage is applicable, so a missing terminal region cannot disappear behind a green summary. A second program that imports the

producer's formulas or generated table is useful implementation diversity, not an independent mathematical derivation; record the common cut explicitly.

Exploratory floating-point search may choose a candidate, but its value never enters the accepted record until the frozen certificate route encloses it. If a feasibility predicate, caveat, or coverage condition is later lost or contradicted, downgrade the result immediately, retain the withdrawn value, and require a new claim/attempt revision rather than re-labelling the old run.

The certificate boundary is fail-closed. Every printed prerequisite must feed the exit status, and final success text is emitted only after their conjunction is true. Exact rational endpoints or outward enclosures define every universal domain; control-flow floats may not shrink it. Represent coverage as intervals and mechanically check their union and overlap. Never use Python `assert` as the acceptance mechanism. A skipped heavy stage has a distinct status from a full rebuild. Digest the certificate, primitive inputs, checker, environment lock, source revision, complete log, and every trusted upstream artifact as separate fields rather than one ambiguous bundle hash.

For every certificate family, classify the following mutations by applicability: force each prerequisite false; perturb an endpoint by one representable unit; delete a band; create a one-unit coverage hole; spoof the final verdict string; delete one primitive coefficient or row; mismatch parameter metadata; run optimized Python; reuse a stale heavy-stage artifact; and replace a universal domain with sampled points. Every applicable mutation that actually violates a frozen acceptance predicate must fail. Record a typed `not_applicable` with a reason for mutations whose field or predicate the certificate does not possess. In particular, an endpoint perturbation is a negative test only when it creates a forbidden gap, overlap, or bound violation; a different but still valid enclosure is not required to fail. A same-library downstream replay declares its trusted upstream values and dependency overlap; it is not an independent proof merely because it lives in a second file.

Rational probabilities do not imply rational logarithmic information values. Use a symbolic identity or a certified interval for a proof claim. Label an uncertified high-precision value as a reference. Do not call a decimal reference an exact entropy oracle.

### 3.6.7 6. Run the required 20-core/50-lens adversarial audit

Complete one applicability matrix for every major claim and every live artifact that states or transfers it. Every row receives exactly one disposition: `PASS` with evidence, `CORRECT` with the old and corrected statement, `NARROW` with the retained scope, `OPEN` with an obligation or blocker, or `NEGATIVE` with a retained counterexample. `N/A` is allowed only with a typed reason. If one lens exposes separable sub-obligations with different dispositions, split that lens into typed subrows; never collapse them to the most favorable disposition. A paragraph saying that an audit occurred is not a completed matrix. The first 20 rows below are mandatory. Add at least 30 named artifact-specific rows for a consequential decision; choose them from the actual failure surface, not from a fixed vocabulary solely to reach a count. The durability blueprint gives one complete 70-row worked example: 20 mandatory core lenses plus 50 additional artifact-specific lenses. Its answers do not transfer to a different artifact.

#### Lenses 1–10: scientific object and inference contract

| Lens | Required applicability question |
|---|---|
| 1. Estimand | Is the exact functional, estimator, transformed estimand, or diagnostic named without substitution, and is the order of conditioning, fitting, mixing or averaging, nonlinear transforms, and Möbius inversion fixed? |
| 2. Types | Are domain, codomain, variable roles, data representation, and output type explicit? |
| 3. Quantifiers | Is the full order of existence, universality, limits, uniformity, probability, and conditioning preserved? |
| 4. Mappings | Is every transfer backed by a positive, premise-checked mapping theorem, including one compatible joint version or witness when simultaneity is required; otherwise does the route abstain? |
| 5. Support/reference measure | Are population support, dominating/reference measures, density or RN premises, boundaries, and singular cases declared? |
| 6. Lattice/source count | Is the exact finite antichain poset, order, source count, event semantics, and Möbius direction fixed? |
| 7. Units/gauge | Are logarithm base, units, metric, scale, preprocessing gauge, and conversion limits explicit? |
| 8. Population/empirical | Are population law, empirical PMF, estimator, fixture, and represented output kept distinct? |
| 9. Sampling | Are row law, i.i.d./stationary/ergodic/dependence coefficients, rates, splits, and conditioning stated? |
| 10. Selection/UQ | Are fitting, tuning, reuse, multiplicity, null, coverage, and abstention contracts justified without calibration transfer? |

## Lenses 11–20: evidence, custody, implementation, and release contract

| Lens | Required applicability question |
|---|---|
| 11. Formal correspondence | Does a reviewed prose-to-formal map bind the actual objects and conclusion to a target fixed before candidate access, rather than a candidate-derived or convenient surrogate? |
| 12. Kernel/axioms/toolchain | Are exact versions, complete import closure, trusted kernel, permitted axioms, fresh replay/confinement route, `--trust=0` or equivalent, and unformalized bridges inventoried; does an unavailable verifier fail closed? |
| 13. Route dependency diversity | Is the functional/epistemic/institutional independence vector stated, with shared cut sets and common causes, without treating model/session/role separation as independence? |
| 14. Numerical/binary64 | Are exact-real, high-precision, interval, and binary64 claims separated with bounds for rounding, cancellation, overflow, underflow/subnormals, signed zero, NaN/±Inf propagation, rounding mode and FMA contraction, platform `libm`, and compiler/target variation; and is one total error budget smaller than every strict claimed margin? |
| 15. Compiled/wrapper parity | Is every claimed Rust feature/build path, backend, Python wrapper, serialization, and failure surface checked? |
| 16. Counterexample/mutation | Are boundary falsifiers, malformed fixtures, wrong mappings, target/definition/judge edits, axiom/import expansion, weakened premises, coordinated status reseals, and optimized-mode mutations required to fail closed? |

| Lens | Required applicability question |
|---|---|
| 17. Custody/threat/refusal | For each freeze, are the object, pre-candidate task revision, candidate write scope, judge authority, error threat, enforcement refusal point, same-actor limit, and theorem need recorded? |
| 18. Citations/novelty | Are primary sources version-pinned, exact imported statements and application hypotheses preserved, and the provenance class and no-novelty boundary correct? |
| 19. Ecosystem/authority | Are consumer snapshots, authority direction, capability gaps, acceptance status, and stale derived projections explicit? |
| 20. Resource/platform/release | Are resource ceilings, stopping/selection rules, complete attempt and withdrawal dispositions, cancellation, determinism, OS/architecture/toolchain scope, package identity, release assets, and non-implications closed? |

For every strict sign, ranking, or separation claim, bind an exact or enclosed positive margin and one aggregate error budget covering every applicable approximation, tail, dependence, finite-range, representation, and compilation contribution. Componentwise bounds do not close the claim unless their joint composition is proved. If the aggregate budget can reach the margin, report the result as unresolved rather than selecting the favorable sign or ordering.

An audit must try to find a counterexample, a violated premise, a wrong mapping, or a numerical failure. It must not only restate the proof. Record the challenge, disposition, exact evidence, non-implications, and revision. If the revision changes an assumption, create a new claim-packet revision. Apply this matrix retrospectively: an older green gate or polished paper is not grandfathered.

### SxPID definition-compatibility firewall

The routing problem begins with multi-source PID. Within the finite MGW/shared-exclusions construction used here, for any one-element antichain $\{a\}$ whose member $a$ is a nonempty source-index subset, the cumulative self-redundancy equals local mutual information for the joint source block $S_a$ and averages to $I(S_a;T)$. This is a cumulative-node identity: it neither says that the Möbius atom at $\{a\}$ equals mutual information nor adds another atom to the lattice. This statement is not transferred to every PID proposal. PID is not one uniquely defined functional, and shared exclusions itself has typed constructions that must not be collapsed. MGW is the finite-PMF pointwise construction implemented by the stable categorical code. Schick-Poland proposes an auxiliary-indicator/RCP/RN construction for a finite source family in its stated locally compact Radon/Borel setting. At $P(R) > 0$, the standard conditional probability is $P(T \in A \mid R) = P(T \in A, R)/P(R)$, which supplies the normalization needed for the MGW specialization. The reviewed arXiv v2 display writes only the numerator, so that display by itself does not establish the claimed recovery without a correction or clarified definition. Stable pid-rs code computes MGW and provides no separate general Schick-Poland route. Its reference-measure/disintegration argument additionally invokes Standard Borel and countable-generation/separability machinery whose bridge from the displayed topology/measure clauses requires an explicit theorem or sufficient full-measure reduction, and target-local RN derivatives are only a.e. representatives without a selection rule even on Euclidean spaces. Eventwise RN derivatives need a simultaneous kernel/version theorem, a general source variable has no pointwise inverse, a Borel isomorphism alone does not supply the atom-plus-Lebesgue density representation displayed with Corollary 3.1 (physical PDF page 9; a Cantor law is a singular-continuous diagnostic), and a bimeasurable bijection need not be bicontinuous. The reviewed bicontinuity step is in Section 4.3.3 (physical PDF page 16). For null indicator events, RCP uniqueness is also only almost everywhere, so pid-rs treats these standard-Borel/RN-

representative and indicator-value selection/limit bridges as open rather than universally validated rules. Ehrlich gives a practical, gauge-dependent continuous analytic density/quasi-density route with a source-disjunction kNN neighbourhood estimator inspired by, but not identical to, Schick-Poland and default-off here. The disjunction is not a probability-union law, and its quasi-density need not integrate to one. Ehrlich Definition 1 and the explicit non-normalization discussion are on physical PDF page 5. Appendix B derives a logical-statement quasi-density and includes mixed discrete/continuous examples; Appendix K begins on physical PDF page 27 and separately sketches a mixed-system treatment/estimation ansatz and one symbolic example, but no demonstrated or calibrated mixed estimator. Neither is the continuous PID3 branch-dimension problem, and pid-rs implements no generally calibrated mixed estimator. A theorem, fixture, estimator result, atom sign, or calibration statement from another PID or another construction must not be used as evidence merely because both methods use the words redundancy, unique information, and synergy. Before importing a PID result, bind all of the following:

1. the exact measure and frozen-per-claim-revision definition identity;
2. the target, ordered sources, source collections, and antichain order;
3. the construction-native local or target-outcome-specific information object, any cumulative-event semantics and Möbius convention it actually uses, and a typed `not_applicable` for absent fields; for MGW, bind the informative, misinformative, and signed-net terms explicitly;
4. the probability domain, estimator, preprocessing map, units, and numerical representation;
5. the source theorem's hypotheses, including every positivity or identity axiom; and
6. an explicit mapping theorem stating the property preserved by the transfer.

Shared atom names, the same three mutual-information coordinates, a similar benchmark value, or a common lattice are not a mapping theorem. In particular, Williams–Beer $I_{\min}$, BROJA, CCS, MMI, SID, and other authors' PID definitions are comparison objects unless this mapping is proved.

Two Lyu–Clark–Raviv works require separate theorem records. The published PRE article "Multivariate Partial Information Decomposition: Constructions, Inconsistencies, and Alternative Measures" (reviewed arXiv:2508.05530v2) assumes its stated Axioms 2–4, independent identity, and cross-subsystem reconstruction package. MGW's independent-COPY and signed-XOR values place it outside that premise package, so the published theorem neither refutes MGW nor is refuted by those values. The later "Structural Impossibility of Antichain-Lattice Partial Information Decomposition" (arXiv:2604.03869v2) instead stipulates recoverability-descriptor atoms; its impossibility transfers only to a PID that factors through that descriptor. Sharing an antichain carrier supplies neither premise package nor descriptor factorization.

The Schick-Poland finite-discrete specialization can map to MGW by standard conditioning when the supported event has positive probability and the required `/P(R)` normalization is present; the reviewed arXiv v2 Section 4.3.1 display (physical PDF pages 13–14) omits that factor, so the displayed recovery remains an open source-level correction/clarification obligation. This does not close the general kernel, representative, invariance, or null-indicator obligations. Ehrlich's matched refining-bin calculation is a narrower premise-bound asymptotic motivation from discrete inclusion-exclusion to a density expression; it is not identity of all three constructions, a general convergence proof, or a pid-rs fixed-quantizer convergence theorem. A discrete exact-real theorem still does not validate a continuous estimator or an unquantized estimand. A measure-independent theorem may be imported only after its abstract objects and all hypotheses are instantiated with

the actual typed construction. Record failed or missing mappings as negative/open results and abstain rather than silently transplanting them.

Terminology firewall: "categorical SxPID" means finite/discrete category-valued random variables and a finite PMF. The population functional may use a declared finite population PMF, whereas the pid-rs direct row-data path computes $F(\hat{p}_n)$. That value is a descriptive empirical functional when the empirical law is the target and a plug-in estimator when the target is the population functional $F(p)$; binary64 representation and error remain separate. This usage is unrelated to category theory. Möbius inversion here is on a declared finite antichain poset. Category-theoretic machinery or infinite-poset inversion requires a separate typed theorem and cannot be inferred from the word "categorical."

### Imported-theorem application map

For every external theorem that carries a load-bearing step, retain an application record:

```
Source theorem and version-pinned revision:
Exact source statement:
Named source arrow (domain -> codomain):
Property and direction imported (injective/surjective/isomorphic/zero/exact):
Local variables-to-source variables map:
Required hypotheses:
Evidence for each hypothesis:
Exceptional cases and exclusions:
Uniformity and constant dependence:
Named local arrow (domain -> codomain):
Arrow-level type and direction correspondence:
Source-language ambiguities and resolution evidence:
Conclusion actually imported:
Local obligation closed:
What was checked against the source:
What was not re-proved:
```

Checking that a theorem exists is not checking its application. Checking the application is not re-proving the source theorem. State both boundaries. In particular, audit whether an implied constant is uniform over the changing family used locally, whether an exception clause is exhaustive, and whether a transformed object remains in the source theorem's domain.

### Citation-edge type check

An imported statement containing more than one morphism, an exact sequence, a commutative diagram, or an anaphor such as "which," "it," or "this map" cannot close an obligation until the following gate passes:

1. Give every source morphism a stable local identifier and a typed signature, for example $f_s : A_s \to B_s$ and $g_s : B_s \to C_s$.
2. Bind every imported predicate to one named arrow and direction: for example `Mono(f_s)`, `Epi(g_s)`, `Iso(g_s)`, or `Zero(g_s)`. A free-floating "is an isomorphism" fails.
3. Name the local arrow and show the domain, codomain, variance, indexing, and direction match the source arrow under the recorded variable map.
4. Resolve ambiguous source prose from the upstream proof, adjacent lemmas, errata, or a specialist source check. If competing readings remain, retain both and mark the obligation `BLOCKED`; do not choose the reading that makes the local proof work.

5. Re-derive the local implication from the typed predicate without copying the source prose. For an exact sequence, explicitly use the appropriate kernel/image consequence rather than transferring injectivity, surjectivity, or isomorphism to an adjacent arrow.
6. Run an adjacent-arrow mutation: attach the predicate to each neighboring arrow in turn. The typed correspondence or a retained countermodel must reject every wrong attachment. Keep the displayed `Z/2` exact-sequence witness above as the minimal human-readable regression for the failure exposed above.
7. Separate source retrieval from consequence checking. A model that retrieves and then audits the same ambiguous sentence is not an independent route. Include a source-blind derivation of the local consequence and a separate primary-source/proof inspection for the imported premise.
8. Bind the same arrow identifier, signature, and predicate to any Lean theorem, executable schema, or certificate field. Treat a deep unformalized source theorem as an explicit typed premise; do not imply that an axiom or interface theorem verifies its truth.

Record `CLOSED` only when all applicable fields pass. `OPEN` means evidence remains to be supplied; `BLOCKED` means the source correspondence is ambiguous or false. The PDF checker retains the template, this gate, and its negative control in the human-readable artifact. That artifact gate does not inspect every future proof automatically; each claim packet must instantiate and review the record.

The deterministic `check-citation-edge-countermodel.py` gate exhausts the finite $C_2$ witness's map tables, images, kernels, and isomorphism predicates. The companion mutation self-test script, `check-citation-edge-countermodel-self-test.py`, requires rejection of wrong-arrow binding, witness collapse, broken exactness, stale Markdown/LaTeX parity, and removal of the typed source-arrow field. Both run in the mathematical-workflow PDF gate, its `just` route, and CI. They reject the local inference schema; they do not interpret motivic homotopy or verify a PID theorem.

The orthogonal implementation check `PidCitationEdgeCountermodel.lean` states the same $C_2$ witness with Mathlib additive homomorphisms and proves the three image/kernel exactness equalities, right-arrow bijectivity and surjectivity, adjacent-arrow non-bijectivity and non-surjectivity, and middle-group nontriviality. Its pinned checker audits nine theorem declarations and their axiom inventories; its mutation gate rejects collapse of $C_2$, replacement of the identity by zero, erasure of the kernel from exactness, and both false adjacent-arrow conclusions. This is useful implementation and kernel diversification against a defect in the Python checker's custom finite-group semantics. It remains the same mathematical countermodel and the same logical route, not an independent counterexample. It does not formalize motivic homotopy, validate either cited source theorem, establish the source-to-Lean arrow correspondence, or prove anything about PID.

### Cheap invariant probes

Before a long proof or expensive certificate run, derive low-cost invariants that every candidate must satisfy. Use them as early falsifiers and retain the smallest failure. Examples include:

- an integer-valued quantity must remain integral;
- a mathematical probability vector in an exact-real or rational representation must have non-negative entries summing exactly to one; separately rounded binary64 entries instead need a proved rounding enclosure or a construction that enforces the represented normalization;

- a Möbius transform and its zeta inverse must reconstruct every coordinate;
- a symmetry claim must commute with the declared source permutation;
- an outward interval for a partial sum plus a proved two-sided remainder enclosure must combine into an enclosure of the total; in the special one-sided case, first prove $0 \le r \le R$ and require $L \le s$ and $U \ge s + R$ for exact partial sum $s$;
- a claimed covariance or concentration proxy must have the required sign and limiting behavior;
- a resource estimate must dominate the exact serialized object it is intended to bound.

An invariant can refute a route. Passing it does not prove the route. Whenever an invariant catches an error, add a durable correction-ledger entry and retain a negative control when privacy, licensing, security, and custody permit. Otherwise record the omitted fixture and its exact reason; never claim that an unavailable control was preserved.

### 3.6.8 7. Use a blind holdout protocol

A proof and a holdout test answer different questions. A proof can establish a theorem under stated assumptions. A holdout test can estimate the behavior of an estimator or implementation on a defined benchmark distribution. Do not use one as a substitute for the other.

Call a benchmark blind only when the development and implementation roles cannot access the holdout rows, holdout seeds, target values, or unredacted adjudication output before the frozen analysis produces the complete first result table. Preregister every reported output. Record an access matrix for all roles and assets. Record who holds each sealed artifact and, as applicable, its binding-and-hiding commitment or encrypted-package digest. Never treat a raw digest of an enumerable secret as hiding. If these conditions do not hold, call the design a controlled holdout, not a blind holdout.

For theorem or proof benchmarks, the inaccessible set also includes reference solutions, earlier attempts, transcripts, scratch directories, archives, judge reports, and unredacted feedback. Moving or renaming an archive is not isolation: a shell-capable candidate may enumerate the whole filesystem. Use a fresh ephemeral worktree or container per attempt, read-only exact dependencies, an allowlisted filesystem, and disabled network unless access is declared and allowlisted. Record network interactions to a stated capture boundary and deny strict-isolation credit for unknown or omitted interactions. Best-of-$N$ trials are not independent when they share artifacts, feedback, caches, candidate-visible state, or an unsealed filesystem; state the actual dependency vector.

Use this protocol for estimator and pipeline claims:

1. Freeze the estimand, data-generating families, parameter grid, sample sizes, metrics, tolerances, failure rules, and analysis code.
2. Seal the generator, seed list, and targets. Before holdout access, publish a binding-and-hiding commitment anchored by third-party time evidence or a separately controlled adjudicator record. For low-entropy secrets, use a fresh high-entropy nonce retained under separately controlled custody or bind an encrypted sealed package. Record the applicable institutional/custody independence dimensions. A raw digest of an enumerable seed or target is not hiding.
3. Define the development, implementation, execution, and adjudication roles. Record all role overlaps and the access matrix.

4. Separate development, calibration, and holdout data. Do not tune on the holdout data.
5. Keep the holdout rows, seeds, target values, and unredacted result output unavailable to the development and implementation roles until the adjudicator records the complete first result table.
6. Include positive controls, negative controls, boundary cases, and known failure regimes.
7. Use repeated holdout draws for sampling claims. State confidence limits and preregistered acceptance criteria.
8. Report every planned cell. Do not remove a cell after a poor result.
9. Report effect errors, interval coverage, abstentions, failures, and resource limits.
10. Apply the stated multiplicity control when the decision uses many cells or hypotheses.
11. Record the first holdout result before a revision. A revision needs a new holdout version.
12. Publish the generator, manifest digest, adjudicator, and full result table after the blind phase.

The benchmark must define its scope. Synthetic Gaussian performance does not prove performance for mixed, singular, quantized, or dependent laws. A fixed benchmark does not prove a general theorem.

### Minimum blind-benchmark commitment

The pre-access commitment must contain enough information to detect a change after result access. Record these fields:

| Field | Required content |
|---|---|
| Benchmark ID | Stable versioned identifier |
| Claim IDs | Scoped empirical or calibration claims that the benchmark can accept or reject; never an unspecified universal or exact theorem |
| Analysis-plan digest | Digest of the metrics, tolerances, confidence limits, multiplicity rule, and acceptance rule |
| Code identity | Version-pinned source identity and environment or package-lock identity |
| Generator identity | Generator digest, parameter-grid digest, and sampling-law version |
| Sealed inputs | Binding-and-hiding commitment to the seed list or target bundle, or digest of an encrypted sealed package; record nonce/key custody separately |
| Role separation | People or systems that develop, implement, execute, and adjudicate, including all role overlaps |
| Access matrix | Permitted access for each role to code, rows, seeds, targets, keys, and result output |
| Commitment custody | Party that holds each sealed artifact, opening material, and binding-and-hiding commitment or encrypted-package digest, as applicable |
| Access rule | Event that permits target access and the party that records that event |
| External or separately controlled time evidence | Third-party timestamp or a separately controlled adjudicator record, with the applicable institutional/custody independence dimensions |

| Field | Required content |
|---|---|
| Failure rule | Treatment of crashes, non-finite values, abstentions, missing cells, and resource exhaustion |
| Result identity | Digest of the complete first result table before any revision |
| Deviations | Every difference from the frozen plan |

A local Git commit time alone is not external or separately controlled time evidence. Do not store a secret seed, target, decryption key, or credential in the repository. The adjudicator must retain failed cells and must run the frozen analysis without manual result-dependent changes. If a confidentiality constraint prevents public release of a sealed input, publish a binding-and-hiding commitment and state who controls the sealed bytes and opening material.

This protocol reduces result-dependent tuning and selective reporting. It does not make a benchmark distribution representative, prove that an analytic target is correct, or replace a theorem.

### 3.6.9 8. Maintain a claim-to-evidence matrix

Use the matrix to prevent evidence substitution.

| Claim class | Evidence that can support it | Evidence that does not complete it |
|---|---|---|
| PID definition and provenance | Defining paper, exact repository definition, and provenance marker | Similar name or similar output |
| Möbius reconstruction identity | Symbolic derivation and machine-checked finite algebra | Random numerical examples |
| Population functional theorem | Proof with explicit assumptions and counterexample audit | Passing implementation tests |
| Finite-alphabet consistency | Probability proof, stated mode of convergence, and formalized subclaims | One Monte Carlo curve |
| Numerical reference value | Symbolic value, rigorous interval, or high-precision value with error limit | Default floating-point agreement |
| Rust implementation conformance | Separately specified oracle with a recorded independence vector, mutation test, feature-path parity, and boundary tests | A paper citation |
| Scoped estimator calibration | Declared data-generating family, repeated preregistered holdout draws, confidence limits, and acceptance criteria | Reused development fixtures or one unqualified draw |
| Dependence-aware inference | Valid dependence assumptions plus block or permutation design checks | An independent-row test |
| Downstream suitability | Consumer-specific input contract and acceptance fixture | A generic PID example |
| Limitation or impossibility | Explicit counterexample or proved obstruction | Failure to find a proof |

Each new or revised scientific claim must identify its applicable claim class or classes, evidence, and scope. A result can use more than one claim class. For example, a correct functional identity does not establish estimator calibration.

## 3.7 Application to shared-exclusions PID

The primary theoretical target in this project is the shared-exclusions PID line. Keep paper-defined quantities separate from project-defined diagnostics, wrappers, and workflows. `method-catalog.json` and its generated `METHODS.md` rendering are the authorities for method origin, code availability, and validation status. Their correlated checker enforces declared structure and route-specific semantic markers; neither the catalog nor checker is independent evidence that those scientific statements are true. The primary literature, typed mapping arguments, source/code correspondence, and counterexample audit remain separate obligations.

### 3.7.1 Exact claim packet for a new PID theorem

The packet must define:

- the ordered source variables and target;
- the construction identity: MGW finite-PMF, Schick-Poland general measure-theoretic, Ehrlich practical continuous, Williams–Beer $I_{\min}$, or another explicitly named functional;
- the nonempty source-index subsets, admissible antichains, and declared redundancy order;
- the construction-native local or target-outcome-specific information object and decomposition convention; for MGW, the pointwise informative, misinformative, and signed-net terms; where a construction has no such split, a typed `not_applicable` plus its native object;
- the averaging or conditioning law, or a typed `not_applicable` when the construction has neither;
- the logarithm base and units;
- the population support and any minimum-mass condition;
- the domain/codomain, sigma-algebras, dominating/reference measures, densities or RN derivatives, and measurement/preprocessing gauge;
- the row dependence model;
- the estimator and all fitted preprocessing;
- the requested conclusion and its convergence mode;
- the status of negative atoms;
- all cases where the method must abstain.

If the theorem concerns a plug-in estimator, distinguish the population functional from the empirical law and the implementation. If it concerns continuous data, state the support model. Do not infer full-dimensional absolute continuity from unique observed rows.

### 3.7.2 PID routing checkpoints

- Route by declared population law, source/target roles, source count, scientific goal, and gauge; never by storage dtype, observed ties/no ties, feature availability, or failure of another route.

- `SupportContract::AssumeRegularFullDimensional` is a conservative project-defined fail-closed runnable gate, not a verbatim statement of every cited paper's analytic domain. It can reject paper examples with singular joint source support; rejection narrows code availability rather than refuting the paper functional.
- Standard KSG and continuous shared-exclusions estimator interpretation uses i.i.d. rows under its density/geometry premises. Eligibility requires unrounded rows from one fixed joint law, full-dimensional absolute continuity of every required marginal and joint law, finite mutual information, and declared boundary, smoothness/density, ambient-coordinate metric, and local-geometry conditions. A support declaration and observed uniqueness do not prove these premises. Subsampling or dependence-aware UQ does not itself prove estimator consistency for dependent rows; a separate theorem must name dependence coefficients and rates.
- KSG estimates mutual information using joint-neighborhood radii and marginal counts. Ehrlich redundancy uses a source-disjunction quasi-density/neighborhood construction, not a probability union law; the quasi-density need not integrate to one. Reuse of KSG-style neighbor counting as an implementation basis does not identify mutual information with Ehrlich shared-exclusions redundancy. Kraskov et al.'s higher-dimensional "redundancy" is multi-information/total correlation, not PID redundancy.
- O-information is a target-free Shannon scalar, not a PID atom or redundancy selector. With exactly two variables its formula is identically zero; redundancy- versus synergy-dominated sign interpretation begins only at three variables.
- The paper-defined target-conditioned average degrees $\bar{r}$ and $\bar{v}$ and pid-rs' project-defined target-free $\mathrm{Red}^\circ/\mathrm{Vul}^\circ$ ratios are different Shannon diagnostics. For $\bar{r}/\bar{v}$, bind one coherent target, law/sample, preprocessing, units, and positive denominator before forming the ratio. The target-free ratios are analogues for screening, not the published target-conditioned quantities. Neither family is a PID decomposition, atom, or redundancy selector, and no result transfers between them without a mapping theorem.
- `experimental::isx_heuristics` contains project-defined, formula-labelled comparison baselines. They do not estimate Ehrlich et al.'s paper-defined continuous shared-exclusions functional. Sharing KSG terms, an API shape, a wrapper alias, or a numerical coincidence does not supply a mapping.
- If a fitted quantizer artifact $\mathcal{Q}$ is random, distinguish the functional $F(P_q)$ conditional on one realized frozen artifact $q$, the artifact-averaged functional $E_{\mathcal{Q}}[F(P_{\mathcal{Q}})]$, and the artifact-marginalized-mixture functional $F(E_{\mathcal{Q}}[P_{\mathcal{Q}}])$. The second averages functional values; the third applies the functional to a mixture law. Nonlinearity supplies no equality. Stable fitted calls condition on the realized artifact and do not integrate over artifact randomness; hashes cannot prove row identity, observation nonoverlap, or fit/evaluation roles.
- Do not identify every equal-width implementation with the stable fitted-edge quantizer. pid-rs' experimental same-sample route computes each column's range on the same rows passed to the categorical estimator and selects bins by exact integer scaling of the computed binary64 fraction's significand; it materializes no edge vector. At the ordinary boundary $[0, 1]$ with ten bins, binary64 `0.3` maps to bin 2 by that rule but to bin 3 when tested against the stable route's rounded edge vector. The two routes therefore define different represented categorical transforms at some boundaries. Bind the exact transform identity as well as `num_bins`; a shared "equal-width" label is not a mapping theorem.
- Sign rules are method-specific. Williams–Beer $I_{\min}$ has nonnegative exact-real finite-PMF atoms; a negative pid-rs output whose magnitude exceeds a separately justified numerical

error enclosure is a defect. Binary64 arithmetic alone supplies no universal tolerance. MGW informative and misinformative component atoms are separately nonnegative in the exact-real theorem, while their signed nets may be negative. Ehrlich continuous raw PID atom estimates may be signed. Population mutual information is nonnegative, but a finite-sample raw KSG estimate may be negative and has its own scalar `NegativeHandling` policy. Componentwise clamping of a negative estimated input or atom inside a PID composition can break raw identities unless the other atoms are changed. Ehrlich's suggested atom rebalancing can retain consistency equations (physical PDF page 18), but it is a transformed decomposition rather than the raw atoms. No pid-rs continuous-PID API implements that atom rebalancing; KSG scalar clamping is a separate MI reporting policy, not a PID transform.

- "Mixed-dimensional PID3" means continuous antichain branches with different source-block dimensions, not mixed discrete/continuous support. Ehrlich Appendix B's logical-statement quasi-density and Appendix K's research-sketch mixed-system estimation ansatz do not supply a generally calibrated pid-rs mixed estimator.
- Barà et al.'s discrete-target/continuous-source route uses Williams–Beer minimum-specific-information, not shared exclusions. Reversing target/source support roles or sharing natural-log units does not preserve the estimator.
- A fitted quantizer or PCA/PLS projection defines transformed variables. For a claim conditional on a frozen artifact, a held-out evaluation, or population inference under a fixed transform, train using only declared disjoint fit information, freeze and bind the artifact to evaluation, and type which variables informed fitting. Target-supervised fitting on disjoint training targets is permitted when declared. An explicitly same-sample adapter instead defines a data-dependent transform and downstream empirical estimator. Under the current pid-rs evidence and status, it is exposed only for its declared exploratory or descriptive target; it does not inherit that fixed-transform or held-out theorem, and population inference would require a separate theorem for the joint fitting-and-evaluation procedure. Evaluation-target or same-row confirmatory-target fitting is leakage relative to a held-out claim. No general theorem preserves PID atoms through PCA/PLS or makes fixed bins converge to the unquantized continuous functional.

### 3.7.3 Complementary PID audit families

For each new theorem, use at least five genuinely failure-diverse applicable audit families. A shared oracle, imported lemma, generated table, implementation, or formalization seam counts once at its common cut even when several suite names exercise it. If a listed family is inapplicable, record a typed reason; inapplicability is not replacement evidence and does not reduce the required 20-row baseline plus at least 30 named object-specific lenses for a major claim.

- **Combinatorial:** antichain order, Möbius inversion, atom reconstruction, and source symmetry.
- **Analytic:** continuity, support boundaries, limiting arguments, and perturbation bounds.
- **Probabilistic:** concentration, dependence colorings, mixing assumptions, and resampling.
- **Computational:** exhaustive small alphabets, exact count tables, and counterexample search.
- **Formal:** Lean or SMT statements for finite algebra and deterministic inequalities.

- **Statistical:** coverage, power, null calibration, selection effects, and blind holdout behavior.

These families can support each other. They are not interchangeable. A formal lattice identity does not prove a concentration theorem. A concentration theorem does not prove that a continuous kNN estimator targets the intended functional.

### 3.7.4 Required counterexample search

For each new PID claim, test the relevant cases:

- independent sources and target;
- copied sources;
- XOR and other synergy gates;
- duplicate or deterministic sources;
- zero-probability and near-zero-probability cells;
- support deletion and support creation;
- singular and mixed-dimensional continuous laws;
- exact ties and quantized observations;
- row dependence and a false dependence partition;
- cancellation in reconstructed atoms;
- source permutation and bijective target-state relabeling; separately, noninjective target coarse-graining as an estimand-changing map;
- admissible state/event identifications and incidental overlaps, including collapsed source or target categories and duplicate coordinates;
- serial and parallel implementation paths.

Keep a counterexample when it breaks a proposed unconditional claim. State the corrected assumption next to the retained example.

### 3.7.5 Formal and software evidence

Formalize the exact theorem statement before large certificate generation. Retain a reviewed prose-to-formal correspondence table for every domain, codomain, quantifier, order relation, event, premise, and conclusion. Record the exact proof-assistant/kernel/library toolchain, invoke `--trust=0` or the closest available minimal-trust audit, inventory every theorem's axioms, and name all unformalized bridges. Kernel acceptance establishes that the exact recorded checker/toolchain accepted the encoded term. Deductive proof credit is conditional on the encoded calculus and the checker/kernel implementation being sound and on no applicable soundness exploit. Record and review known soundness advisories against the exact version, including whether a rejected version must be replayed on a fixed release. Even then, acceptance does not prove that the encoding is the intended prose/scientific object.

For a Möbius claim, formalize the declared finite antichain carrier and its actual partial order. Do not substitute an untyped matrix, category-theoretic story, or infinite-poset inversion without

a separate correspondence theorem and, where applicable, local-finiteness/convergence premises. For finite domains, use exact enumeration or certified intervals when possible. Then replay the same cases through the Rust API and any claimed Python wrapper.

Run checkers under normal Python and Python with optimization enabled (the `-O` option); assertions must not be the acceptance mechanism. Require mutation tests for proof and evidence scripts. Malformed fixtures, wrong theorem bindings, weakened premises, altered source counts/orders, missing branches, and false conclusions must fail closed. Where two solvers or implementations are claimed, state their functional/epistemic/institutional independence vector and ensure a mechanistically separate checker recomputes the obligation rather than merely parsing the producer's claimed answer. A different model lineage may improve epistemic diversity but is not institutional independence; institutional custody requires separately controlled authority, access, and refusal capability. Run every API, feature, backend, compiler mode, wrapper, and build mode named by the claim. None of these checks proves generic proof-kernel soundness, compiler correctness, or platform-independent transcendental arithmetic.

For a major formal result, prefer a comparator-style trust split when the proof assistant and theorem shape permit it:

1. keep a small trusted package containing definitions, hypotheses, and theorem statements but no solution implementation;
2. keep the proof terms or solution library in a separately classified untrusted package;
3. compare elaborated theorem statements exactly, including implicit arguments, universes, type classes, and binder order;
4. compile and replay the solution with the primary kernel at the pinned toolchain;
5. inventory every axiom and proof escape against an exact allowlist;
6. where a mechanistically distinct checker exists, replay exported proof objects with it and record its own parser, calculus, platform, resource, and completeness assumptions; and
7. retain a separate reviewed prose-to-statement map and implementation-refinement map.

Formal acceptance is conjunctive. Bind the task revision, pre-candidate separately reviewed target bytes with the review dependency vector recorded, theorem fully qualified name and elaborated signature, candidate bytes, complete import closure, Lake/toolchain/kernel/checker/wrapper identities, permitted and observed transitive axioms, environment and sandbox policy, command, exit status, full standard streams, structured report, and before/after artifact identities. A nonzero exit, missing/skipped verifier, unavailable binary, malformed report, timeout, crash, target mismatch, forbidden axiom, or artifact change means `unverified`; no regex or pretty-printer sub-diagnostic can override a checker-wide failure. Atomically associate the verdict with the exact bytes checked and retain the raw report.

At minimum, hostile cases include local `abbrev`/`def` shadows of a trusted predicate; weakened, renamed, deleted, empty, or wrong-FQN targets; direct/helper/imported `sorry`/`admit`; new axioms, opaque/unsafe/partial paths; a forbidden axiom combined with an alpha-equivalent type diagnostic; ambient `LEAN_PATH`/`LEAN_SRC_PATH`/`LEAN_SYSROOT`/`ELAN_TOOLCHAIN`; replacement modules and stale `.olean` files; missing/skipped/crashed/timed-out verification; structured-report deletion; and a file change between checking and verdict recording. A positive canonical baseline must pass, and every mutation must reject under normal and applicable optimized execution.

Comparator credit is conditional on its own preconditions. Bind the complete trusted Challenge import closure and Lake files; prove that no earlier adversarial Solution build contaminated the trusted environment; pin a compatible `lean4export`, primary kernel, and second kernel; isolate the unprivileged build/export; and record cache provenance, operating system, hardware, resource limits, and the exact sandbox controls. Where comparator uses `landrun`, systemd mitigation, or another confinement mechanism, its version, policy, availability, bypass surface, and failure mode are part of the trusted computing base. A missing or stale sandbox does not become safe because statement comparison later succeeds.

Record formal assurance as an orthogonal status ledger, not a single linear maturity score:

| Status dimension | Allowed values and required receipt |
|---|---|
| Trusted formal objects/definitions | `unreviewed`, `reviewed`, or `rejected`; bind exact source and object map |
| Challenge/Solution statement equality | `not-run`, `passed`, or `failed`; bind elaborated signatures, comparator, and environment |
| Proof elaboration | `not-run`, `passed`, or `failed`; bind compiler/toolchain and exact proof source |
| Primary-kernel replay | `not-run`, `passed`, or `failed`; bind kernel and exported/elaborated object |
| Axiom inventory | `open`, `accepted`, or `rejected`; bind complete inventory and allowlist |
| Second-kernel replay | `not-run`, `N/A`, `passed`, or `failed`; bind exporter, parser, calculus, and kernel when run |
| Paper/scientific-to-formal correspondence | `unreviewed`, `reviewed`, `blocked`, or `rejected`; bind the premise/quantifier/object map |
| Formal-to-implementation refinement | `unreviewed`, `reviewed`, `blocked`, `rejected`, or `N/A`; bind code paths and representation map |

No dimension implies another. A theorem can have a primary-kernel pass while its paper correspondence is blocked; an optional second kernel can remain `not-run` without preventing a separate correspondence review. Record release tag, tag object, commit, toolchain, library graph, local/hosted execution state, and whether each receipt is self-reported or held under separately controlled custody with the applicable independence dimension recorded. A repository audit file is not a hosted execution receipt.

Process artifacts also need typed provenance. Distinguish raw event logs, exact visible dialogue, editorially selected dialogue, summarized tool calls, reconstructed narratives, and hidden or unavailable reasoning. Record redactions and missing launches. Count agents, ideas, retained routes, commands, and executed jobs as different quantities. Attribute an idea at the event and artifact level when several agents repair or simplify one route; a polished retrospective label must not silently overwrite that lineage.

### 3.7.6 Statistical and ecosystem evidence

Define separate benchmark strata for categorical MGW SxPID, fitted quantized SxPID, discrete $I_{\min}$, continuous KSG mutual information, Ehrlich continuous redundancy, continuous PID2, incomplete PID3 availability diagnostics, research mixed-dimension PID3 coordinates/branches, project-defined continuous heuristics, Shannon invariant families, uncertainty procedures, and Rust/Python wrapper parity. A composed report belongs to a further stratum that names every input estimand and mapping; it does not merge their evidence. Each stratum needs its own analytic target or certified numerical reference. If a stratum has neither, label it diagnostic. Each stratum also needs its own tolerance and abstention policy. Each mapping or routing benchmark must include at least one neighboring non-target method or estimand that it must reject, so numerical coincidence cannot serve as a mapping theorem.

For Prisoma, Haldir, Galadriel, and Crebain, record a consumer contract. The contract must identify the required estimand, data support, dependence model, scale, uncertainty output, resource limit, and failure behavior. Do not claim consumer readiness until a versioned acceptance suite checks every contract field. One fixture supports only its declared case. Do not infer consumer readiness from general unit tests.

### 3.8 Full semantic closure

A model, solver, proof assistant, generator, or test must check the complete object in the claim packet. For a PID claim, this can include:

- every nonempty antichain for the declared source count;
- every event union and target intersection in the definition;
- every supported realization;
- every source permutation named by a symmetry claim;
- every empirical count table in a declared exhaustive domain;
- informative, misinformative, net, tie, and abstention branches;
- every API, feature, serial or parallel path, and specialized or general path named by the claim;
- every fitted transform and split named by a consumer contract.

If complete enumeration is impossible, supply a proved reduction or a complete separation oracle that terminates on the declared domain and returns either a violator or a checkable no-violation certificate. A semidecision search that may run forever in the no-violator case remains a falsifier search. Random examples do not establish a universal equivalence statement. State the exact domain and resource bound of every finite search or oracle call.

### 3.9 PID exceptional-case checklist

Boundary analysis is a separate obligation. For each applicable item, state whether the theorem extends, changes statement, or requires abstention:

- zero or near-zero supported mass;
- support deletion or creation;
- an event denominator that approaches zero;
- exact or near `I_min` ties;
- informative and misinformative cancellation or an atom near zero;
- duplicate, copied, deterministic, or constant variables;
- an invalid dependence coloring;
- a transform fitted on evaluation rows;
- drift, adaptive selection, or repeated use;

- a zero KSG radius or nonunique neighbor shell;
- unequal intrinsic dimensions or incompatible reference measures;
- a mixed-dimensional or singular law;
- integer, allocation, recursion, and cancellation boundaries;
- platform-dependent transcendental arithmetic.

A derivation that divides away a boundary must keep a separate obligation for that boundary.

Determinism requires typed support analysis. For Euclidean (hence standard-Borel) variables, if $Y = f(X)$ for Borel-measurable $f$ and $P_Y$ is non-atomic, the measurable graph is $(P_X \otimes P_Y)$-null but has joint probability one, so KL mutual information is infinite. The blanket statement "deterministic maps have infinite MI" is false: a thresholded Bernoulli output can have finite $I(X;Y) = H(Y)$, and a constant output has zero MI. Those atomic-output examples still do not satisfy an ordinary full-dimensional continuous KSG contract.

## 3.10 Layered assurance and go/no-go gates

Use each applicable layer. No layer substitutes for another.

| Gate | Required closure | Claim disposition while open | Prohibited wording while open |
|---|---|---|---|
| G0 Claim identity | Frozen packet, sources, non-solutions | `blocked` | Claim scope is final |
| G1 Conventions and premises | Convention and assumption map | `blocked` | Premises are discharged |
| G2 Mathematical core | Proof or counterexample and boundary audit | `active` or `blocked` | The theorem or disproof is complete |
| G3 Formal semantics | Actual objects and implication in a proof checker | `active` or `blocked` | Formally verified |
| G4 Certified numerics | Rigorous enclosure and unresolved semantics | `active` or `blocked` | Certified sign or tie |
| G5 Executable conformance | Refinement or bounded complete equivalence | `active` or `blocked` | Verified implementation |
| G6 Estimator calibration | Preregistered scoped calibration | `active` or `blocked` | Calibrated beyond the tested scope |
| G7 Consumer qualification | Versioned contract and acceptance suite | `blocked` | Consumer-ready or authority-grade |
| G8 Release archive | Reproducible builds, hashes, and first-result record | `blocked` | Final release assurance |

An inapplicable gate needs a written reason. A release statement must name the closed and open layers. `falsified` is terminal only after an accepted counterexample closes a negative result; it is not an "open" G2 state.

For a major claim, keep a revision-preserving directory:

```
claims/<CLAIM-ID>/
  claim-v1.md
  conventions.md
  obligations.md
  routes.md
  failures/
  certificates/
  formal/
  implementation/
  benchmark/
  audit.md
  evidence-matrix.md
  decision.md
```

Do not add empty placeholders. Create a file when it has evidence or a required open obligation. Do not overwrite old claim revisions, failed routes, certificate inputs, or first-result records.

### 3.11 Acceptance rule

Maintain an append-only accepted-result evidence list and a separate claim disposition. Each record binds exactly one evidence class plus its scope, assumptions, exact artifact identities, adjudication, and invalidation/supersession state. Render the empty list literally as “no accepted evidence”; later records never erase contradictory or invalidated records. Allowed classes are:

- theorem proved under stated assumptions;
- machine-checked finite obligation;
- certified numerical bound;
- preregistered empirical result;
- counterexample;
- diagnostic observation.

The disposition is exactly one of proposed, active, blocked, falsified, or complete. `complete` requires every applicable obligation to be closed; closing a finite or machine-checked sub-obligation does not close its parent. A counterexample may complete a disproof, never the original proof claim.

Contradictory accepted-looking evidence forces `blocked`. Retain both artifacts, identify the smallest disputed obligation, and resolve it before either route can close the claim.

Never mutate a record or change a disposition without new evidence. Append an invalidation or supersession event and link every artifact to the claim ID. This ledger exposes gaps; it does not prove the claim.